

电子科技大学

UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

硕士学位论文

MASTER THESIS



论文题目 基于 MongoDB 的多租户消息推送系统的
设计与实现

学科专业 软件工程

学号 201321220113

作者姓名 吴亚骏

指导教师 唐雪飞 副教授

分类号_____密级_____

UDC ^{注 1}_____

学 位 论 文

基于 MongoDB 的多租户消息推送系统的设计与实现

(题名和副题名)

吴亚骏

(作者姓名)

指导教师

唐雪飞

副教授

电子科技大学

成 都

(姓名、职称、单位名称)

申请学位级别 硕士 学科专业 软件工程

提交论文日期 2016.3.18 论文答辩日期 2016.4.18

学位授予单位和日期 电子科技大学 2016 年 6 月

答辩委员会主席_____

评阅人_____

注 1：注明《国际十进分类法 UDC》的类号。

Design and Implementation of Multi-tenancy Push System Based on MongoDB

A Master Thesis Submitted to
University of Electronic Science and Technology of China

Major: Software Engineering

Author: Yajun Wu

Supervisor: Xuefei Tang

School: School of Information and Software Engineering

摘 要

随着基础网络的不断发展,智能手机普及率的不断提高,移动互联网正在逐步成为我们获取信息的主要方式。而推送系统作为传输信息的“发动机”,也应该与移动互联网的理念相适应,应该更容易获得并具有普遍性。但是对于中小企业来说,要实现一个自己的推送系统,其开发、管理和维护的成本是比较高的,但是消息推送又是现在移动应用的必备功能,因此有了本论文的研究内容。

本论文设计并实现的基于 MongoDB 的多租户消息推送系统是在“移动+云”这一趋势下孕育而生的,是成都康赛公司普通教育云平台的一个衍生项目。它作为一个多租户系统,不仅可以向租户提供多平台的定制化推送服务,在此基础上,还为普教云租户提供了 Web 推送管理,向第三方提供了简单的推送接口。

首先,本文会对选题的背景、来源、研究意义做介绍,并分析国内外研究现状,以及对相关理论基础与涉及的关键技术进行简要介绍,在此基础上,通过自己的使用、分析对一些现有的推送服务进行了很客观、全面的总结,并对其中的关键技术进行深入研究,为本文的研究、设计与实现工作做好铺垫。

然后,会详细介绍 MongoDB 多租户数据层研究与设计实现工作,通过对关系型数据库存在的问题进行分析,给出为什么使用 MongoDB 做为多租户系统数据层的原因。接着从多租户四项设计特性出发,包括数据隔离性、定制化、可扩展性、数据安全,会在 MongoDB 上对四项特性进行研究设计。然后根据设计特性与需求分析进行系统数据库的设计,并通过实验与传统关系型数据库进行对比,说明 MongoDB 的定制化优势、时间性能优势,还会介绍本人对 MongoDB 的相关研究工作及成果。

最后,会详细介绍多租户推送系统应用层的设计实现工作,会对系统做整体设计,然后会对系统中各个模块进行详细的设计与实现,包括 Android 平台客户端、服务端,IOS 平台客户端、服务端,短信推送模块、Web 推送管理模块、第三方推送服务调用模块,会对系统测试结果进行简要介绍。

本论文创新点有四个:使用 MongoDB 作为多租户系统的数据层;将推送功能做成一种基础服务,不仅向租户提供统一的、透明的方法进行消息推送,而且会向租户提供多种实用功能;引入读写压力机制对 MongoDB 的 Auto Sharding 机制进行优化;提出很实用的 MongoDB 分页读取数据的优化方法。

关键词: MongoDB, MongoDB 优化, 推送系统, 多租户

ABSTRACT

With continuous development of the network infrastructure and increasing prevalence of smartphones, The Mobile Internet is gradually becoming our primary way to access information. The Push System, as the engine of transmission of information, should evolve to be in line with the concept of Mobile Internet and be more accessible and prevalent. For many small and medium-sized enterprises that want to establish their own Push System, the cost for development, management and maintenance is fairly high. Message push has become a must-have function for mobile applications, and this subject has been studied. The Multi-tenancy Push System Based on MongoDB is born in the “mobile+cloud”, which is a derivative project originated from General Education Cloud Platform. It, as a Multi-tenancy system, can not only provide customized and Multi-platform push service for customers, but also provide tenants of General Education Cloud Platform with Push Management on Web, serving as a simple and convenient interface to a third-party.

Firstly, this article introduces the background, the origin, the significance of this topic, the analysis of the research status at home and abroad, the related theories and key technologies involved. Based on that, through first-hand use, analyzing some existing push systems, comprehensive summary and in-depth study of key technologies involved, provide preparatory work for this article.

Secondly, detailed introduction of both data-layer research and designing work of Mongoddb's Multi-tenancy technology is offered. After comparing existing problems of relational database, the article offers reasons for using MongoDB as the data layer of multi-tenancy system. Study and designing of the four features of multi-tenancy Platform's (data isolation, customization ability, time performance advantage and data security issue) are done on the MongoDB. Then, through designing systematic database based on analyses of features and demands, the article expands on MongoDB's competitiveness in customization and time. Moreover, there is a brief introduction of the writer's innovative research in the use of MongoDB.

Finally, the article offers detailed introduction of designing work of multi-tenancy push system's applications, designing of all the modules (the Android platform client, server, IOS platform client, server, SMS push send module, Push Management module,

the third-party service invocation module) and test results of the system.

There are four innovative aspects in the article: the data layer which uses MongoDB as the Multi-tenancy system; designing the push as a basic service(both providing users with unified and transparent ways to push messages and providing a wide range of practical functions for tenants); initiating a reading-and-writing pressure mechanism to optimize MongoDB's Auto Sharding mechanism; offering practical methods for pagingreading of MongoDB to be optimized.

Keywords: MongoDB, optimization of MongoDB, Push System, Multi-tenancy

目 录

第一章 绪论	1
1.1 研究背景	1
1.2 研究现状	1
1.2.1 多租户技术研究现状	1
1.2.2 MongoDB 研究现状	2
1.2.3 推送技术研究现状	3
1.3 课题来源	4
1.4 研究意义	5
1.5 主要工作内容	5
1.6 论文组织	6
第二章 相关基础理论和关键技术介绍	7
2.1 多租户技术相关	7
2.1.1 SaaS 模型	7
2.1.2 多租户技术详细介绍	7
2.1.3 多租户技术实现方式	8
2.2 MongoDB 相关	9
2.2.1 NoSQL 介绍	9
2.2.2 CAP 理论介绍	10
2.2.3 MongoDB 特性分析	10
2.3 推送相关	11
2.3.1 常用推送技术	11
2.3.2 Android 推送技术	12
2.3.3 IOS 推送技术	13
2.3.4 第三方推送服务优缺点分析	14
2.4 本章小结	15
第三章 系统需求分析	16
3.1 系统介绍	16
3.2 系统应用场景分析	16
3.3 系统范围定义与用户定义	17
3.3.1 系统范围定义	17

3.3.2 系统用户定义	17
3.4 功能需求	17
3.4.1 多租户推送功能	17
3.4.2 Web 推送管理平台	18
3.4.3 第三方提供推送服务	20
3.4.4 收集反馈信息	20
3.5 性能需求	21
3.6 多租户设计特性需求	22
3.6.1 数据隔离性	22
3.6.2 定制化	22
3.6.3 可扩展性	23
3.6.4 安全性	23
3.7 本章小结	23
第四章 系统多租户数据层的研究与设计	24
4.1 关系型数据库的问题	24
4.1.1 可定制性问题	24
4.1.2 时间性能问题	28
4.1.3 可扩展性问题	28
4.1.4 问题总结	29
4.2 数据隔离性研究与设计	30
4.2.1 多租户数据层的隔离模型	30
4.2.2 MongoDB 完全隔离模型设计	31
4.2.3 MongoDB 半共享隔离模型设计	32
4.2.4 MongoDB 完全共享隔离模型设计	32
4.2.5 本文数据隔离模型的选择	33
4.3 定制化研究与设计	34
4.3.1 无模式方案	34
4.3.2 嵌套设计方案	36
4.3.3 本系统定制化方案选择	37
4.4 可扩展性研究与设计	39
4.4.1 MongoDB 可扩展性优势	39
4.4.2 本系统的设计与实现	39
4.4.3 MongoDB Auto-Sharding 机制优化方法	41

4.5 数据安全研究与设计	46
4.6 系统数据库设计	47
4.6.1 MongoDB 数据库设计原则	47
4.6.2 系统数据库中 collection 设计	48
4.7 MongoDB 相关研究成果	51
4.8 本章小结	53
第五章 系统应用层的设计与实现	55
5.1 系统整体设计	55
5.1.1 系统物理架构	55
5.1.2 系统模块设计	56
5.2 IOS 平台模块设计与实现	56
5.2.1 准备工作	56
5.2.2 推送流程	57
5.2.3 关键部分设计与实现	58
5.3 Android 平台模块设计与实现	63
5.3.1 技术基础	63
5.3.2 整体设计	64
5.3.3 客户端关键模块的设计与实现	68
5.3.4 服务端的设计与实现	72
5.4 短信推送模块的设计与实现	78
5.4.1 短信推送流程	78
5.4.2 关键实现详细介绍	80
5.5 Web 推送管理模块	81
5.5.1 管理平台流程设计	81
5.5.2 功能模块设计	81
5.6 第三方推送服务调用模块	85
5.7 本章小结	87
第六章 系统测试	88
6.1 MongoDB 与 MySQL 多租户下性能试验	88
6.1.1 实验说明	88
6.1.2 MySQL 实验设计	89
6.1.3 MongoDB 实验设计	91
6.1.4 测试程序	92

6.1.5 实验结果及结论	93
6.2 系统测试	94
6.2.1 功能测试	95
6.2.2 整体测试	98
6.3 本章小结	98
第七章 总结与展望	100
7.1 总结	100
7.2 展望	100
致 谢	102
参考文献	103
攻读硕士学位期间取得的成果	105

第一章 绪论

1.1 研究背景

自云计算这一概念被谷歌在 2006 年提出来以后, IT 巨头们如亚马逊、IBM、微软、英特尔等公司都对其表现出极大的热情, 推动着云计算的发展。在 2009 年 ABI Reseach 这一机构发布了研究报告, 首次提出“移动互联网+云”这一概念以后, 云计算与移动互联网的结合成为了 IT 产业主流发展趋势^[1]。

在我国, 根据由中国互联网络信息中心发布的研究报告可知, 中国网民的数量一直在不断增长, 其总人数已经超过了 6.18 亿, 而且其中通过移动客户端上网的用户占到了总量的 81%, 这就是说使用移动客户端上网的人数有 5 亿多, 这个比例是非常大的, 逐步代替 PC, 成为人们获取信息的主要手段^[2]。而且随着我国基础网络设施的建设, 4G 网络资费不断降低, 节省流量已经不是移动开发所关注的重点, 取而代之的是能够迅速的获取信息, 及时的进行信息处理的能力, 而消息推送就能够提供这种能力, 所以对于各个公司来说, 消息推送将成为“移动互联网+云”环境下的必备能力。

1.2 研究现状

1.2.1 多租户技术研究现状

多租户技术是为建立 SaaS 层应用而生的一项关键技术, 这项技术的建立思想是, 使用相关技术, 让多个组织之间能够共享软件资源、硬件资源, 运营管理资源、维护资源, 通过这种方式可以大幅度降低 SaaS 应用的生产成本。

现在国内外的企业级软件提供商比如 SAP、Salesforce、用友、东软都使用多租户技术将其产品升级为 SaaS 层应用。例如由 Salesforce 公司所开发的 CRM 应用系统是广为人知的在实际运用中获得成功的案例之一, 其业务定制具有很高的灵活性, 它也是业界定制性最好的 CRM 解决方案, 每个租户可以根据自己用户的个性化需求, 以及业务的不同进行定制, 从而满足不同规模、不同业务、不同文化习惯的企业的需求。它是通过按需付费租赁的方式进行运营的, 通常在企业使用这个产品后的几个月之内, 企业即可得到令人满意的回报。除此之外, 国内很多公司都陆续推出其多租户产品, 例如中服软件就提供 SaaS 层的多租户应用, 如 OA 系统, 人事管理系统, CRM 系统等。除此之外, 国内有家公司的 CRM 系统是做得最好的, 八百客公司, 其是国内大型企业级 CRM 的提供商, 也是国内最早推

出基于多租户技术 SaaS 层应用的企业。其 CRM 系统提供精简的交互界面、面面俱到的业务处理能力，以及多种多样的自定义功能，而且其也首先引入了移动互联网，只要有连接互联网的终端，如 PC 机，手机，销售人员就可以完成从销售线索的跟进，到完成销售获得提成的整个过程，并且不同的角色有不同的权限。八百客通过多租户技术实现的 SaaS 层 CRM，因为其自身产品质量高，以及其富有竞争力的性价比，其客户覆盖了很多大型企业用户，其中包括中国移动、NEC、中易安担保等^[3]。

综上所述，随着网络通信技术的不断发展，企业获得良好网络通信服务的成本越来越低，SaaS 模型将会有不可限量的发展前景，依靠互联网技术的发展与云平台技术地支持，这种 SaaS 模型提供软件应用服务的方式会越来越深入人心，到目前为止，虽然在国内很多软件行业中还相当多的行业未使用相 SaaS 模型，但是随着中小型企业自身发展的信息化需求，SaaS 模型这种高性价比的软件服务将会成为一种必然的趋势，并且其功能会越来越全面，定制化也会越来越好，结合企业自身特点定制，使之可以更好地满足不同企业的业务需求，为企业提供更佳性价比、质量好的信息化服务，为企业创造更大的收益。

1.2.2 MongoDB 研究现状

目前国内外对 NoSQL(Not Only SQL, 非关系数据库，可以说是一种新型的数据库)的研究可谓是进行得如火如荼，MongoDB 就是文档类 NoSQL 中最炙手可热的。整体上来说，MongoDB 是一个分布式的、面向文档存储的、并且开源的非关系型数据库。它的特点是：它是非关系型数据库当中最像关系数据库的，拥有丰富的功能，甚至可以使用很多 SQL 语句进行关联查询等，而且其又具有非关系型数据库的性能优势、部署方便、无痛扩展、存储无模式的特性^[4]。

在国外运用 MongoDB 最典型的案例就是思科。2012 年的 11 月份，思科发布了 WebEx Social，这个应用是为企业员工提供一个合作与交流的平台，简而言之，WebEx Social 就是一个提供给企业使用的“FaceBook”。项目的最初版本中，思科开发人员是将现有的关系型数据库作为整个应用的数据底层，项目上线后发现系统在处理复杂的 SQL 查询时，耗费的时间是难以接受的。在新版本中 WebEx Social 应用加入 MongoDB 作为数据底层，完成部分数据存储，主要用于对写性能要求高于读性能要求的社交数据进行存储。而且在此基础上，因为 MongoDB 自身带有 MapReduce 的功能，可以简便的、自动的生成很多推荐，比如用户可能会点击哪些连接，想要加入到哪一个社团，对哪些内容有兴趣等等。在使用了 MongoDB 以后，WebEx Social 项目组让写入的性能有了质的飞跃，并且大幅度减少了对缓存的

使用。有了 WebEx Social 平台的成功，思科计划在未来向 MongoDB 上面迁移更多的项目^[5]。

目前国内 MongoDB 的普及度还比较低，很多的企业都是使用其做 DEMO 测试和进行技术预演，还有一些基于 MongoDB 的研究论文。因为其在国内中文文献少，社区支持少，做开发难度很大。目前国内比较成功的 MongoDB 案例是视觉中国网站，视觉中国的数据量是比较少的，属于千万级别，MongoDB 完全能满足其业务需求。除此之外，优酷的在线评论业务已部分迁移到 MongoDB，并会在两年内完成整体迁移。目前 MongoDB 在国内发展火热，越来越多的公司都在将一些功能迁移到 MongoDB 上，并且将 MySQL+MongoDB 这种混合模式的数据层作为行业标杆。

1.2.3 推送技术研究现状

消息推送这一功能最初是由黑莓公司推出的，BlackBerry Push Mail Notification 是黑莓机手机一大卖点，被用于向黑莓手机用户推送新 Email 消息通知。随着移动互联网的发展，以及 4G 网络使得移动传输效率大大提高，所以消息推送功能已经不止是用于推送 Email 消息通知，而是每一个 APP 所必须的功能^[6]。

现阶段国内外对于推送系统的研究主要集中在推送机制以及如何实现第三方推送服务上。推送机制的研究主要是以 Android 为主，因为目前在 IOS 平台上消息推送是由苹果公司提供的统一方案，对于应用开发者来说苹果公司提供的方案是很容易集成实现且成熟可靠，而且因为 IOS 的封闭性，也没有条件进行其它的推送机制的研究工作。可是在 Android 平台上的消息推送机制就可以有多种实现方式，比如基于 GCM 服务，这一服务是由谷歌公司推出的，与 IOS 平台上消息推送的方案相同，但是在国内很难使用且不兼容低版本，除此之外还有基于 MQTT 协议、XMPP 协议、以及 HTTP 协议轮询的机制，这些机制的详细研究将在下章进行说明。

关于第三方服务提供商的发展现状，在国外 PubNub 是一家 2011 年成立的初创企业，它作为第三方提供即时消息的推送服务，用户可以根据文档进行二次开发，并在其 APP 中加入其 SDK，就能够收到推送消息。PubNub 提供了多语言支持，包括 Java、PHP、C++、Node、Python 等，并且对消息进行了 SSL 加密，可以为推送数据提供安全性。

除此之外，国外有款名为 androidpn 的软件十分著名，全称 Android Push Notification，是一款基于 XMPP 协议、Java 语言开发的开源工具包，可以用于向 Android 客户端进行消息推送。androidpn 包含了一套健全的服务器端 jar 包与移动

端 SDK。用户可以很方便的进行二次开发实现相关推送功能^[7]。对于 androidpn 的详细研究在下一章会有介绍。

在国内有极光推送，与 PubNub 一样是作为第三方提供即时消息的推送服务，其实现方法也与 PubNub 相同。其提供的服务是免费的，这是其一大优势，缺点是随着其用户量增大，出现了推送成功率降低情况。

除了极光推送，个推也是市场占有率极高的第三方推送服务提供商，其支持的平台包括 IOS、Android、Windows Phone 等，个推服务是需要收费的，但是对于使用者来说，个推能以较低成本提供有效的消息推送服务，已经有许多公司选择了个推，比如新浪微博、去哪儿网、PPTV、唱吧等，我们普教云现有的推送也是基于个推的。本文在下一章会分析个推详细优缺点。

1.3 课题来源

选题来自于实际产品的需求，本人攻读硕士学位期间在导师公司实习时全程参与了普通教育云平台这一产品的研发，该产品是基于多租户技术的 SaaS 层产品，经过几年的发展，现在该产品的用户规模已经达到租户有 200 所中小学，用户数大约 35 万人，年产值上亿。现在产品已经迭代到 3.2 版本。最新版本会提供 Android 版与 IOS 版移动端服务，我作为移动版的项目负责人，开发出一套 Native APP + Web APP 混合模式的通用 ComsysAPP 框架，这个 ComsysAPP 框架的特点是，业务模块都以一个 Web App 插件（也可以说是一套 Web 前端代码）的形式存放在本地，插件可以下载、更新、删除，Native APP 就提供插件的管理、联系人、消息查看，以及提供 Webview 展示插件等通用功能。因为这个 ComsysAPP 对于不同业务模块在 Android 与 IOS 上都只需要提供一个相同的插件，而且业务模块的需求变更只需要更新插件即可，Native APP 是很稳定的，基本不需要更新。所以公司里的其它产品也都使用该 ComsysAPP 框架来实现产品的移动化。

此基础上，很多产品，包括普教云都要求实现消息推送功能，以使用户能及时、准确获得与自己相关的信息，对于这一需求，现在的实现方式是在 MySQL 数据库中给每个租户都建立独立的消息表，然后通过触发器将新的消息写入共享发送消息表，Spring Task 对共享发送消息表进行定时扫描，最后基于第三方推送服务（个推）将消息推送到用户终端。但是此解决方式在实际使用过程中有很多问题，这些问题会在下一章中介绍。所以经过与导师以及公司架构师商量，决定由我来实现一个 SaaS 层多租户应用，即基于 MongoDB 的多租户推送系统，并以此作为硕士毕业论文。

1.4 研究意义

1、解决现有多租户环境下消息推送业务层实现复杂、关系型数据库负载高，以及依赖第三方推送服务时功能限制、安全性、费用高等问题，使普通教育云平台这一产品更为完整，可以消除第三方依赖，不仅仅实现消息推送，还要将消息推送这一功能作为一个可定制化的业务模块整合进普教云产品中，也会做为一种基础服务向其他产品提供推送服务；

2、通过对 MongoDB 与多租户结合的深入研究，论证 MongoDB 做为多租户数据层的优势，并对其中关键技术进行设计与实现，为普通教育云平台这一多租户产品迁移数据库进行一次技术预演，为数据库使用 MongoDB + MySQL 策略提供实质性的意见与实现方式；

3、通过对 MongoDB 的实际使用，提出自己对一些问题的创新性优化方式、解决方式，并撰写为专利，提升产品的创新性以及为 MongoDB 这项技术的发展做出自己的贡献；

4、在系统的研究、设计、实现中会有很多困难，可以锻炼自己的研究能力以及工程领域的能力。

1.5 主要工作内容

1、前期工作主要是进行理论与现有产品的分析研究，了解推送系统的常用实现方式、多租户技术的实现方式、MongoDB 的使用场景，通过前期工作，确定系统实现的总体架构，在此基础上对 XMPP 协议以及 MongoDB 进行了深入的研究工作。

2、将根据多租户数据层的设计特性，包括数据隔离性、定制化、可扩展性、数据安全，在 MongoDB 上进行研究设计，研究成果将在本系统中使用并成为公司新框架中重要一部分，并提出一种对 MongoDB Auto-Sharing 机制的优化方法。

3、将在上述研究设计的基础上，根据需求分析对系统数据库进行设计，并将通过对比实验与传统关系型数据库进行定制化时间性能对比，说明其相对于传统关系型数据库的优势，并提供安全访问数据的方案。

4、将设计并实现基于 MongoDB 的多租户推送系统，完成需求分析、整体设计，以及各个模块的详细设计与实现，并协助测试人员完成测试，保证系统的上线运行。

5、将根据项目中实际使用情况，对 MongoDB 做了优化研究，并将《一种基于 MongoDB 的负载均衡优化方法》与《一种基于 MongoDB 的数据分页优化方法》作为专利发表。

1.6 论文组织

本论文其余部分组织如下：

第二章，将对论文中涉及到的基础理论与关键技术进行介绍，会分为三个方面：多租户、MongoDB、推送技术，还对第三方推送服务优缺点进行了分析。

第三章，进行用户需求分析，通过对用户的需求进行详细分析确定系统需要提供哪些功能，设计与实现才能满足用户的需要。

第四章，这是本文的重点章节，以 MongoDB 多租户数据层研究与设计内容为主，首先会通过分析传统关系型数据库在多租户环境下的问题，给出本文使用 MongoDB 的原因。然后会根据多租户数据层的设计特性，包括数据隔离性、定制化、可扩展性、数据安全，在 MongoDB 上进行研究设计。在此基础上，会根据需求分析对 MongoDB 数据库进行设计。最后会通过对比实验与传统关系型数据库进行定制化时间性能对比，并给出实验结果及总结。除此之外，本章还会对与 MongoDB 相关的专利进行简要介绍。

第五章，这是也是本文的重点章节，以应用层设计实现内容为主，介绍与推送系统相关的工作，首先会对进行整体设计，然后会对系统中各个模块进行详细的设计与实现，包括 Android 平台客户端、服务端，IOS 平台客户端、服务端，短信推送模块、Web 推送管理模块、第三方推送服务调用模块。

第六章，将介绍与测试相关的内容，首先会通过对比实验，将 MongoDB 与传统关系型数据库进行定制化时间性能对比，然后会对系统测试进行简要介绍。

第七章，总结与展望，会对本文主要工作内容进行总结，并对下一步的研究方向、系统设计实现进行展望。

第二章 相关基础理论和关键技术介绍

本章节将对论文中涉及到的关键技术进行介绍，会分为三个方面：多租户、MongoDB、推送技术。

2.1 多租户技术相关

2.1.1 SaaS 模型

多租户技术是为建立 SaaS 层应用而生的一项关键技术，所以在讨论多租户技术之前，得先简单的对 SaaS 应用进行介绍。在传统的软件模型中，一个传统企业级软件提供商的应用或是部署在企业内部的机房，或是部署在软件提供商内部的机房，但是都有个特点，就是一个组织对应于一个应用服务实例，软件提供商需要对不同的服务实例进行研发、部署、维护。近几年，互联网的快速发展让一种新的模型逐渐成为了企业级软件提供商的新宠——SaaS，全称为 Software as a Service^[8]。软件提供商通过前期的大量需求调研，需求分析，然后设计并开发出应用，应用整体框架以及大部分功能都相同，但也具有个性定制化功能，不同的企业组织可以通过租用的方式获得应用服务。

SaaS 模式的前身是 ASP (Application Service Provider, 应用程序服务提供者服务) 模式，ASP 也是让用户向软件提供商租用专业的应用软件，例如客户关系管理系统、财务系统、仓储系统等应用。

但是经过市场的验证，ASP 模式被证明是失败的。虽然 ASP 是失败的，但是它不会影响 SaaS 的成功^[9]。可以说 SaaS 模式是在 ASP 模式上进行了改进，而在我看来 SaaS 模式并不是对 ASP 模式进行了简单的改进，而是进行了质的飞跃。就用 CRM 软件来举例说明，ASP 就是单独对接每个企业，然后会按照各个企业的需求从头开始来开发出一套软件，但对于 SaaS 模式来说，CRM 已经是一套已经开发好的软件，不需要根据企业的需要从零开始来开发软件，只需要直接进行定制化使用。

2.1.2 多租户技术详细介绍

多租户技术其直观定义是在一台服务器上运行单个应用实例，它为多个租户提供服务。其主要研究内容包括多租户如何共享相同的系统和组件，以及保证租户间数据的隔离性、安全性^[10]。

根据前文的介绍，可知 SaaS 层的多租户这一模式可以从技术与商业两方面来

理解。在技术上来说其实就是一个系统的隔离使用，让用户觉得自己是在私有地使用这个系统，但是实际上是将应用软件统一部署在服务器上，然后供多个用户使用。在商务上来说多租户是多个租户以租赁形式向服务提供商租赁软件应用服务，通常的计费方式是按照租户中用户数，每年进行收费。

可以看出租户是多租户 SaaS 应用的基本单位，租户可以是一个公司、一个学校、或者是一个公司里的一个部门、一个学校里的一个年级，我们通常所说的用户则是租户里面的一个可以供使用者使用的账户。因为多租户共用统一部署在服务器上的系统，所以可以大幅度降低成本，这是 SaaS 模式下多租户系统的最大优势，因为其成本优势明显^[11]，所以这种模式尤其适用于中小型企业以及普通教育的教育信息化中，可以为信息化建设经费有限的组织提供优质的服务，这也是本论文的初衷。

2.1.3 多租户技术实现方式

通过研究国内外现有的多租户技术，从大的角度来说，其实现方式主要有三类，第一系统的虚拟化，第二应用资源管理，第三数据切割，实际上它们是从不同的计算机概念的层面（三类分别对应了系统层、程序层、数据层）对租户来进行隔离^[12]。下面是对三种方法的简要介绍：

在系统层面，主要采用的是虚拟技术，把实体系统从逻辑上切割成多个平行的虚拟机，租户在不同的虚拟机上运行应用，保存数据，达到隔离目的，这样的隔离方式最简便，但是代价较大，目前阿里云就是使用的这样的方式提供了云服务。

在程序层面，在程序上对每个租户设定特有进程，让租户之间无法通过进程进行通信，提高数据库的操作权限，让很少人可以操纵数据库，加上进程的隔离，租户的应用就可以得以隔离，但是国内外现在对程序层面隔离的研究相对另外两层较少，原因有两点，一是因为这种方式不易实现，二是因为实现的效果也不是很理想。

在数据层面，通常是使用切割数据库、存储区等方式，因为其数据可能会有交叉，可能会公用一个数据库，所以非法用户可能有机会刺探其他租户数据，所以必要的时候会以不同租户的密钥进行对称或非对称加密，对一些敏感数据进行保护。在数据层也有不同的隔离方法，它们有不同的实现复杂度、也对应了不同的风险以及效率，因为其较为容易实现，以及其多种实现方式产生的灵活性，成为了实现多租户的首选实现方式。

2.2 MongoDB 相关

2.2.1 NoSQL 介绍

MongoDB 是 NoSQL 数据库的一种，所以在介绍 MongoDB 之前，有必要先介绍一下 NoSQL。NoSQL(Not Only SQL)也被称之为非关系型数据库，其实它是一种统称，代表多种类型的与传统关系型数据库之间存在着比较大的差异的数据管理系统。不了解 NoSQL 的人，都会根据其字面意思理解，认为其与关系型数据库最主要的区别就是它不会使用传统的 SQL 语句作为数据操作语言。事实上 NoSQL 的最大特点是，在其中存储的数据是不需要关系型数据库的特定表结构。通常情况下，NoSQL 是不支持多表连接操作的，也不具有完整的 ACID(即原子性、一致性、隔离性和持久性)事务属性，但是具备优秀的可扩展性，这些特点都是制约关系型数据库性能的瓶颈所在。

自从 NoSQL 这一概念进入人们的视线以来，出现了多种多样的 NoSQL，其中最常用的有三类：第一是面向列存储，代表是 Google 的 BigTable；第二是面向 Key-Value 存储，代表是 Amazon 的 Dynamo；第三是面向文档存储，代表是 10gen 的 MongoDB。不同的类型设计的出发点与可应用的场景都不相同，但是 NoSQL 数据库都会具备如下四个特性^[13]：

1、具备水平扩展能力

能够支持的数据库的水平扩展，而且扩展的代价很小。随着系统的发展，数据也会随之增长，当到达一定规模时，数据库可以用添加廉价服务器的方式来对数据库性能进行提升。

2、复制和分区的能力

NoSQL 数据库大多数情况下都是建立在大规模集群上的，集群大多是廉价的服务器，甚至是 PC 机，所以单机性能是不稳定的，容易出现故障，相同的数据会分布到多个不同的服务器上。

3、较弱的事务模型

对于关系型数据库来说，其事务都具备四个特性，即 ACID 属性，分别对应于原子性、一致性、隔离性和持久性，这些事务特性是关系型数据库性能瓶颈所在。比较而言，NoSQL 数据库对事务的支持不如关系型数据库，只需要能够达到最终一致性，这种机制可以提高数据库的并发性能。

4、无模式

存储的数据是无模式的，不需要像关系型数据库那样预先定义好数据的存储模型，这使得在需求发生变更时或在多租户定制化的环境下，不用对数据库的结构

进行变更，只需改变应用层代码即可。

2.2.2 CAP 理论介绍

CAP 理论是 NoSQL 的基础理论，其中 C 是代表 Consistency 一致性，A 是代表 Availability 可用性，P 是代表 Partition Tolerance 分区容忍性，任何 NoSQL 都是建立在 CAP 理论之上。该理论是由 Brewer 学者提出，有着对现代可扩展的大型分布式数据库完整性的概括。CAP 理论的中心思想是对于 NoSQL 数据库来说，因为其具有大型分布式的特点，所以不可能同时具备 CAP 三个特性，每个 NoSQL 数据库都需要做出权衡，只能从三个特性中选出需要的两个^[14]。

一致性指的是对数据读写的一致，对于不同的数据使用者来说，并发操作看到的是有效的、一致的数据状态，即不会没有脏数据。在单个数据库服务节点上，一致性可以依靠 ACID 来实现，但是在分布式环境下，这就会变得复杂得多。

可用性是指数据库系统能够在应用层对数据进行访问时提供服务。如果数据库出现无法连接、访问没有响应，则表明服务是不可用的。在 CAP 理论下，可用性的定义更加严格，对于很小、很轻微的延迟或者是访问阻塞，就认为是不可用的。

分区容忍性是指数据库集群在遇到任何分区出现故障的时候，仍然能够保证对外可以提供满足一致性和可用性的服务，除非是整个网络环境都发生了故障。简单可以理解为，多副本的分布式存储^[15]。

任何 NoSQL 数据库都是具有良好的可扩展性的，所以在 CAP 理论中的分区容忍性是必备的。MongoDB、Hbase、Redis、MemcachedDB 都是选择了 Consistency 与 Partition Tolerance。对于 C+P 这种模式，在极端情况下，会出现有节点出现故障时，导致整个数据库服务不可用，直到数据库各个节点恢复到一致的状态。除了故障情况，其中一些分布式数据库操作，比如备份、快速复制和错误恢复，也会导致在很短的一段时间里也会出现服务不可用。但是却能让数据更加有效，而大部分情况的轻微不可用是可以接受的，并不会导致灾难性的后果，所以大部分 NoSQL 数据库都选择了 Consistency 与 Partition Tolerance，包括 MongoDB^[16]。

2.2.3 MongoDB 特性分析

我通过对相关论文以及博文的研究，从理论与项目实际使用的角度出发对 MongoDB 做了较为系统的归纳，对本文的撰写有着非常重要的价值，概况起来说 MongoDB 的主要特点如下：

1、文档型数据库，与关系型数据库不同，其数据的存储，不需要像关系型数据库那样，将一个对象的数据通过外键的关联存储在多个表中，而是就像文档一

样依次记录一个对象的所有数据，而且文档是可以嵌套的，所以使用文档型数据库的好处就是，数据存储和程序中所呈现的并无两样，无需复杂的映射过程

2、“面向集合”(collection-oriented)，它是以集合(Collection)为基本单元进行存储，用于存储复杂的、不一致类型的对象数据有明显的优势。对照来看，在关系型数据库中，数据库表(table)的概念与集合的概念相近似，两者的最大区别在于集合是无模式的^[17]。

3、无模式(Schema-Less)，也叫做自由模式(schema-free)，对像不会像关系型数据库那样先进行表的结构定义，然后根据表结构严格控制记录的插入与修改，使得插入修改操作自定义性强；

4、MongoDB 由于采用与 JSON 语法风格相同的 BSON，容易理解和掌握，也是以键值对的形式进行存储，而且每个文档都会有一个唯一的“_id”键；

5、对嵌入子文档有支持，其它 NoSQL 数据库对此支持很少，这可以实现关系数据库中外键这一功能；

6、CRUD 简单，可以实现大多数关系型数据库 CRUD 操作，并且语法接近 SQL，降低开发人员学习成本；

7、查询快，并且并发性能优越；

8、支持多种存储模式，既可以存储富文本，也有 GridFS 利于小文件的存储，但是都是以二进制的格式进行存储；

除了上述特性以外，MongoDB 还支持多种开发语言，比如 JAVA，PYTHON，C++，RUBY，PHP 等，而且都有相当多的开源 MongoDB 框架，简化数据层的操作。

根据 MongoDB 的特性总结，我个人认为，MongoDB 这一数据库对于设计多租户数据层来说，其最大的优势在于 MongoDB 存储数据的方式十分的灵活，没有繁琐的、复杂的、固定的结构来约束数据地存储，因此我们在设计数据库的时候，就可以使得多个租户的各不相同的数据文档存储在相同的集合中。同一个集合中的各个文档可以具有不同的列数，每列可以存储不同的数据类型，每一列又可以是个文档，这就使得多租户中定制化这个问题能够很好地得以解决，而且其效率很高。

2.3 推送相关

2.3.1 常用推送技术

1、轮询(Pull)方式

这是一种“假”的推送技术，可以实现推送的效果。这种方式是基于 HTTP 连

接的，有消息时服务器不能主动向客户端建立连接，只能通过客户端按照特定的频率向服务器建立连接，并且在建立连接后询问服务器是否有新的消息。这种方式的问题在于很难权衡请求的频率，如果请求的频率过高，就会导致耗电量与所用流量大增，如果请求频率过低，就会导致不能够及时的获得的消息。

2、SMS(Push)方式

可以将消息编码后通过短信发送，Android 系统提供了拦截机制，应用将短信拦截，然后进行分析，可以通过分析取得数据或者向服务器进行请求。SMS 这种方式的特点是，稳定性、可靠性非常高但推送的成本也非常高，需要根据推送量向电信运营商支付费用，而且如果在用户卸载了 APP 以后，服务器是不能立即获得此消息，就会导致用户收到奇怪的数据短信，降低用户体验。

3、持久连接(Push)方式

其设计、实现的难度是要高于以上两种方式的，如果想要保证良好的时效性、可靠性、稳定性，就需要系统开发人员付出更多的劳动。其实 IOS 平台上的 Push 服务就是通过持久连接来实现的，但是 IOS 因为其特殊的封闭性，所有 APP 的推送都会进行统一管理，所以推送能够做到可靠、及时、有效、低功耗。Google 也推出了过同样的服务，Android 版本 2.2 及以上的版本都可以使用，命名为 C2DM(Cloud to Device Messaging)，现改名为 GCM(Google Cloud Messaging)。^[18]

2.3.2 Android 推送技术

在 Android 平台上的消息推送，我们会使用持久连接的方式，目前国内外 Android 平台上的常用的持久连接推送技术主要有以下三类：

1、基于 GCM 服务

这种方式就是基于上文中所提到的 Google 提供的 GCM 服务，最开始叫做 C2DM 服务。该服务会向软件开发者提供一个轻量级的解决方案，我们服务器有消息后就与 Google 服务器请求通信，Google 服务器在收到请求之后，因为客户端会与 Google 服务器建立持久连接，就可以立即通知客户端有消息需要接收，然后客户端就会向我们服务器发起网络通信，连接之后就能够从服务器获得最新的数据。在使用原生的 Android 系统时，一开始就会要求用户添加 Google 账户，只有通过添加该账户，才能够与 Google 服务器保持持久连接，从而使用 GCM 服务，就可以收到推送消息。在国外，特别是欧美国家，几乎所有 Android 用户都会设置其 Gmail 账户，但在国内 Gmail 的用户数并不多，并且用户基本会跳过 Google 的设置。除此之外，在国内大部分手机都不会使用原生的 Android 系统，绝大部分手机厂商都会移除 Google 服务，所以用户根本无法进行 Google 账户设置，更别提使

用 GCM 服务了。

其实按理说这种方式应该是 Android 平台推送最为完美的实现机制，因为其耗电量以及在网络流量的消耗上都是很低的，并且对于开发人员来说其实现十分容易，无需编写太多代码和部署推送服务器，可是由于国内的网络环境问题，这个服务经常不可用，所以 GCM 并不适合在中国推广。

2、基于 MQTT 协议

MQTT (Message Queuing Telemetry Transport, 消息队列遥测传输) 是 IBM 公司开发并推广的一种即时通讯协议，它设计的最初目的是为了能够适应物联网环境中的通信，满足其带宽不够和通讯是间断性的需求^[19]。目前，IBM 提供了 MQTT 协议的 C++版的服务端组件，可以用于消息推送的开发。

MQTT 协议作为一种新兴的协议，其推广速度比较快，它的优点是协议精简、可扩展能力强，而且因为其物联网的特性，对移动设备的流量、电量的节省是非常多的。其缺点是它还不成熟、可用资料较少、设计与实现难度较大、没有开源组件，需要专用的硬件、成本较高，最主要的问题是它不是为推送“而生”，所以把其作为推送协议是不安全的，可能会导致推送的成功率不高。^[20]

3、XMPP 协议

XMPP (Extensible Messaging and Presence Protocol) 是一种基于 XML 传输的实时通信协议，是目前主流的四钟 IM (IM: instant messaging, 即时消息) 协议之一。事实上，上文中所提到的谷歌公司 GCM 服务，其实现就是基于 XMPP 协议的，由此可以看出 XMPP 协议在消息推送技术中的重要地位。

XMPP 协议的优点是它已经足够的成熟、功能也很强大、由于其消息承载于 XML，所以可扩展性也很强，它的缺点也较多，协议较为复杂、冗余度高（也是因为基于 XML）、相比 MQTT 协议来说更费流量、更费电，但是因为其较高的推送成功率以及其较为成熟的方案，所以选择它作为本论文的底层通讯协议。

2.3.3 IOS 推送技术

目前在 IOS 平台上提供的消息推送服务，应该是最好的，最成熟的，但是因为 IOS 的封闭性，这一服务只对苹果手机开放，Android 的 GCM 服务就是对其的模仿。基本原理是服务器有消息后就与苹果公司服务器交互，让苹果公司服务器以持久连接的方式通知移动客户端，然后移动客户端主动建立连接与我们应用服务器进行通信，就可以及时的从我们应用服务器上获得最新的信息了。

其实现的大致步骤分为五步：第一步，开发者需要为 App 向苹果公司申请推送服务；第二步，IOS 系统会向 APNS(Apple Push Notification Service, 苹果的推

送服务器)请求设备令牌(deviceToken), 然后传递给 App; 第三步, App 会将 deviceToken 发送到自己的应用服务器; 第四步, 如果有推送消息, 应用服务器就会发送请求到 APNS; 第五步, APNS 会根据应用服务器发来的请求, 推送消息到指定的 IOS 移动客户端^[21]。

2.3.4 第三方推送服务优缺点分析

目前普教云平台的推送是依靠第三方服务个推来实现的, 我独立完成了这一方案下的设计、开发工作, 所以我对个推的有着比较深刻的理解, 以下是对其优点的总结:

- 1、对于精通 XML 配置和面向对象开发的程序员, 上手比较容易;
- 2、学习与开发者对面向对象开发熟练度以及 Android 开发功底有关, 技能熟练的开发者几乎可以边看文档边写代码;
- 3、服务端开发可以使用的语言有 JAVA、PHP、C#以及 Python, 可以满足绝大部分项目的需求;
- 4、集成简单、方便、快捷, 开发者不需要编写太多代码, 就能让 APP 快速集成推送服务。

对于第三方服务, 优点是显而易见的, 但是通过对个推以及其它第三方服务进行研究分析, 总结出第三方推送服务的缺点如下:

- 1、现有的产品大多采用用户数量低于 10000 时免费, 大于 10000 时开始逐步收费的收费方式, 这一点对于普通教育云平台这个产品来说, 因为其推广力度大, 在很多省市都进行推广, 导致租户的不断增加, 保守估计, 其用户将达到 35 万, 所以支付给第三方的费用也会不断增加, 增加产品运维成本;
- 2、第三方服务会给每台手机自动生成 clientId, 我们不能自己设置 clientId, 这样每个手机的 clientId 就是固定的, 我们在进行消息推送时, 首先就会根据 userId 查询 clientId, 用户越多系统负载就越大;
- 3、对于上一个不足之处, 如果将多租户再考虑进去, 系统负载将会变得非常大, 有可能导致消息发送阻塞, 甚至会有宕机的危险;
- 4、数据安全性问题, 这个问题来自于两个方面, 一是第三方推送服务可能会“监守自盗”, 得到发送的隐私消息做分析, 而且免费服务对数据安全没有责任说明; 二是消息保存在第三方服务器, 且不会对内容加密, 导致在客户端可以通过 BroadcastReceiver 对消息进行拦截并发送给不法分子;
- 5、不能得到用户反馈信息, 比如用户是否打开消息, 用户查看消息的时间等等;

6、使用第三方服务，第三方服务可能会以被嵌入 APP 的名义推送广告，这会造成用户体验大幅度降低；

7、使用第三方服务对开发者来说，它是一个“黑盒子”。开发者不能根据自己的需求对推送进行个性化修改，固定的模式使开发者本来在推送服务上做一个很简单的改变就能满足需求，变成需要在自己系统业务层进行复杂的操作才能满足需求；

2.4 本章小结

本章主要介绍了本论文研究过程、系统设计实现过程中所用到的关键理论以及技术，包括 SaaS 模型、多租户模型、多租户实现方式、NoSQL 相关理论、MongoDB 特性分析、常用推送技术、Android 推送技术、IOS 推送技术，还对第三方推送服务优缺点进行了分析，为论文以后的研究性工作、工程性工作都做好了铺垫。

第三章 系统需求分析

在进行系统的设计与实现之前，本章会进行用户需求分析，通过对用户的需求进行详细分析确定系统需要提供哪些功能，设计与实现才能满足用户的需要。本章主要对系统的功能性需求、性能需求与多租户设计特性需求做详细介绍。

3.1 系统介绍

本论文设计并实现的基于 MongoDB 的多租户消息推送系统是在“移动+云”这一趋势下孕育而生的，是成都康赛公司普通教育云平台的一个衍生项目。它作为一个多租户系统，不仅可以向租户提供 Android 平台、IOS 平台、短信消息推送服务，而且可以根据租户的不同需求提供定制化服务，在此基础上，还为普教云租户提供了 Web 推送管理，包含多种实用的、可定制化的功能，例如定制化报表、自定义推送方式、手动推送消息等，还向第三方提供了简单的推送接口，方便他们进行消息推送。本系统也是作为一个数据的收集系统，各个租户推送相关的数据可以全部存储到 MongoDB 中，这些数据可以作为大数据分析的基础数据，提供给其他系统进行数据分析。

3.2 系统应用场景分析

系统应用场景是作为普通教育云平台这一产品的衍生项目，该产品是基于多租户技术的 SaaS 层产品，经过几年的发展，现在该产品的用户规模已经达到租户有 200 所中小学，用户数大约 35 万人，年产值上亿，现在产品已经迭代到 3.2 版本。最新版本会提供 Android 版与 IOS 版移动端服务，我作为移动版的项目负责人，开发出一套 Native APP + Web APP 混合模式的通用 ComsysAPP 框架，因为这个 ComsysAPP 对于不同业务模块在 Android 与 IOS 上都只需要提供一个相同的插件，而且业务模块的需求变更只需要更新插件即可，Native APP 是很稳定的，基本不需要更新，所以公司里的其它产品也都使用该 ComsysAPP 框架来实现产品的移动化。

此基础上，很多产品，包括普教云都要求实现消息推送功能，以使用户能及时、准确获得与自己相关的信息，对于这一需求，经过与导师以及公司架构师商量，决定由我来实现一个 SaaS 层多租户应用，即基于 MongoDB 的多租户推送系统。

按照公司整体架构的视角来看，本系统规划为整体架构中的一个基础设施型服务模块，用来向其它移动产品提供消息推送服务。因为现有产品都需要进行移动

化，所以租户的数量将不断增加，且各移动产品都会有其不同需求，在这一大背景下，本系统的应用场景就变得较为复杂。

3.3 系统范围定义与用户定义

3.3.1 系统范围定义

本系统 V1.0 版本用于成都康赛公司内部使用，为公司移动产品提供消息推送服务，并且会为普教云平台租户提供 Web 推送管理，包含多种实用的、可定制化的功能，例如定制化报表、自定义推送方式、手动推送消息等，凡是与这两方面相关的内容都属于系统的职责范围。除此之外，系统可以将推送数据提供给其他系统，用于进行数据分析，但这不属于系统的职责范围。

3.3.2 系统用户定义

按照用户需求描述，系统可以确定有三个参与者：移动客户端用户、普教云租户管理员、第三方开发人员。

移动客户端用户：此参与者是使用基于本系统提供推送服务的移动应用人员，目前其包括使用普教云系统的老师、家长、学校管理人员，以及其它移动产品的使用者，他们是推送消息的对象。

普教云租户管理员：此参与者是普教云各个租户的管理员，他们会使用 Web 推送管理平台，进行手动消息推送、自定义推送方式、生成定制化报表等。

第三方开发人员：此参与者是其他移动产品的开发人员，他们需要按照系统要求进行开发，然后使用推送接口进行消息推送。

3.4 功能需求

因为本系统采用的迭代模型进行开发，本论文撰写时的功能性需求包括：向多个租户提供 Android 平台、IOS 平台，短信消息推送功能；向普教云租户提供 Web 推送管理平台；向第三方提供推送服务；收集反馈信息。

3.4.1 多租户推送功能

系统可以为多个租户提供主流移动平台的推送服务，此功能是本系统最重要的功能，只要开发者按照我们系统要求进行 APP 开发，然后应用服务器只需调用系统提供的接口就可以通过我们的推送服务器进行消息推送了，用例图如图 3-1 所示：

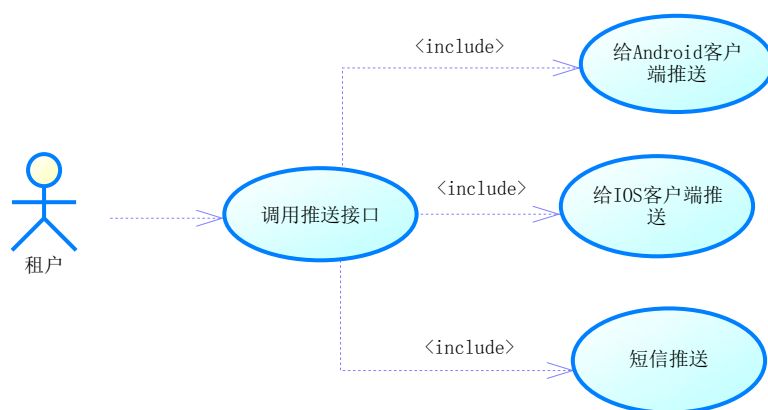


图 3-1 推送功能用例图

应用服务器在调用了消息推送接口后，会根据租户自定义的推送方式选择短信推送和客户端推送进行组合，完成消息推送。当推送服务器完成消息推送后，会把消息推送的结果反馈给应用服务器。

3.4.2 Web 推送管理平台

由于本系统主要服务于普教云平台，目前只对普教云的租户提供 Web 推送管理平台，用于手动推送消息、设置推送方式、查询定制化报表等，用例图如图 3-2 所示：

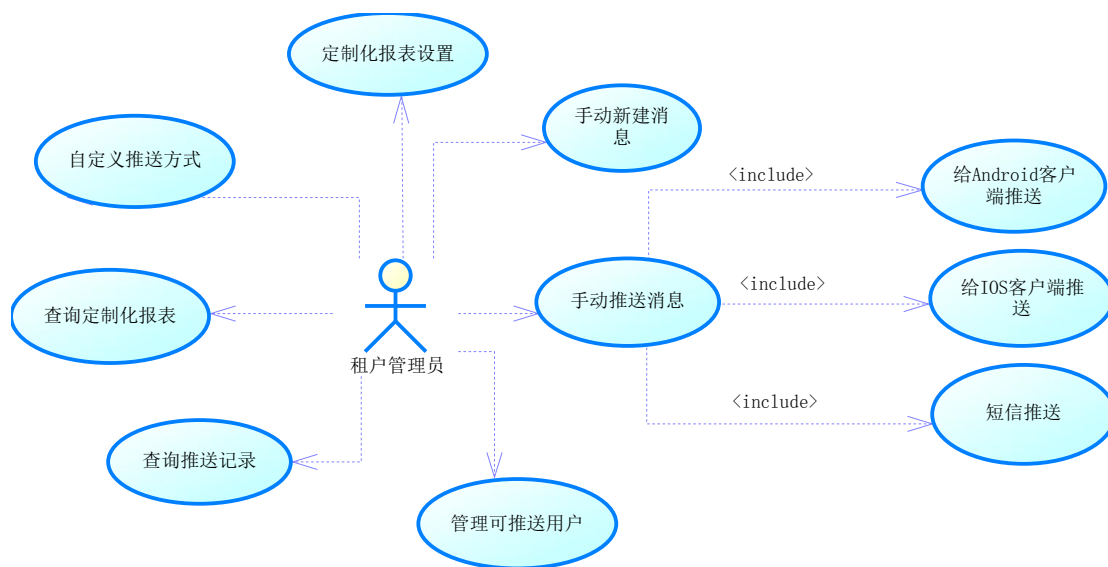


图 3-2 Web 推送管理平台用例图

通过用例图 3-2 可以看出，租户管理员是不需要登录系统这一功能的，因为此 Web 推送管理平台是以一个插件模块的形式开发，依托于普教云平台，用户的登

陆、权限的验证都由普教云平台完成。

1、手动新建消息

租户管理员点击新建消息 Tab, 转到新建消息页面, 管理员需要填写消息标题、消息内容, 以及点击选择用户, 弹出一个当前租户用户树, 通过用户树选择消息推送的用户, 完成上述操作后, 管理员可以点击保存按钮来保存推送消息, 或者直接点击发送按钮发送推送消息。

2、手动推送消息

租户管理员点击推送消息 Tab, 转到推送消息页面, 在此页面中, 会以分页列表的形式列出管理员新建的还未推送的消息, 管理员点击每条消息, 弹出消息详情页面, 可以对消息做修改操作, 标题、内容、接收人都是可以修改的, 也可以删除消息。消息列表第一项是复选框, 管理员可以勾选需要发送的消息, 也可点击全选, 然后点击列表上方的发送按钮发送推送消息。

3、管理可推送用户

租户管理员点击管理用户 Tab, 转到管理用户页面, 在此页面中, 会以分页列表的形式列出当前租户哪些用户已经使用了移动端登录系统。列表展示了用户的登录设备, 推送状态等, 第一项是复选框, 管理员可以通过勾选需要操作用户, 也可全选, 点击禁用推送按钮改变用户推送状态为禁用, 点击恢复推送按钮改变用户推送状态为正常。

4、查询推送记录

租户管理员点击查询记录 Tab, 转到查询记录页面, 在此页面中, 会以分页列表的形式列出当前租户消息推送的记录, 包括手动推送和应用层通过接口调用的推送, 列表会展示消息的标题、内容摘要、推送时间, 推送状态, 管理员点击一条消息会显示消息详情, 包括完整的内容以及消息接收人。还提供搜索查询功能, 可以以标题、内容、时间段做为查询条件。

5、自定义推送方式

租户管理员点击自定推送 Tab, 转到自定推送页面, 在此页面中, 管理员可以勾选两种方式, 移动端推送、短信推送, 两者可以同时勾选, 但不能都不勾选。除了推送方式的设置, 还可以自定义消息推送的次数, 设置完成后点击保存。

6、查询定制化报表

租户管理员点击查询报表 Tab, 转到查询报表页面, 在此页面中, 管理员可以查看消息推送的统计情况, 可以按时间查看柱状图, 查看每日、每周、每月消息总量, 点击每个节点可以显示推送的详情, 有多少 Android 移动端推送、多少 IOS 移动端推送、多少短信推送; 也可以查看租户当前移动端情况, 可以以饼状图的

形式展示,登录过移动客户端与未登录过移动客户端的比例、Android 与 IOS 比例;如果有定制化功能,还会展示定制化报表,对于单个用户,可以查看其消息反馈的点击时间的变化趋势等。

3.4.3 第三方提供推送服务

根据其它移动产品开发者的需求描述,第三方使用本系统时,只需要按照使用说明,在其 APP 中加入系统提供的文件,然后调用推送接口即可,用例图如图 3-3 所示:

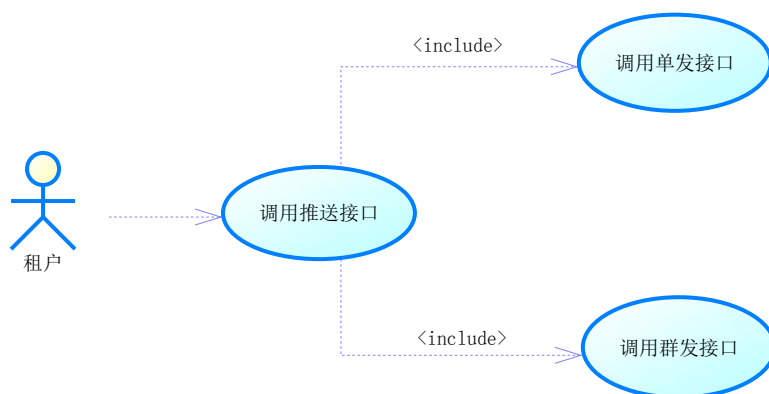


图 3-3 第三方推送服务用例图

第三方服务器需要以 POST 的方式向我们服务器提交数据,其中包含消息标题 title, 消息内容 content, 租户 tenantId, 其中标题不超过 20 个字符, 内容不超过 120 个字符, 还需要提供我们提供的 COMSYS_PSW, 如果是调用单发接口, 就提供用户 userId, 如果是群发接口, 就将 userId 用 “;” 隔开, 作为一个字符串提交, 最大用户数不超过 200 个。

3.4.4 收集反馈信息

根据用户的需求描述,租户的管理员需要了解移动客户端的使用情况,这就需要进行收集反馈信息,收集反馈信息主要是针对移动客户端推送进行收集,不包括短信推送,用例图如图 3-4 所示。

从用例图 3-4 可以看出,消息反馈包括 Android 消息反馈与 IOS 消息反馈,其实现机制有很大区别。

在 IOS 平台上的 feedback 只能得到用户点击消息的时间,而且需要通过苹果提供的反馈服务器才能获得,所以对于 IOS 平台 feedback 的获取,只需要进行服务端的设计来获得用户点击消息的时间。

对于 Android 平台用户在客户端点击消息之后,会通过客户端生成 FeedBack

XMPP 消息，其中可以包括 `location` 点击位置与 `clickTime` 点击消息时间，然后反馈给服务器，服务器会将反馈信息进行相应的处理。

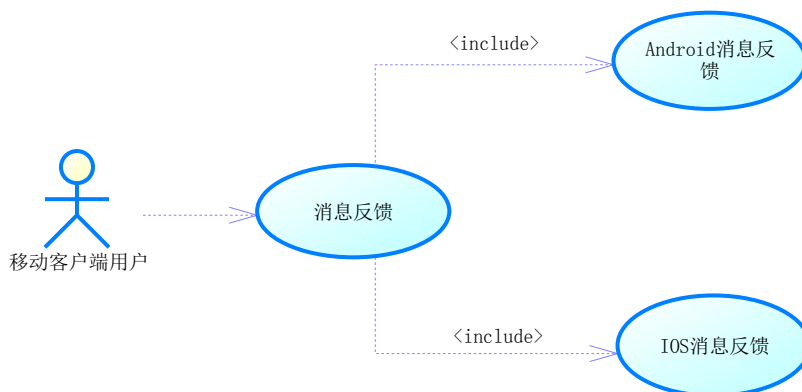


图 3-4 第三方推送服务用例图

3.5 性能需求

本系统可以说是类似于“中间件”，需要为用户提供稳定有效的、及时的推送服务，满足租户的需求，所以对性能方面有如下需求：

1、及时性

消息的推送应该在租户发起推送请求后，在移动端用户网络良好的情况下，30秒内完成消息推送，保证消息能够及时的送达给客户端，使用户办事效率得到提高，及时的获取通知消息，提高用户体验。

2、准确性

准确性就是要保证消息能够正确的推送到相应用户的移动客户端，不会出现将本属于用户一的消息推送到用户二，也不会有本属于用户一的应用一消息推送到应用二。

3、高并发

当租户增多以后，可能会出现大量推送请求同时发出的情况，也会出现移动客户端剧增的情况，对于高并发问题的出现，首先要保证消息能够推送到移动客户端，牺牲一些及时性，然后通过消息队列、增加推送服务器的方式提供系统的并发处理能力。

4、稳定性

稳定性是指的移动客户端与推送服务端两部分都需要能够提供持久的、稳定的服务。对于移动客户端来说，稳定性主要表现在网络良好的情况下，能够持续的收到推送消息，不会出现网络良好却收不到消息的情况。对于服务端来说，稳定性不仅是要持久地、稳定地提供推送服务，还需要定期对数据做备份，保证数据

的安全，因为推送的数据是具有很大价值的，能够提供给用户做决策分析。

5、资源消耗小

虽然随着 4G 网络的普及以及移动客户端电池容量越来越大，消息推送的耗电量与流量消耗可以说是无关痛痒了，但是还是需要尽量减少资源的消耗。

3.6 多租户设计特性需求

本系统做为一个多租户推送系统，其架构设计，大致可以总结为，在多租户的系统里，每个租户都会具有基本相同的软件应用实例与基础架构，但是必须保证多租户数据的隔离性。因为租户不会全部具有相同的业务需求，所以在此基础上，每个租户都有可能具备不同的服务，甚至具有定制的独享服务。

通过与导师讨论，以及相关项目人员访问，对本系统进行分析后，总结出数据隔离性、定制化、可扩展性、安全性是本多租户系统最重要的四项关键设计特性需求。

本文会从多租户数据层设计的四项关键特性出发，并基于 MongoDB 这一 NoSQL 数据库，对多租户数据层的设计进行详尽的研究工作。

3.6.1 数据隔离性

每个租户的数据，不管是相同的软件服务，或是其特有的软件服务，都要求互不影响的分配和使用。在我们推送系统中，必须保证租户间的数据在应用上是相互隔离的，虽然租户的数据可能都存在于一张 collection 中，但是我们必须通过有效的数据隔离模型、良好的设计、安全的应用层来保证数据的隔离性。直观地说，就是要让不同的租户感觉是在独立地使用我们推送系统。

3.6.2 定制化

在共有基础架构和应用实例的基础上，能够支持不同租户拥有不同的特性。多租户系统设计得成功与否的一个重要标志就是其定制性的好坏。在我们系统中，现阶段不同租户的定制化需求包括对反馈信息的不同需求，有的租户需要得到用户点击消息的时间，有的租户需要得到用户点击消息的地点；对推送方式的定制化需求，租户可以组合使用短信推送与移动端推送，可以自行设定；有的租户还要求能够控制消息的推送次数；对用户进行管理的定制化需求，有的租户需要对推送用户进行控制。

3.6.3 可扩展性

可以很方便的、较低成本的横向扩展系统的基础架构，从而可以支持建立更多的租户，并且能保证稳定的性能以及服务效率。在传统关系型数据库作为数据层的架构中，数据库是很难做扩展的，虽然可以进行一些读写分离、垂直切分、水平切分，但是如果涉及到多租户的数据模型，而且当租户数量和访问量不断增长的时候，关系型数据库不能像 web server 那样通过前端服务器实现负载均衡转发，然后只需要通过不断的添加硬件和服务节点来提升扩展性和负载能力。

在我们的系统需求中，对可扩展性会有着较高的要求，因为我们的用户数量是与日俱增的，而且增幅很大，每天产生的数据将越来越多，而且数据都是要保留下来做为数据报表分析的基础，所以系统容量是需要不断增加的。所以系统需求能够在数据库运行时动态的添加节点进行扩容，不影响数据库服务的使用；而且改变对于应用层来说是透明的，应用层可以以相同的形式进行操作访问，而且不会因为做扩展影响数据库功能，也不用修改任何代码，让应用层能更好的关注于自己的业务。

3.6.4 安全性

保证各个租户的数据安全，不会受到其它租户数据的影响，也要阻止非法的数据访问，防止恶意租户刺探其他租户的数据。系统需求的安全性也是要对其他应用层业务透明，应用层只需要按普通方式使用数据库对象，但是会对不同租户用户做过滤，保证不影响其他租户的数据。而且能够防治恶意用户的一些攻击，例如正则表达式匹配攻击、SQL 注入攻击等。

3.7 本章小结

本章首先进行系统的介绍，介绍系统的用途、应用场景、范围定义、用户定义，然后通过对功能需求、性能需求以及多租户设计特性需求进行详细的描述，明确了系统设计与实现的出发点，为系统的设计与实现打下良好的基础，为最后系统成功上线做好良好的铺垫工作。

第四章 系统多租户数据层的研究与设计

本章是本文的重点章节之一，首先会通过分析传统关系型数据库在多租户环境下存在的问题，给出本文使用 MongoDB 的原因。然后会根据多租户数据层的设计特性，包括数据隔离性、定制化、可扩展性、数据安全，在 MongoDB 上进行研究设计。在此基础上，会根据需求分析对系统数据库进行设计。除此之外，本章还会对与 MongoDB 相关的研究专利进行简要介绍。

4.1 关系型数据库的问题

随着移动互联网的崛起，系统所要存储处理的信息量越来越大，而且因为有了 SaaS 层的应用，信息化产业规模也越来越大。根据这两点，可以很容易推断出，未来的多租户系统，在数据层方面三项关键技术会成为多租户系统设计的重点，也是系统开发成功与否的关键点所在，那就是可定制性、时间性能与可扩展性，在关系型数据库中，对于这三项关键技术的设计与实现存在着很多问题。

4.1.1 可定制性问题

在传统关系型数据库中，由于多个租户共享相同的软件架构和计算机资源，不管哪一种数据隔离模式下要向差异用户提供个性化服务都是件非常困难的事情，若是再考虑关系型数据库中的种种设计规范、数据库范式以及约束，定制化的实现将会难上加难。

有很多种方法尝试将关系数据库作为多租户系统架构的数据层，如下就是常见的五种关系型数据库定制化方案^[22]及其优缺点：

1、Private table

这种方法会对需要定制化的业务相关表特殊化处理，会为每一个租户建立不同结构的私有的表，来满足不同的业务需求，如图 4-1 所示。

这种模式实现定制化的缺点是表数量巨大，因为如果业务有定制化需求的话，每个租户都需要建立不同的表，业务定制化的要求越来越高，再加上租户的增多，这会使得数据库里的表数量急速增长，导致系统性能急速降低。

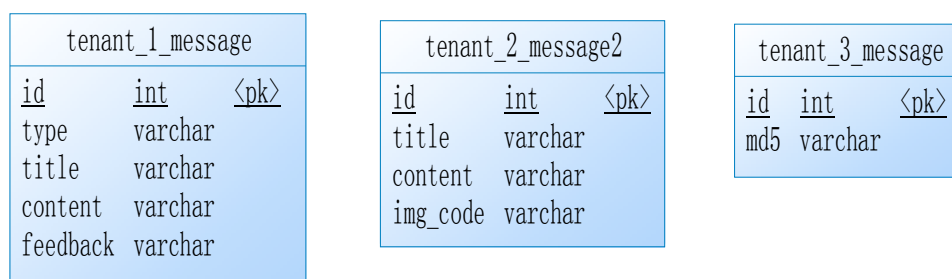


图 4-1 Private table 表结构设计图

2、Universal table

将相同业务数据放入一张表中，表会包括每个租户所需要的字段，然后不同租户只需使用自己定制化相关的字段，其他租户将不使用的字段留白，设置为空值。

这种模式的缺点是显而易见的，就是空间浪费严重，因为这种留白的处理方式，就算只有一个租户需求的字段，也会为每个租户都设置，会导致大量的字段为空，但是空值也是会占用存储空间的，所以就会导致表如同稀疏矩阵一般，造成巨大的空间浪费，如图 4-2 所示，就是基于这种方式的消息表，可以看出空间浪费很大。

id	tenant_id	title	content	location	click_time	feedback	telephone	device_token	push_times
1	1	请假	申请病假	(Null)	(Null)	同意	(Null)	(Null)	(Null)
2	1	信息上报	流感疫情	(Null)	(Null)	(Null)	(Null)	(Null)	3
3	2	请假	申请病假	53°33'E,33°51'N	(Null)	(Null)	(Null)	(Null)	(Null)
4	3	预约申请	多媒体教室	(Null)	(Null)	(Null)	15196612312	(Null)	

图 4-2 Universal table 数据存储结构图

3、Pivot table

这种方式是一种不常见的方式，实现与理解都比较复杂，与 Extension table 相似，会建立共享表存放共享结构，但是它会将业务表里的扩展字段进行标记，用 table_id、row、col 这三个值可以标记所有的扩展字段，然后将相同类型的扩展字段存放于同一张表中^[23]。

这种设计的缺点是实现难度大，实现后又很难维护、扩展，所以在实际情况下用得不多，只是理论上的一种可行方式。

4、Dividing table

会将每个租户所具有的不同数据结构提取出来，建立一张共享表存放共享结构，然后将多租户的定制化数据裁分到不同的定制化 dividing 表里，一种 dividing 表存储一种定制化方案，这些表的主键都是由租户 id(tenant_id)和数据在租户中第几行(row_id)组成的复合主键，两种表都会通过主键进行级联查询数据，除此之外，还需要一个 configuration 表来记录各个租户使用的 dividing 表，靠其记录租户不同的

定制性。图 4-3 是我设计的在关系型数据库下，推送系统中推送消息的表关系，其就是使用 Dividing table 的方式进行设计的：

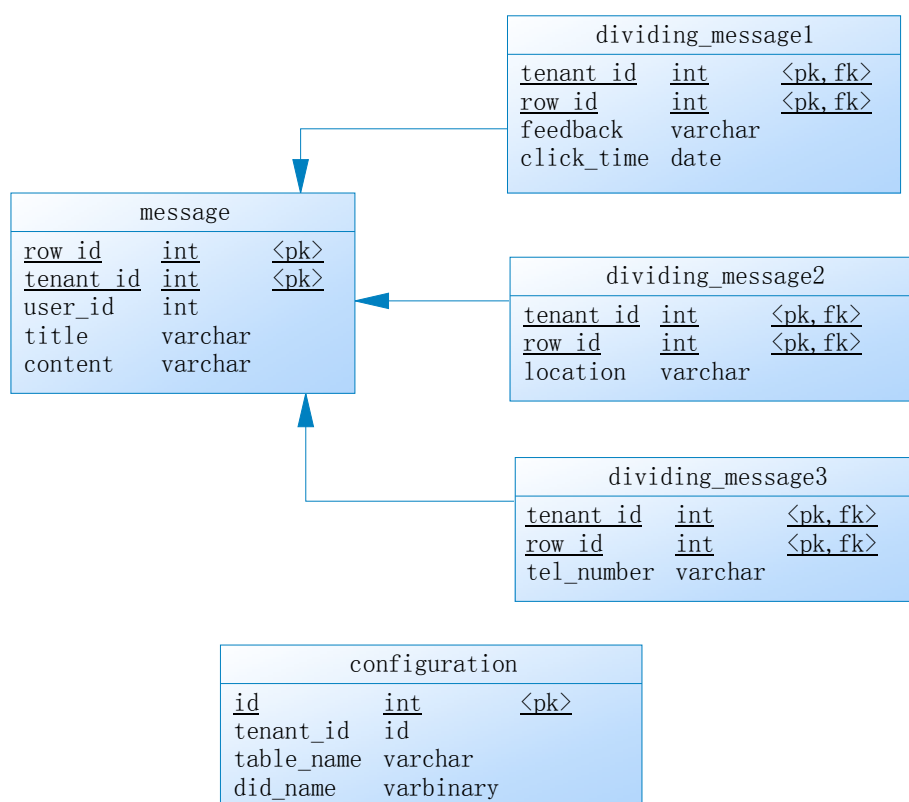


图 4-3 Dividing table 表之间关联关系图

这种方式不浪费空间，也不会造成表的数量过大，但是可以看出其数据库的设计是比较复杂繁琐的，而且其插入、删除、查询都涉及到跨表的操作，这在多租户的环境下是很影响效率的，因为不仅是数据量大，而且并发量也大，对于这种级联表的设计，随着系统的扩张，瓶颈就会越来越明显。除此之外，上述的设计是我对推送系统消息表的设计，这在多租户系统的业务中算比较简单的业务了，其他业务不仅比这个复杂，而且表的变动是很大的，租户的定制化需求多种多样，就有可能造成很多 dividing table，甚至一个租户一个 dividing table，维护十分的困难。

5、Extension table

这中方式与上面的方式有相同点，都是会把租户相同的数据结构提取出来，首先会建立一张共享数据表存放共享的结构，然后通过建立配置表 configuration 和扩展表 extension 来存放租户间差异化的定制数据。配置表 configuration 的作用是记录不同租户的定制信息，其中包括 config_id 会与扩展表相关联，用来存取定制数

据, `tenant_id` 用于区别不同的租户, `table_name` 存放需要定制化的表名, `field_name` 存放定制化属性, `type` 存放数据类型, `is_null` 表明定制属性是否可以为空。扩展表 `extension` 就是用来存放定制化数据的, 其中会有个 `ext_id`, 因为在 Oracle 中有 `sequence` 机制, MySQL 需要靠程序实现 `sequence` 机制, 所以所有数据的主键都是唯一的, `ext_id` 就是需要定制化的数据的主键, 此表中 `config_id` 与配置表相关联, 识别出扩展数据的种类, `ext_value` 存放的就是扩展数据。与 Dividing table 方式不同, 这种方式通过上述两张表就可以实现定制化, 不需要为每种需求建立一个扩展表。图 4-4 就是使用 Extension table 的方式进行设计的消息表:

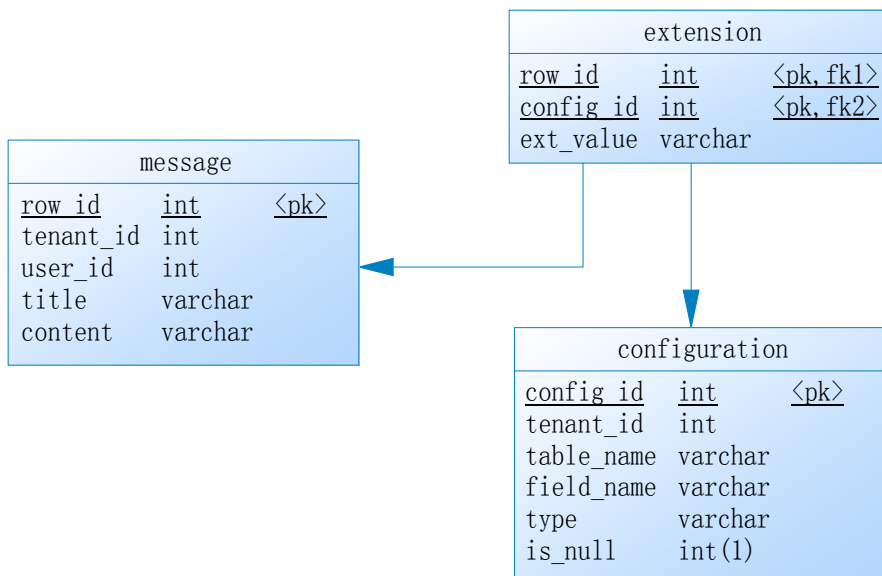


图 4-4 Extension table 表之间关联关系图

在 `configuration` 表中与 `message` 表相关的定制化属性如图 4-5 所示, 其不同租户的定制化需求能够简单明了的展示出来:

config_id	tenant_id	table_name	field_name	type	is_null
24	1	message	click_time	date	1
25	1	message	location	varchar	1
26	2	message	location	varchar	1
27	3	message	feedback	varchar	1

图 4-5 configuration 表数据存储结构图

根据上面的设计可以看出 Extension table 这种方式与 Dividing table 相比, 需要的扩展表只有两张, 表的设计简单明了, 节约空间, 但是缺点也很明显, 其不仅仅会因为多表关联导致性能的降低, 而且随着系统的扩大, 租户的增多, 配置表

和扩展表这两张表中的数据量肯定会急剧增加，性能问题就会更加突出，然后为了改进就会涉及到对表做水平切割，把一张表切割成多张表，这又会加大开发的工作量，也让维护变得困难。

通过对关系型数据库定制化模型的详细说明以及简单的设计，可以得出结论，只要是基于关系型数据库的架构，不论使用上面的哪一种模式，都会受制于关系型数据库的根本特征，定制化都会设计到多张表，有共享表，也有扩展表，这种依靠多表实现的方式，会严重影响数据库的效率，影响用户体验。

4.1.2 时间性能问题

现在的 SaaS 层系统，因为移动互联网的引入以及规模的不断扩大，需要数据库可以提供比以前更好的性能支持，可以执行大批量的查询与插入操作，数据库承受的压力与日俱增，关系型数据库下的多租户系统具有很多的表关联关系，很多的级联查询，插入也会有很多复杂的、严苛的约束条件，所以这对关系型数据库来说是一个很难解决的问题，也可以说是因为这个原因，导致关系型数据库不再适合用做于多租户系统的数据层。

在本章进行数据库设计之后会进行 MongoDB 与 MySQL 的对比实验来论证时间性能这一问题。

4.1.3 可扩展性问题

可扩展(Scalable)也可以叫做伸缩性，主要针对性能、容量方面的可扩展，与功能的可扩展(Extendable)是有很大区别的^[24]，本文中所说的可扩展性是指伸缩性，可扩展性是多租户系统的必备属性，而且随着租户数的增加，会频繁的进行扩容操作。

说到数据库的可扩展性，不得不提 sharding 机制，简而言之，sharding 机制就是将大量的应用数据通过水平切分，分割成不同部分，然后在不同的物理机器上进行存储。数据库的水平切分，是指把一个数据表或者 collection 中的数据按照一定的规则，切分存储到多个数据库中，每个数据库中都有结构相同、名称相同的表或者 collection。对于不同的数据访问，会根据切分规则，直接定位到相应数据库上。因此 sharding 机制可以利用大量的廉价硬件资源，通过负载均衡的思想，将压力均匀分布到各个硬件上，可以解决单个服务器性能瓶颈问题^[25]。

在传统关系型数据库作为数据层的架构中，数据库是很难做横向扩展的，虽然可以进行一些读写分离、垂直切分、水平切分，但是如果涉及到多租户的数据模型，而且当租户数量和访问量不断增长的时候，关系型数据库不能像 web server

那样通过前端服务器实现负载均衡转发，然后只需要通过不断的添加硬件和服务节点来提升扩展性和负载能力。所以这种扩展性能低下的事实，让技术人员对数据库系统进行升级和扩展感到十分的痛苦。在传统数据库，比如 MySQL、Oracle 上实现 sharding 机制存在很多问题，不仅实现起来繁琐、难度大，而且会有性能问题，其中有三个很棘手的问题，如下：

1、扩容的问题，举例说明，一个生产环境中的 MySQL 数据库集群，因为系统规模的扩大，需要进行水平扩容，将原来 5 个节点扩展到 10 个节点，原来数据按 id 进行模 5 运算，按照运算分别存放在 5 个相应的节点上，现在就需要对每条数据进行模 10 运算，然后进行迁移，通过数学分析可知至少需要迁移一半数据。而且如果节点数增加的不是当前节点数的倍数，比如上例中水平扩展到 7 个节点，通过数学计算分析可知，集群中的绝大部分数据都会进行迁移，这样是非常地消耗性能的。除此之外，通常在进行数据迁移的时候，数据库是没办法提供服务的，这会造成长时间中断服务，这是难以接受的。

2、数据路由带来的问题，数据库做了 sharding 以后，访问数据库时，就需要应用程序先去定位数据位于哪个节点之上，这会造成一定的性能损失。

3、跨 sharding 操作的问题，在生产环境中肯定会涉及到多个节点的查询结果进行合并这一操作，通常这种操作会依靠数据库中间件实现，如果自己实现，其代价是非常大的，即便使用了数据库中间件，也很难做到完全的透明化。除此之外，关系型数据库的很多功能在数据库进行了 sharding 以后，是没办法正常使用的，例如 join、事务这种功能在跨 sharding 操作时难度非常大，因此会带大量额外工作。

4.1.4 问题总结

传统关系型数据库对于 NoSQL 的优势在于事务的一致性、读写实时性和复杂 SQL 提供的多表关联复杂查询，但是这些优势在本文的推送系统，或者现在一些 web2.0 系统中显得不那么适用。

要维护事务的一致性的代价是很大的，而对于像推送系统这种系统，对事务的一致性没有太高的要求，所以这种严格的一致性是一种浪费。

同样，在大部分关系数据库中，都有读写实时性的要求，比如一个事务做了插入或者修改操作然后提交，其它事务是可以立即查询到其改变的，但是对于本文的推送系统来说，对实时性的要求并不高，比如有个租户需要推送一条消息，它会将其写入数据库，但是推送系统可以过几秒甚至十几秒后，才读取到此条消息，这也是完全可以接受的。

现在的 SaaS 层应用都相当于是大数据量的 web 系统，这样的系统是非常排斥大表的级联查询的，以及使用复杂的 SQL 语句进行数据分析、报表查询，对与推送系统这种应用，要从需求分析、产品设计开始就要避免这种情况，所以说强大的 SQL 查询功能已经没有太多用武之地了。

所以对于 SaaS 层应用来说，关系型数据库的优势显得并不重要，甚至可有可无，但是其缺点确是系统瓶颈所在。

因此本文选用了 MongoDB 作为多租户系统的数据层，接下来将详细介绍本文多租户系统数据层的研究与实现。

4.2 数据隔离性研究与设计

在前文中已经对 MongoDB 的特性做了总结，所以可以知道在 MongoDB 这一数据库上的多租户数据模型的设计与传统关系型数据库会有很大的区别。所以基于 MongoDB 的特性，以及项目中实际情况，本文对三种多租户数据层的隔离模型进行了设计，并分析其各自特点、优劣。

4.2.1 多租户数据层的隔离模型

通过第二章的介绍，多租户技术的实现有系统层、程序层、数据层三种方式，根据对推送系统进行需求分析，可知，要实现本文多租户推送系统应该是在数据层这个方面进行研究，其原因有三个：

- 1、相比于系统层和应用层两种实现方式，在数据层进行实现较为容易，性价比很高；
- 2、推送系统本来就是一个移动互联网起辅助功能的系统，并不需要为每一个租户都建立虚拟机，让其拥有逻辑上完整的计算资源，这会是一种矫枉过正的方法；
- 3、设计实现高量的、稳定的、安全的数据隔离方法，能够有效地减少多租户系统的运营维护成本，而且如果存在问题，可以很方便的通过分析数据层设计，找出问题的瓶颈，亦可分析出改善软件服务质量的有力措施。

目前根据租户间资源共享程度的高低，可以采取不同的数据层隔离模型，有三种不同的隔离模型：独立数据库（完全隔离）、共享数据库独立结构（半共享）以及共享数据库共享结构（完全共享）^[26]：

独立数据库（完全隔离）：这种隔离模型下会为每个租户建立各自的数据库，租户独立使用各自的数据库进行数据存取；

共享数据库独立结构（半共享）：这种隔离模型下，多个租户会使用同一个数

数据库，但是可以通过多种方式，让每个租户拥有各自的表，比如 Oracle 的表空间模式，或是每张表加上租户 ID 的前缀，这样就可以实现不同的租户共享数据库，独立使用各自的表；

共享数据库共享结构（完全共享）：这种隔离模型下，多个租户不仅会使用相同的数据库，而且不同的租户的数据还会存在同一个存储单元里，通常是用一字段租户 ID 来表明数据属于哪一个租户。

接下来本文将研究以上三种方式在 MongoDB 上的实现方式，并分析其各自特点。

4.2.2 MongoDB 完全隔离模型设计

这种隔离模型下 MongoDB 的数据隔离模型设计与传统的关系型数据库大致相同。将为每一个租户建立各自的独有数据库，数据都将存储于各自的数据库中，软件应用层根据租户 ID 访问不同的数据库，所以它们可以具有不同 collection，在这种模式下 collection 就与关系型数据库里的表相等效，document 就与一条记录相等效，隔离模型如图 4-6 所示：



图 4-6 完全隔离数据模型图

这种模型设计的特点：租户的数据隔离效果是最好的，能够完全隔离不同租户的数据，具有最佳的安全性，但是也具有最差的共享性，对资源的要求最高。

优点：由于每个租户拥有独立的数据库，所以就可以让各个租户的数据库中的 collection 各不相同，这能够更容易地对不同租户进行定制化设计，而且在数据库出现故障的时候，可以使数据的恢复操作更加容易，因为 MongoDB 是文档型数据库，在运行系统中，其每个数据库都是个文件，可以很方便的对不同的租户进行数据库的拷贝、冷备份。

缺点：最大的问题就是每个租户都需要一个数据库，需要建立很多数据库，成本较高，而且随着租户的不断增多，所带来的扩建成本与管理运维的问题肯定不止是成线性增长的。

如果我们的客户是政府单位、大型国有企业、金融企业等，这种要求安全性高而且能够承担较高费用的租户，使用这种模型就是比较合适的。但是如果我们的客户大多数是中小型企业、中小学等，而且对数据的隔离性要求不高，这种模型对他们来说就是过犹不及的。

4.2.3 MongoDB 半共享隔离模型设计

在这种隔离模型下的设计，所有租户的数据都存储在相同的数据库里，但是数据库里的任意 collection 都只能属于一个租户，虽然在这种隔离模型下租户的数据都存在一个数据库中，但是每个租户都相当于一个抽象的、独立的模型存在于数据库中。MongoDB 下的数据隔离模型设计与关系型数据库实质上大致相同，每个租户也是由多个 collection 组成，每个 collection 会加上租户 ID 的前缀，所有具有相同前缀的 collection 就可以组成一个完成的、抽象的数据库。在这种模型下 collection 就与关系型数据库里的表等效，同样 document 就与一条记录等效，隔离模型如图 4-7 所示：

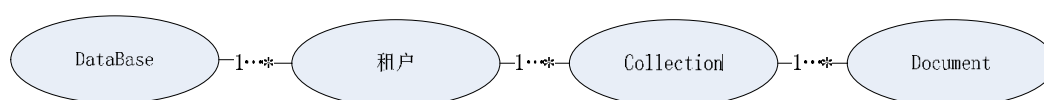


图 4-7 半共享型隔离数据模型图

这种模型的特点：各方面平衡是它的最大特点，其实现相对容易，而且成本也不高，有一定的安全性，也能做到数据隔离，但它肯定不能像上一种模型一样做到完全隔离。

优点：就如上文所说，这种模型最大的优点也是平衡，各个方面都可以表现得中规中矩。可以说很多软件供应商都是使用的这种模型。

缺点：这种模型的缺点中最为突出的一个问题就是数据的备份与恢复问题，举个例子，由于一个数据库中储存有多个租户的数据，如果有个租户的数据出现问题需要回滚，就只能找出一个租户的 collection 进行回滚操作，并且这种做法是很不安全的，如果用程序实现的话，其处理起来将会有一定的难度。除此之外，在实际项目中也遇到了比较大的问题，比如跨租户数据统计，因为租户的数据都存在于各自的表里，而且区别不同租户只能依靠表名，由于这种模型的建表方式，要进行跨租户的数据统计，只能依靠上层程序，而不能依靠数据库函数，这样就会造成性能问题，而且程序的设计也比较复杂。而且用这种方式实现，就不能体现出 MongoDB 这一 NoSQL 数据库的无模式优势。

4.2.4 MongoDB 完全共享隔离模型设计

这种隔离模型下的设计能够体现出 NoSQL 的优势。所有租户都可以存储在一个数据库中，而且租户的数据也会存在于相同的 collection 中，在这种模型下，collection 在数据隔离模型层次里租户的上一层，而租户是可以从不同的 collection

中抽象出来成为独立存在的实例，由抽出的若干 document 组成。虽然在同一个 collection 中存放的是多个租户相同类型的数据，但是因为其是无模式的，所以不同的租户可以拥有不同的属性，这是该数据隔离模型设计的关键点，属性就对应于 document 中的键值对。因为 MongoDB 是文档型数据库，以及其 JSON 风格的数据存储方式，所以可以很方便的给每条 document 加上一个键值对 tenant_id 来表明其属于哪个租户。如图 4-8 所示：

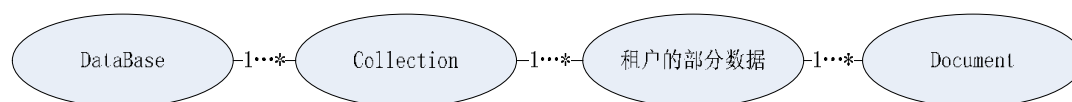


图 4-8 完全共享型隔离数据模型图

这种模型的特点：数据库完全共享，数据的隔离性肯定是最底的，这也会导致较低的安全性，但是可以通过多种方式规避安全问题，而且因为其成本最低，而且 MongoDB 无模式的特点与其不谋而合。

优点：这种模型最大的优点就在于它成本低，其所需硬件基础成本最低，也能降低管理成本，而且其能支持的租户数量理论上无限的，实际上也是三种模型中用得最多的。

缺点：显而易见，它的缺点就在于数据的隔离性和安全性上，所以在这种隔离模型下设计与实现上层系统应用的时候应该特别注意安全问题，防止有安全漏洞，规避风险；还有个问题是租户数据的备份与恢复，这个问题应该是这种模型下最大的问题，其备份恢复操作必须逐表逐条的进行，所以遇到系统故障时的操作最为麻烦。所以其适合对安全性、稳定性要求不高的系统，而且期望用最少的服务器来提供服务，这种模型就是非常合适的，比如本文中的多租户推送系统就是对安全性要求不高，从而可以使用此模型降低系统的成本。

4.2.5 本文数据隔离模型的选择

根据前文的数据隔离模型设计，对三种数据隔离模型特性的分析、优劣的对比，本文决定使用完全共享这一隔离模型作为本多租户系统的数据层的模型。其主要依据是：

1、推送这一功能作为移动互联网时代的重要功能，是每个 APP 都所需要的，它可以及时的告知用户消息，但是其也是个辅助的功能，所以如果选择用多租户的方式实现这一推送系统，就不用去追求数据的完全隔离性、系统的绝对稳定性，因为用户是可以接受一些小差错的。

2、正如上文所说，这个功能既然是辅助功能，我们就不会把其作为一个单独

的系统进行租赁，而是通过附送的方式提供给用户，也就是免费提供的，我们现在就是做的中小学普通教育云平台，在用户租赁移动版后，就会附送给用户推送系统，所以说这个项目的成本应该要很低，这也是选择完全共享这一隔离模型的原因。

3、又因为本文是基于 MongoDB 这一 NoSQL 数据库的研究，所以就要完全利用 MongoDB 相对于传统关系型数据库的优势，完全隔离与半共享隔离这两种隔离模型其实和传统关系型数据下的设计一样，只是会利用 MongoDB 的性能优势这一特点，而完全共享隔离这一模型不仅可以利用 MongoDB 的性能优势，还可以利用 MongoDB 的无模式这一特点，对于多租户系统的定制化更为有利。

4.3 定制化研究与设计

多租户系统设计得成功与否的一个重要标志就是其定制性的好坏。正如前文所说，因为不同租户的业务需求会有差异，所以对系统来说租户间不同属性的处理方式就变得十分关键。

MongoDB 在定制化方面有着先天的优势，首先因为 MongoDB 是文档型 NoSQL 数据库，所以与关系型数据库有很大不同，MongoDB 中没有各种各样的约束条件限制数据存储，然后其没有固定结构、无模式、可嵌套的特点，给多租户的数据层设计带来了本质上的改变，MongoDB 上的定制化模式必然与上文 4.1.1 小节中多种关系型数据库定制化模式大不相同。

根据 4.2.5 小节的结论，本文选择完全共享数据模型作为本文的数据隔离模型，所以会详细对其定制化进行研究设计。由于在这种模型下，租户会共享 collection，而且租户对定制化要求也不相同，所以如何利用 MongoDB 的特性设计出良好的定制化模式就是数据层模型设计成败的关键。

根据上文 4.2.5 小节对数据隔离模型的选择，本文会将相同的业务数据存放在相同的 collection 中，由 MongoDB 无模式和嵌套性两个的特点，我提出了两种定制化设计方案。

4.3.1 无模式方案

根据无模式的特点，因为 MongoDB 中同一个 collection 中的 document 没有特定的模式、列数，document 的字段都是任意的，甚至相同字段所存储的数据类型都可以不同，这个特性使得定制化更加容易。

考虑对上文 4.1.1 小节中 Extension table 里对推送表的设计，以此举个简单的例子，如果系统中有三个租户，三个租户对推送的要求各不相同，租户一想要获

得用户点击消息的时间就会添加“click_time”属性，租户二想要获得用户打开消息的地理位置就会添加“location”属性，租户三却不想要这些信息，就只需要简单的推送功能。如果我们采用关系型数据库，我们要实现上述的不同定制化，只有采用上文 4.1.1 小节中表字段留白或者扩展子表的方式实现，对于推送表这种读写并发都很大，而且数据量巨大的表采用这种设计，上文已经讨论过其危害了。

对于这一例子在 MongoDB 中实现的话，本小节就依据 MongoDB 无模式的特性，做出如下定制化设计。根据上文中的分析，本文会采用完全共享这一隔离模型，租户的数据就是由各个 collection 中的若干 document 组成的。根据应用场景，我们在消息 message_collection 中只需要在租户一的 document 里加入键值对“click_time”，在租户二的 document 里加入键值对“location”，而租户三则保持不变，维持基本的结构，所以这种定制化的方式简单的来说，就是通过有定制需求的、业务相关的 collection 中不同租户 document 键值对的灵活改变实现的。设计出的结构如图 4-9 所示：

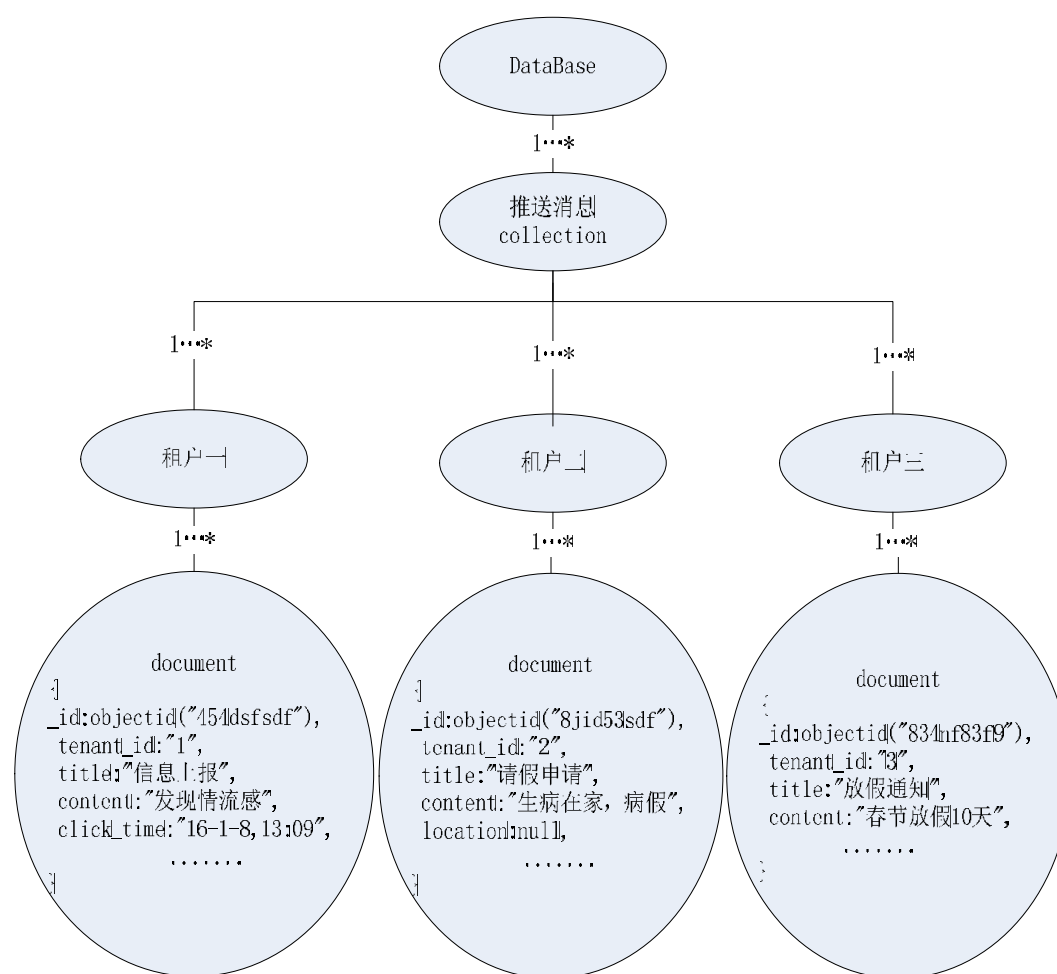


图 4-9 无模式方案数据模型图

上面的方案中，可以看出三个租户在数据项的结构上存在着不同，但是利用 MongoDB 无模式的特性，这三个租户的 document 依然能够存储于相同的 collection 中，不需要使用关系型数据库下为定制化设计的存储方案，这样的方式不仅实现简单、便于理解，而且其效率也将会提高，本文会在 6.1 节中用实验证明。

4.3.2 嵌套设计方案

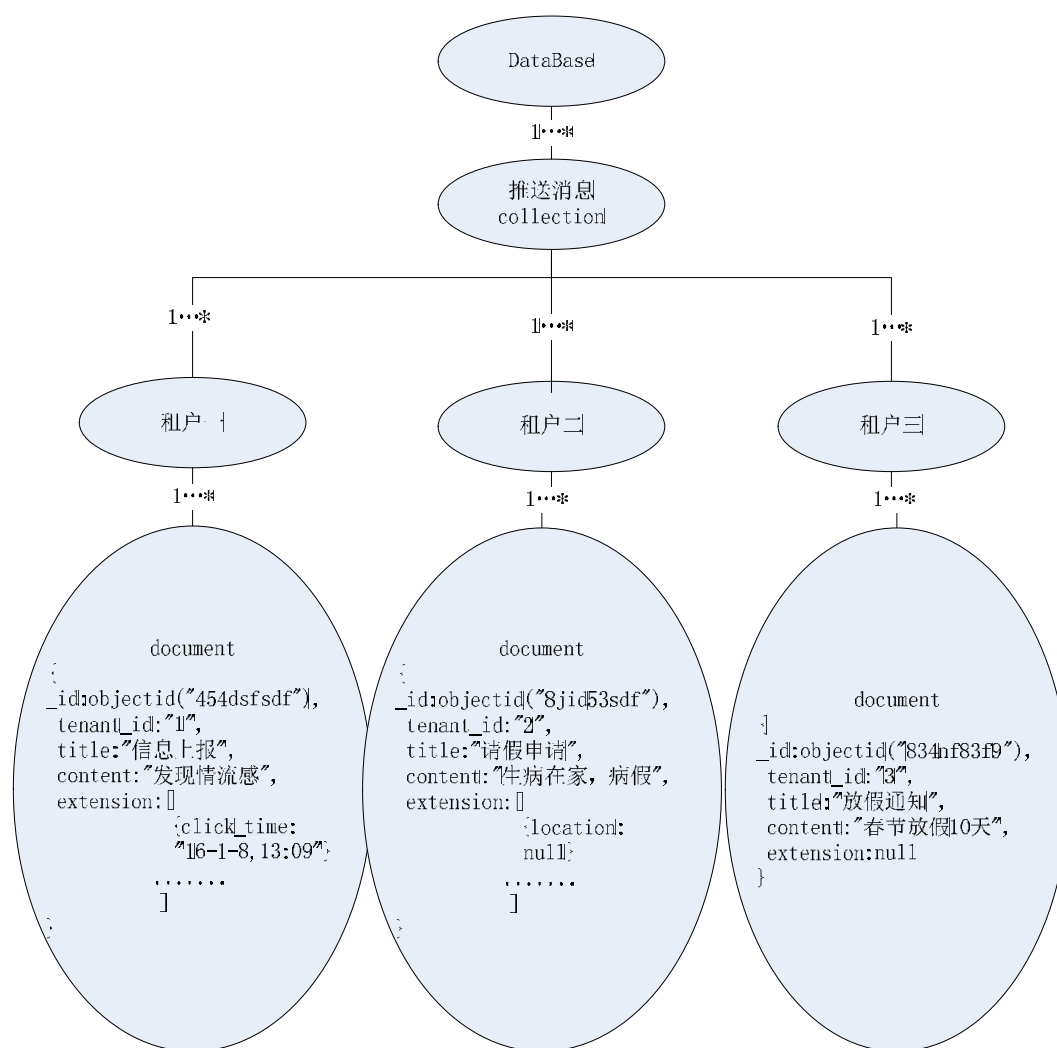


图 4-10 嵌套方案数据模型图

MongoDB 里有一个嵌入式子文档，也就是嵌套这个特性，这是 MongoDB 中解决一对多，多对多关系的一种新模式。关于嵌套的概念，简而言之就是说在 document 中，一个属性下可以存储数组或者 document，一个属性就可以包含一个或者多个其他类型的值。嵌套的好处是显而易见的：嵌套文档使得数据能在逻辑上保持其完整性，可以将一个 document 就看做一个实体对象来进行操作。在关系

式的数据库中，为了设计出符合范式的表，只能通过一对多的方式实现，然后通过外键获取数据，这对性能的影响比较大。

利用 MongoDB 这一特性，我们可以设计出一种很简单易懂的定制化方案，与无模式方案一样此方案也是建立在完全共享这一隔离模型上，而且方案的设计大体上与无模式方案相同，都是将租户抽象出来作为一种独立的对象，实体就是由各个 collection 中的若干 document 组成的。区别就在于 document 的设计上，首先每个 collection 中的 document 都会包含一个业务的公有属性，如 title、content 等属性，以及 tenant_id，然后将每个 document 都添加一个 extension 这个嵌套子文档，这个 extension 就是实现定制化的关键，我们可以将定制化的属性全部放到 extension 中去，通过这种方式来实现关系型数据库中扩展子表的效果，设计结构如图 4-10 所示。

4.3.3 本系统定制化方案选择

嵌套设计方案能够保证 collection 的结构大体一致，便于阅读，但是通过与无模式方案的对比研究发现，该方案的实用价值不高，原因有两点：

1、这种方式的查询、插入、修改操作会因为其是嵌套文档而变得臃肿，其应用层的实现代码难度也比无模式方案复杂；

2、就是查询效率问题，因为嵌套的特点，在查询时需要取出整个 extension 属性，然后再做解析，这种方式的效率肯定与无模式按需查询相比效率要低很多。所以本文就将这种方案作为一种理想的设计方案，实际采用无模式方案。

除了业务表以外，在 MongoDB 中，会建立两个特殊的 collection，用来存储元数据，一个 pub_metadata_collection 用于存储其余各个业务 collection 的公有属性，另一个 cus_metadata_collection 用于存储不同租户的定制化属性。

pub_metadata_collection 的设计简单明了，其中的 document 都包含了五个属性：_id 是每个 document 的唯一标识，coll_name 存储的是 collection 名称，attr_name 存储的是当前 collection 中属性的名称，type 是存储属性的类型，is_null 表明属性是否可以为空。pub_metadata_collection 存储了其余非元数据 collection 的所有公有存储结构，推送系统中消息 collection 在 pub_metadata_collection 中的 document 如图 4-11 所示：

Key	Value	Type
(0) {..}		Document
_id	56aec1f79b09f814e4a5ee55	ObjectId
coll_name	message_collection	String
attr_name	tenant_id	String
type	string	String
is_null	False	Boolean
(1) {..}		Document
_id	56aec2f59b09f814e4a5ee56	ObjectId
coll_name	message_collection	String
attr_name	title	String
type	string	String
is_null	False	Boolean
(2) {..}		Document
_id	56aec3059b09f814e4a5ee57	ObjectId
coll_name	message_collection	String
attr_name	content	String
type	string	String
is_null	True	Boolean

图 4-11 pub_metadata_collection 数据存储结构图

cus_metadata_collection 是用来存储各个租户中非元数据 collection 的定制化属性的，其中的 document 包含了五个与 pub_metadata_collection 相同的属性，只是每个 document 会多出个 tenant_id 属性，用于记录哪些租户需要此定制化属性。推送系统中消息 collection 在 cus_metadata_collection 中的 document 如图 4-12 所示：

Key	Value	Type
(0) {..}		Document
_id	56aec72d9b09f814e4a5ee5a	ObjectId
tenant_id	1	Int32
coll_name	message_collection	String
attr_name	click_time	String
type	string	String
is_null	True	Boolean
(1) {..}		Document
_id	56aec78b9b09f814e4a5ee5b	ObjectId
tenant_id	2	Int32
coll_name	message_collection	String
attr_name	location	String
type	string	String
is_null	True	Boolean

图 4-12 cus_metadata_collection 数据存储结构图

通过上文的设计、分析，可以看出 MongoDB 这一 NoSQL 数据库在多租户定制化这一关键技术上，与传统关系型数据库相比有着明显的优势，避免了关系型

数据库的多表关联、占用空间大、性能低效等缺点，而且其实现容易，能满足多租户的不同定制需求。

4.4 可扩展性研究与设计

4.4.1 MongoDB 可扩展性优势

MongoDB 的 sharding 机制，是依靠其特有的 Auto Sharding 机制实现的，最大的特点就是会自动地进行 sharding，具体来说有以下三个特点：

1、动态扩容

能够在数据库运行时动态地添加节点进行扩容，不影响数据库服务地使用；

2、自动平衡数据

对于数据的分布情况，MongoDB 可以自动地平衡各个节点上的数据量，使数据能够均匀地分布，分担访问压力；

3、透明的接口

对于客户端来说，MongoDB 做了 sharding 与没做 sharding 的操作是完全相同的、透明的，应用层可以以相同的形式进行操作访问，而且不会因为做了 sharding 影响数据库功能，也不用修改任何代码，这让应用层能更好的关注于自己的业务。

4.4.2 本系统的设计与实现

本论文中的多租户系统也会进行 sharding，集群的搭建及配置按照图 4-13 设计进行：

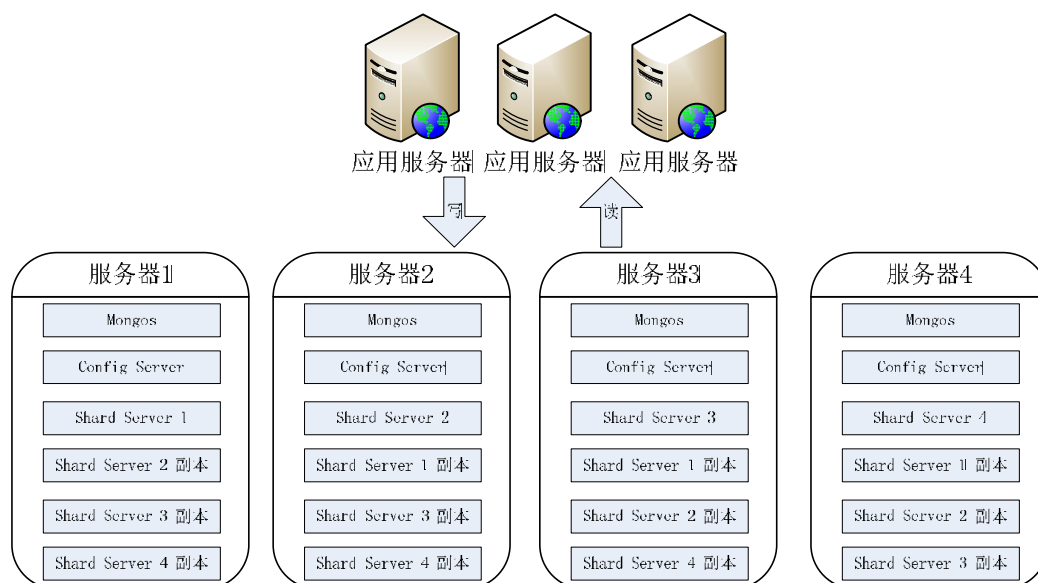


图 4-13 MongoDB 集群物理架构图

从图 4-13 中可以看出，在物理架构上，MongoDB 数据库集群会使用四台物理计算机，应用服务器会透明访问集群。每台 MongoDB 服务起上，又会分为三大部分，Mongos 和 Config Server 和四个 Shard Server，其中 Shard 又有主、副之分。对于应用服务器来说，只会和 Mongos 进行数据交互、网络通信，而不会与 Config Server 和 Shard Server 发生直接的数据交互与网络通信。

图 4-13 中 Mongos 承担着数据路由的作用，它会通过元数据来定位数据所在的 Shard Server，并且会对查询结果进行合并。除此之外，Mongos 还承担着数据迁移与自平衡的任务。在应用层开来，Mongos 的数据接口操作与单机的 MongoDB 相同，因此，可以使用相同的应用层进行访问。Mongos 只是作为一个管理操作节点，是不会记录状态的，其本身不会对任何数据，包括元数据进行存储，因此可以水平扩展出多个 Mongos，通过多个 Mongos 不仅可以分担访问压力，也能提供一定的容错能力，允许有 Mongos 发生故障。所以本文在四台服务器上都会开启 Mongos，防止其中一个挂掉所有的请求都没有办法操作。在应用层的访问代码，只需要在原来代码的基础上，改变获得 MongoClient 时的代码，其余代码不用改变，改变的代码如下：

```
List<ServerAddress> sas = new ArrayList<ServerAddress>();
ServerAddress sa1 = new ServerAddress("10.10.5.69", 20000);
ServerAddress sa2 = new ServerAddress("10.10.5.70", 20000);
ServerAddress sa3 = new ServerAddress("10.10.5.71", 20000);
ServerAddress sa4 = new ServerAddress("10.10.5.72", 20000);
sas.add(sa1);
sas.add(sa2);
sas.add(sa3);
sas.add(sa4);
MongoClient client = new MongoClient(sas);
DB db = client.getDB("push_system");
```

图 4-13 中 Config Server 用于对数据库元数据进行存储，包括数据所在的 Shard Server，以及一些数据库集群与配置相关的数据。如图 4-13 所示，本文会在四台服务器上都会开启 Config Server，这四个 Config Server 中的数据是由 MongoDB 自身进行同步的，Mongos 可以访问其中任意一个。这样的冗余设计也是为了进一步

加强高可用性，在一个或两个 Config Server 发生故障时，也能保证数据库服务的正常使用。

图 4-13 中 Shard Server 就是用于存储数据的实体，本文为了实现更高的可用性，会对于每一个 Shard Server 都创建一个 replica set 副本集，通过冗余的方式来保证 Shard Server 的高可用性，避免单个 Shard Server 出故障时，会导致数据库服务受影响与丢失数据。如图 4-13 所示，本文会对四个 Shard Server 各建立三个副本集，设计三个副本集的好处是既能在主 Shard Server 出现问题时，奇数个副本集可以通过选举产生新的主 Shard Server，避免使用仲裁集，又能获得最高的性价比。

按照本文设计完成物理搭建，然后通过配置，就可以快速实现数据库集群的高可用性、高扩展性，就可以让本文中的多租户推送系统在其上运行，很轻易的就消除了单一数据库的瓶颈，而且集群的扩展可以做到无痛扩展，加入新服务器时不会停止服务，也不会有明显的性能下降，能够不断满足发展的租户数量以及业务。

4.4.3 MongoDB Auto-Sharding 机制优化方法

本文除了完成上述可扩展设计，还会在此基础上对 MongoDB 的 Shard Server 间的数据块转移进行研究，提出了一种基于数据块增删查改频率的负载均衡方法，并已将其发表为专利。

这个题目是源于对 MongoDB 的 Auto-Sharding 用于多租户推送系统时的不合理性，因为推送系统会向第三方提供推送服务，但是没有向其提供 Web 推送管理服务，也是就说第三方不能使用推送统计、报表查询等功能，简单的说就是第三方推送数据所承受的读写压力小于普教云租户的数据，数据块的读写压力就与存在其中的数据相关，所以就有了本方法的研究内容。

这个方法可以克服 MongoDB 现有自动分片机制在负载均衡方面的问题，根据 MongoDB 的实际上的数据使用情况，依据各个数据块的读写频率，动态地进行负载均衡，以达到优化数据库性能的目的。

首先需要介绍下 MongoDB 中一种名为 Auto-Sharding 的机制，即自动分片（分片，是指存储数据的逻辑上特定大小的块，服务器通过管理分片来管理数据）的机制，用于来实现 MongoDB 的自动水平扩展（水平扩展，是指通过加入更多的服务器到集群中，通过将数据分布到多个服务器上，这样就不再需要高性能主机来存储海量数据，承受高压负载，使用更多廉价机提供更多的存储空间或者更高的运算性能）。与其它传统数据库有所区别的是，水平扩展会由 MongoDB 提供的 Auto-Sharding 机制自动地完成其全部过程，不需要进行人工操作，也不需要编写

相关程序，而且可以自动平衡各个服务器上的分片。

通过查阅资料以及查看源码可知，MongoDB 中的 Auto-Sharding 机制是靠一个 balancer 的模块来实现的，控制数据在集群里的各个服务器上的转移，并使之均匀分布，它是一个在后台运行的任务，当任意两个服务器中的数据块的数量之差值达到系统设定的最高阈值时，balancer 就会将数据块从数据块多的服务器移动到数据块少的服务器上，直到数量之差达到最低阈值，图 4-14 是移动之前的数据块分布情况，图 4-15 是移动之后的数据块分布情况。

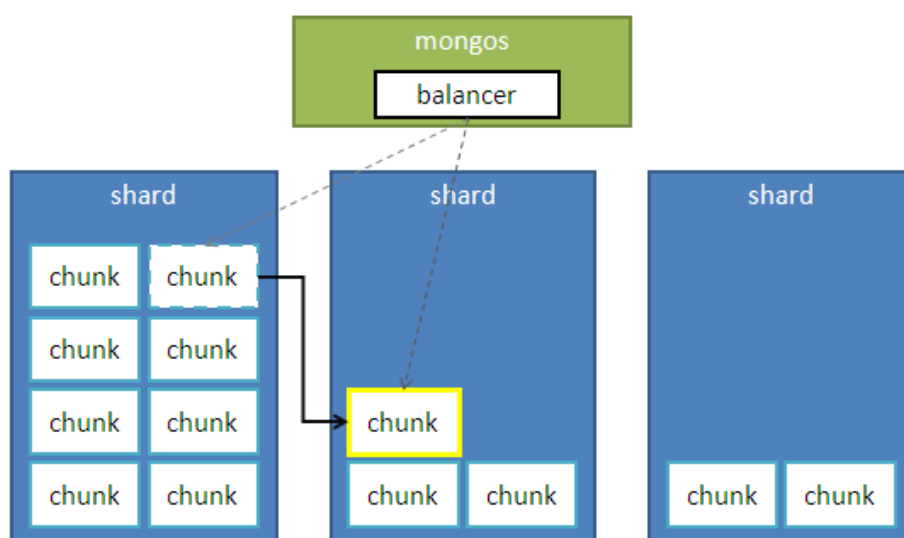


图 4-14 移动前数据块分布情况

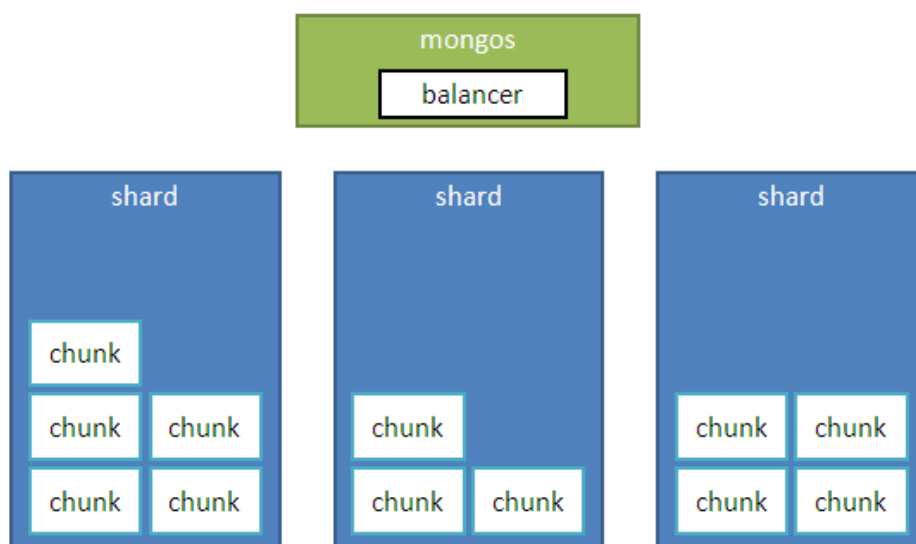


图 4-15 移动后数据块分布情况

通过研究可知, balancer 这一模块中关于数据块移动的算法, 仅仅是依各个服务器上数据块的总量为依据, 来平衡各个服务器上的数据块。这种算法可以使得数据块的移动量最小, 但是选取的数据块都是块键(块键是指每个数据块的唯一标识)最小的。并没有将实际使用情况考虑进去, 因为各个数据块的使用情况大有不同, 有的基本不会访问, 有的查询量大, 有的修改量大, 而查询、修改不同操作对服务器性能的消费是有差别的, 使得各个服务器的负载并不能做到实际上的完全均衡。

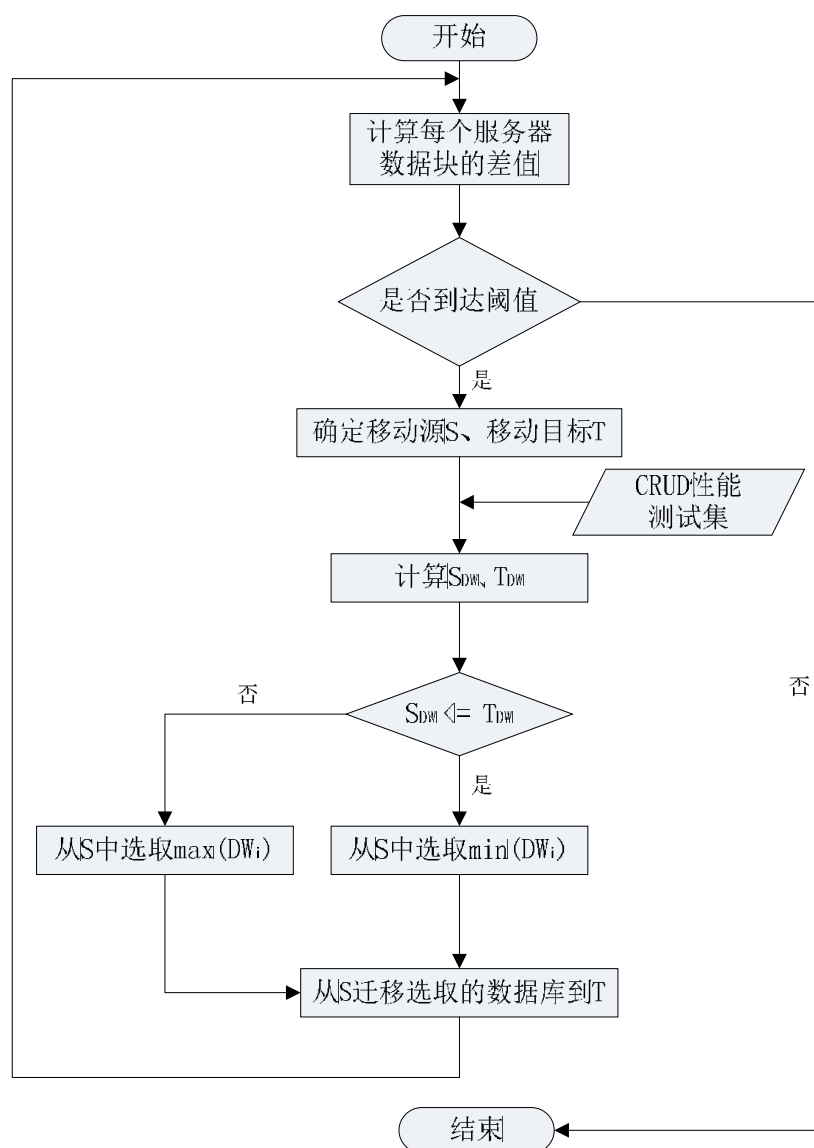


图 4-16 负载均衡优化方式实施流程图

根据上文中 MongoDB 在自动分片机制上的不足之处, 提供了一种基于数据块增删查改频率的负载均衡方法, 以优化 MongoDB 的负载均衡能力, 能有效的提高

MongoDB 数据库的读写性能，使数据库的负载更均衡，减轻个别服务器压力。本方法的关键点就是在于对移动数据块的选择上，将数据块的 CRUD（CRUD 代表对数据库的四种操作，即增删改查）负载考虑进去，可以使得集群不仅仅是在数据块的数量上实现了负载均衡，而且也实现了读写性能上的负载均衡，方法的具体实施步骤如图 4-16 所示。

本优化方法的实施过程如下：

步骤一：在 MongoDB 中建立 CRUD 性能测试集，并且编写执行性能测试的脚本，用于计算出系统的运行各个 CRUD 操作的性能参数。

具体方法：根据系统实际应用场景，从数据库日志中选择插入、删除、修改以及查找语句各 5000 条，作为测试语句，分别记录在 4 个语句脚本中。再建立一个定时（可以以 12 小时为周期）执行 shell 脚本，分别记录下 4 个语句脚本的执行时间作为性能参数如下：插入操作性能参数 T_C 、删除操作性能参数 T_D 、修改操作性能参数 T_U 以及查询操作性能参数 T_R ，因为性能参数是各个脚本的执行时间，所以 T 越大效率越差。周期性的更新性能参数，可以反映出系统不同时间段各个操作执行的性能效率，供后续步骤使用。

步骤二：需要修改 MongoDB 的 config server 中记录数据块参数的数据结构，增加五个变量，分别是：四个操作变量 operateC 代表数据块插入操作的次数、operateR 代表数据块查询操作的次数、operateU 代表数据块更新操作的次数、operateD 代表数据块删除操作的次数以及一个数据块操作量变量 datablockWeight。

每次对数据库进行操作时，都会在相应的操作变量上进行加一操作，用于后续步骤计算使用。数据块操作量变量 datablockWeight 会在每当进行负载均衡时计算得出。负载均衡后会对五个变量进行置零操作，这样能够及时地、时效地反映出各个数据块操作情况。

步骤三：然后按照如下算法，在 MongoDB 源码中加入一个方法来计算每个数据块的 datablockWeight 并且保存到 config server 中。具体步骤如下，我们假设一个服务器上有 n 个数据块，每个数据块可以用 K_i （ $1 \leq i \leq n$ ）表示，服务器上所有数据块操作次数之和用 O_n 表示，每个数据块总的操作次数用 O_i 表示，即公式(4-1)所示。

$$O_n = \sum_{i=1}^n O_i \quad (4-1)$$

用 O_{Ci} 、 O_{Ri} 、 O_{Ui} 、 O_{Di} 分别表示第 i 块数据块插入操作的次数、查询操作的次数、更新操作的次数以及删除操作的次数（来自于步骤二中 operateC 数据块插入

操作的次数、operateR 数据块查询操作的次数、operateU 数据块更新操作的次数、operateD 数据块删除操作的次数), 即公式(4-1)可表示为公式(4-2)。

$$O_n = \sum_{i=1}^n \sum_{j=\{c, r, u, d\}} O_{ji} = \sum_{i=1}^n O_{Ci} + \sum_{i=1}^n O_{Ri} + \sum_{i=1}^n O_{Ui} + \sum_{i=1}^n O_{Di} \quad (4-2)$$

用 DW_i 表示第 i 块数据块操作量, 可以由步骤一中所得到的各个操作的性能参数(插入操作性能参数 T_C 、删除操作性能参数 T_D 、修改操作性能参数 T_U 以及查询操作性能参数 T_R)以及公式(4-2)共同计算得出, 即公式(4-3)。

$$DW_i = \sum_{j=\{c, r, u, d\}} O_{ji} T_j = O_{Ci} T_C + O_{Ri} T_R + O_{Ui} T_U + O_{Di} T_D \quad (4-3)$$

用 W_n 表示服务器上所有数据块操作量, 即可得出公式(4-4)。

$$\begin{aligned} DW_n &= \sum_{i=1}^n \sum_{j=\{c, r, u, d\}} O_{ji} T_j \\ &= \sum_{i=1}^n O_{Ci} T_C + \sum_{i=1}^n O_{Ri} T_R + \sum_{i=1}^n O_{Ui} T_U + \sum_{i=1}^n O_{Di} T_D \end{aligned} \quad (4-4)$$

每次负载均衡时按照上式计算出服务器中各个数据块的 DW_i 然后存入变量 datablockWeight 中。 DW_n 这一公式供后续步骤使用。

步骤三: 修改 MongoDB 中 balancer 模块的 BalancerPolicy 方法, 它是 balancer 模块中用于实现负载均衡策略的, 其中有个 pickChunk()方法用于选择服务器中需要移动的数据块。本发明的关键就在于对其的优化。具体算法步骤如下, 当任意两个服务器中的数据块的数量差值达到系统设定的最高阈值时, 此时会进行负载均衡。包含数据块多的服务器称为数据块移动源, 用 S 表示, 包含数据块少的服务器称为数据块移动目标, 用 T 表示。

移动步骤如下: 移动数据块前都会先计算 S 与 T 的平均数据块操作量分别用 S_{DW} 和 T_{DW} 表示, 用 n_S 和 n_T 表示 S 与 T 中数据块的数量, 用 SDW_n 和 TDW_n 表示 S 与 T 服务器上所有数据块操作量, 分别用公式(4-5)与公式(4-6)进行计算。

$$S_{DW} = \frac{SDW_n}{n_S} \quad (4-5)$$

$$T_{DW} = \frac{TDW_n}{n_T} \quad (4-6)$$

如果 S_{DW} 大于 T_{DW} , 则从 S 中选取 DW_i 最大的数据块进行移动, 如果 S_{DW} 小于

T_{DW} ，则从 S 中选取 DW_i 最小的数据块进行移动。

一次移动完成后，重复执行移动步骤，直到 S 与 T 的数据块的数量 n_S 和 n_T 差值达到最低阈值时，则完成一次负载均衡。

只需在 MongoDB 源码中根据上述步骤进行简单的修改，就可以实现这一基于数据块增删查改频率的优化效果。

4.5 数据安全研究与设计

这里所说的数据安全问题，是指 MongoDB 环境下，多租户间的数据安全问题，我们不仅仅要隔离多租户间的数据，而且要阻止恶意租户访问其它租户的数据^[27]。

要解决数据安全问题，一般有两种方式，第一种通过 DBMS 进行权限限制，第二通过应用层限制。对于第一种方式，在 MySQL 中可以提供行等级的权限控制，就可以很轻松实现完全共享模式下的权限控制，但是目前 MongoDB 只提供了 DataBase 层面的权限控制，无法满足完全共享模式下实现 document 级的权限控制，所以我们只能依靠应用层进行权限控制。

因为我们的应用层是基于 spring 框架开发的，已经引入了 spring-data-mongodb 这一依赖包，对其整合配置等前期工作不再累赘叙述，本文只对在 Spring 上实现 MongoDB 多租户数据安全控制进行详细分析说明。我在进行此项目开发时，为解决此问题，实现了通用的方式，以下是该实现方式的核心部分介绍。

此方式是以 Spring 的 SpEI 为基础，SpEI 是 Spring Expression language 的简写，它使得程序能够在运行时构建复杂表达式，存取对象属性、对象方法调用等。我通过 SpEI 与注解相结合进行权限过滤。首先我们需要一个组件来提供当前访问用户的租户 ID，其代码如下：

```
@Component("tenantProvider")
public class TenantProvider {
    public String getTenantId() {
        //从 session 中获得用户 TenantId
        return request.getAttribute("tenantId");
    }
}
```

这段代码是组件库里的一段代码，其作用就是提供当前租户的 ID。实现很简单，就是通过 session 来获取当前用户的租户 ID，在此基础上，我们可以通过 SpEI 实现应用层的安全访问控制，以我们的 message_collection 为例，我们就可以使用

这个组件在运行时通过 SpEI 表达式构建出我们需要的 Domain，其 Domain 类关键代码如下：

```
@Document(collectionName = "message_collection")
public class Message {
    @Id
    private String id;//document主键
    //通过注释加 SpEI 表达式过滤查询结果
    @Filed(value="tenant_id",def="{tenant_id':#{tenantProvider.getTenantId()}}")
    private String tenantId;

    .....//略去其他属性以及get、set方法
}
```

通过 “#{tenantProvider.getTenantId()}” 这个 SpEI 表达式和注释，可以在程序运行时动态的使用租户的 tenant_id 来控制数据权限，可以做到限制当前租户只能访问到 collection 中 document 的 tenant_id 与自己相同的数据。

这种方式对其他业务是透明的，其他业务只需要按普通方式使用 Message 对象，但是在不同租户用户的 session 里，每个 Message 对象已经做了过滤，都是当前租户的 Message 对象，不会影响到其他租户。而且这种模式可以有效的防治恶意用户的一些简单攻击，因为用户的 tenant_id 是通过 session 取到，在登录时已经进行过验证，不会出现正则表达式这种以匹配方式的攻击，还有就是通过注释的方式可以避免 “tenant_id = 234546 or 1 = 1” 这种 SQL 注入攻击。

4.6 系统数据库设计

本小节会在上文的研究基础上，以及根据需求分析对系统 MongoDB 数据库进行设计。

4.6.1 MongoDB 数据库设计原则

设计 MongoDB 数据库的方式与传统关系型数据库的设计方式有很大的区别，在设计一个关系型数据库的时候，设计的重心与难点都是集中在表该如何合理的划分与处理表之间的相连关系，而且数据库的设计都是在数据库范式的指导下进行的，限制条件较多。相对的构建 MongoDB 数据库时，就没有什么所谓的数据库设计范式，类似于表由多条记录组成，collection 也包含多个 document 对象，但是

在 MongoDB 中同一个 collection 内的 document 对象不必具有相同的属性，就是所谓的无模式，所以 MongoDB 数据库的设计是很灵活、很自由的，不用考虑太多约束条件。

在设计本系统 MongoDB 数据库时遵循以下原则：

- 1、collection 越少越好，尽量把关系数据库中分表表示的关系嵌套进文档里，使其满足“复杂写，简单读”；
- 2、希望以后做什么样的查询，就怎样设计 collection，数据的结构要去适应程序的读写场景；
- 3、一对多关系使用在父文档中内嵌一个数组子文档的方案；

4.6.2 系统数据库中 collection 设计

在 MongoDB 中建立一个 push_system 的数据库，其中 collection 结构如下图 4-17 所示：

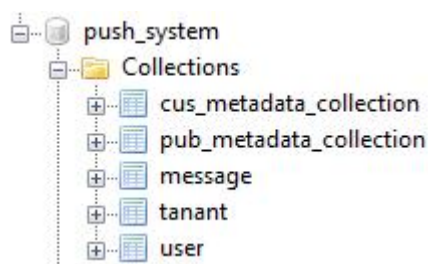


图 4-17 MongoDB collection 结构图

依据 MongoDB 数据库设计遵循的原则，collection 要尽可能的精简，所以一共设计了五个 collection，以下是对 5 个 collection 的详细分析：

1、cus_metadata_collection

这个 collection 与 pub_metadata_collection 都是属于系统的元数据 collection，元素据主要是在应用层操作其他 collection 时提供支持，让应用层能够对不同租户多样的 document 进行读写操作，cus_metadata_collection 用于存储不同租户的定制化属性，其设计如下表 4-1 所示：

表 4-1 cus_metadata_collection 设计

名称	含义	类型	备注
_id	document 标识	ObjectId	是每个 document 的唯一标识
tenant_id	租户 ID	String	记录哪些租户需要此定制化属性
coll_name	collection 名	String	存储的是 collection 名称

attr_name	属性名	String	存储的是当前 collection 中属性的名称
type	类型	String	存储属性的类型
is_null	为空标记	Boolean	表明属性是否可以为空

2、pub_metadata_collection

这个 collection 用于存储各个 collection 的公有属性，其设计如下表 4-2 所示：

表 4-2 pub_metadata_collection 设计

名称	含义	类型	备注
_id	document 标识	ObjectId	是每个 document 的唯一标识
coll_name	collection 名	String	存储的是 collection 名称
attr_name	属性名	String	存储的是当前 collection 中属性的名称
type	类型	String	存储属性的类型
is_null	为空标记	Boolean	表明属性是否可以为空

cus_metadata_collection 与 pub_metadata_collection 这两个 collection，因为其是属于元数据，所以其不会设计到 MongoDB 的无模式的特性，存储的 document 都是会具备所有属性。

3、tenant

tenant 这个 collection 存储的租户的相关信息，其设计如下表 4-3 所示：

表 4-3 tenant collection 设计

名称	含义	类型	备注
_id	document 标识	ObjectId	是每个 document 的唯一标识
tenant_id	租户 ID	String	存储的是租户的 ID
name	租户名	String	存储的是租户名称
admin_ids	租户管理员	String 数组	有权管理租户推送的用户集
push_type	推送类型	int	推送方式，共两位数，第 1 位代表移动推送，第 2 位代表短信推送
need_times	总共需要次数	int	消息总共需要被推送多少次
p_available	推送是否可用	Boolean	租户推送功能是否可用
c_available	统计是否可用	Boolean	租户统计报表功能是否可用
push_total	推送次数	int	租户推送的次数，不是推送消息数
certificate_name	IOS 证书名	string	IOS 推送证书的名称
certificate_psw	IOS 证书密码	string	IOS 推送证书密码

对于 `tenant` 这一 `collection` 来说，其中的 `document` 不会具备以上所有属性，`push_total`，`admin_ids` 就是属于定制属性。`admin_ids` 这一属性目前是普教云平台租户专有的，是个 `String` 数组，在数组中存储的是可以使用推送 Web 推送管理模块的用户，因为推送 Web 推送管理模块是作为一个小模块集成到普教云平台中的，使用了单点登录，用户不用输入密码就能进入推送 Web 推送管理模块。`push_type` 这一属性基于二进制的思想，比如 10 就代表只使用移动推送，11 代表移动和短信推送同时进行。

4、user

`user` 这个 `collection` 存储的是与用户相关信息，其设计如下表 4-4 所示：

表 4-4 user collection 设计

名称	含义	类型	备注
<code>_id</code>	document 标识	<code>ObjectId</code>	是每个 document 的唯一标识
<code>user_id</code>	用户 ID	<code>String</code>	存储的是用户的 ID
<code>name</code>	租户名	<code>String</code>	存储的是用户名称
<code>tenant_id</code>	租户 ID	<code>String</code>	用户所属租户
<code>mobile_type</code>	移动端类型	<code>int</code>	0 是苹果，1 是 Android
<code>deviceToken</code>	苹果标识	<code>String</code>	苹果推送需要的唯一标识
<code>telephone</code>	电话	<code>String</code>	短信推送使用的电话号码
<code>available</code>	是否可推送	<code>Boolean</code>	用于人为控制用户是否可推送

对于 `user` 这一 `collection` 来说，其中的 `document` 不会具备以上所有属性，`mobile_type`，`deviceToken`，`telephone`，`available` 就是属于定制化属性。因为不同租户对上述字段要求是不同的，有的租户是不需要短信推送的，就不需要 `telephone` 这一字段；有的租户是只有短信推送的，就不需要 `mobile_type`，`deviceToken` 字段；有的租户不需要对用户推送进行控制，就不需要 `available` 字段。

5、message

`message` 这个 `collection` 存储的是与推送消息相关的信息，其设计如表 4-5 所示。

可以看出 `message` 这个 `collection` 的属性很多，但是 `push_times`、`push_date`、`need_times`、`location`、`click_time` 都是各个租户的定制属性，因为 MongoDB 无模式的特性，这些属性可以随意按照租户需求进行组合。

由推送系统需求可知，`message` 这个 `collection` 是系统中最大的 `collection`，而且会随着系统规模的扩大、使用时间的增加而不断的扩大，所以对于 `message` 这个 `collection`，我们需要对其进行 `sharding`，实现 `collection` 自动的横向扩展和读写负

载均衡。

表 4-5 message collection 设计

名称	含义	类型	备注
_id	document 标识	ObjectId	是每个 document 的唯一标识
message_id	消息的 ID	String	存储的是消息的 ID
tenant_id	租户 ID	String	存储的是租户的 ID
title	消息标题	String	推送消息的标题
content	消息的内容	String	推送消息显示内容
user_id	用户 ID	String	消息推送的目标用户 ID
push_times	消息推送次数	int	消息需要被推送多少次
push_date	推送时间	String	记录最后一次推送的时间
need_times	总共需要次数	int	消息总共需要被推送多少次
status	推送消息的状态	Boolean	推送状态, true 代表已推送, false 代表未推送
location	打开消息位置	String	用户点击消息时的地理位置坐标反馈信息
click_time	打开消息时间	Date	用户最后一次点击消息时的时间反馈信息

4.7 MongoDB 相关研究成果

本论文题目确定后, 根据 MongoDB 实际使用情况, 本人在导师以及公司架构师的指导下对 MongoDB 做了优化研究工作, 完成了两篇与之相关的研究专利:《一种基于 MongoDB 的负载均衡优化方法》与《一种基于 MongoDB 的数据分页优化方法》。

《一种基于 MongoDB 的负载均衡优化方法》就是 4.4.3 小节中对 MongoDB Auto-Sharding 机制进行优化的方法, 本小节将会对《一种基于 MongoDB 的数据分页优化方法》进行简要介绍。

在项目中经常会使用分页这一功能, 而一般情况下 MongoDB 中是使用一种基于 skip 函数的方法进行数据分页。这种方法是一种很通用的方法, 大多数系统都采用这种方式, 其在数据总量较少时, 分页查询的效率较高, 查询尾页的响应速

度也很快，但是在数据总量变得很大的以后，就会因为 skip 操作的效率问题，使得尾页查询会变得很慢。

首先需要介绍下基于 skip 函数这一方法，其具体操作步骤比较简单，首先根据查询条件，会使用 find 函数（find 函数：可以根据查询条件对 MongoDB 中的某个 collection 执行 document 查找）查找，并返回 result-collection，然后就会根据单个页面数据量以及当前页数，作为数据的偏移量，根据偏移量使用 skip 函数（skip 函数：用于设置返回记录的起点，从跳过前面的 document，从起点开始返回 result-collection）跳过当前页面前的所有 document，从而可以设置用户需要查询的页面的首项为起点，最后，用 limit 函数（limit 函数：从 result-collection 中获得特定数量的 document）取出本次用户请求分页中所包含的 document。根据描述我们可以定义其通用公式(4-6)，其中 A 代表查询条件，n 代表页数，N 代表一页的数据量：

$$\text{db.collection.find}(\{A\}).\text{skip}((n-1)*N).\text{limit}(N) \quad (4-6)$$

可以看出，这种方式来做分页查询是很简单方便的，非常容易实现，所以这是现在大多数人的首选。但是基于 skip 函数这方法在数据量很少的情况下的性能很有高，但随着数据量的增大，尾页的分页数据查询性能就越低。当数据量很大的时候，对尾页数据进行查询时，可以使用 explain（explain：MongoDB 中用来查看查询的过程的一个命令）命令得出查询过程的 nscanned（nscanned：这个值表示当前这次查询一共扫描了 collection 中多少个 document），可以发现在查询尾页数据时，会扫描过全部符合条件的 document，相当于顺序遍历到最后，举个例子，如果需要查询有 1000000 个 document 的 collection 中的第 999981-999990 这 10 条数据，根据上面公式，会使用 skip(999980)，所以 MongoDB 就会扫描 999980 条 document，这就会导致查询性能的大幅度降低。

从上文中可以看出，如果在海量的数据环境下，当要查询的分页的 document 在排序的结果集后面时，需要 skip 的 document 量会很大，很可能会达到几万、十几万，甚至上百万。

所以在这种情况下我们需要避免使用 skip 函数，对 MongoDB 分页技术进行改良，从而使 MongoDB 在海量数据分页查询效率提升，提高用户体验。

本文针对 MongoDB 在上述问题上的不足之处，提出基于索引数组的数据分页方法。索引数组方法的大体思路是，因为我们查询的所有 document 都是会根据关键字排序的，所以首先将满足条件的每条 document 的关键字全部取出排序，注意这里是已经排序了的（假设按从小到大排序），然后将它们存放在一个数组当中，假如有 1000 条 document，就会建立一个大小为 1000 的数组，然后因为会在这个

关键字上建立索引，所以当我们进行分页查询的时候，假设一页会显示 10 条 document，要查询第 100 页的 document，也就是第 991-1000 条 document，我们就可以在数组中取出第 990 或第 991 个关键字，然后就可以直接使用这个关键字加上 where 函数，因为又有索引，就可以使用条件查询直接找到比这个关键字大的 10 条 document。这个方式的核心思想就是通过索引数组进行直接定位到查询页面的第一个或上一个关键字，然后可以直接取出需要的 document，不用遍历不需要的 document。需要说明的是，如果查询的 document 没有使用关键字排序，我们可以使用 MongoDB 中每条 document 都有的 ObjectId 来作为关键字，在海量数据下，查询尾页的速度就会很快。

本优化方法的实施过程如下：

步骤一：将满足条件的每条 document 的关键字全部查询出来并按照分页排序的顺序排序（没有关键字用 ObjectId），然后将读取到的各个 document 关键字字段的值，按照据排序的顺序存储到数组中，此数组将在当前用户会话期间常驻内存，这个过程在基于 skip 函数这一方法中是没有的，这里会有额外的时间消耗，但是在海量数据环境下，这点消耗是一劳永逸的；

步骤二：根据数组长度（也就是总 document 数量）和每页显示的 document 数量计算符合用户请求条件的数据总页数，并默认显示数据的第一页；

步骤三：根据用户查询的页数，定位关键字值，通过索引直接查询用户请求的页面的 document，然后显示该页。具体做法是根据页码记录和数组下标的对应关系，算出数据偏移量，比如一共 1000 条数据，一页 10 条数据，假如我们要查询第 25 页的 10 条数据，就需要找出 24 页最后一个关键字在数组中的位置，就是位于第 240，此时的偏移量就是 240。数据偏移量的值作为关键字数组的下标，找到上一页最后一条 document 或者本页第一条 document 的关键词作为数据标记，我们就可以使用这个数据标记为基础，构建新的查询，例如我们关键字按照升序排列，有了数据标记 A，查询特定一页的 document 就可以直接使用条件查询（条件查询就是用 where 这个函数，后面跟查询条件）where 关键字 > A，因为索引的原因，可以直接定位到数据标记 A 处，然后用 limit 函数从结果集选取一页的数据即可，从而避免使用 skip 函数，提升了大数据量下的查询效率。

4.8 本章小结

本章主要以 MongoDB 多租户数据层研究与设计内容为主，首先会通过对关系型数据库存在的问题进行分析，给出为什么使用 MongoDB 做为多租户系统数据层的原因。然后会在 MongoDB 下完成独立数据库、共享数据库独立结构、共享数据

库共享结构三种数据隔离模型的研究、设计、分析工作。接着从多租户四项设计特性出发，包括数据隔离性、定制化、可扩展性、数据安全，在 MongoDB 上进行研究设计。最后介绍了 MongoDB 的两项专利《一种基于 MongoDB 的负载均衡优化方法》与《一种基于 MongoDB 的数据分页优化方法》。

通过本章，可以得出结论：MongoDB 作为多租户系统数据层是非常具有优势的，对于多租户设计的四个特性，MongoDB 相对于传统关系型数据库都有着更为良好的表现，所以可以论证本文基于 MongoDB 设计多租户系统的观点是正确的、合理的。除此之外，也为应用层的设计做好了良好的铺垫工作。

第五章 系统应用层的设计与实现

本章是本文的重点章节，首先会根据需求分析对系统做整体设计，然后会对系统中各个模块进行详细的设计与实现，包括 Android 平台客户端、服务端，IOS 平台客户端、服务端，短信推送模块、Web 推送管理模块、第三方推送服务调用模块。

5.1 系统整体设计

5.1.1 系统物理架构

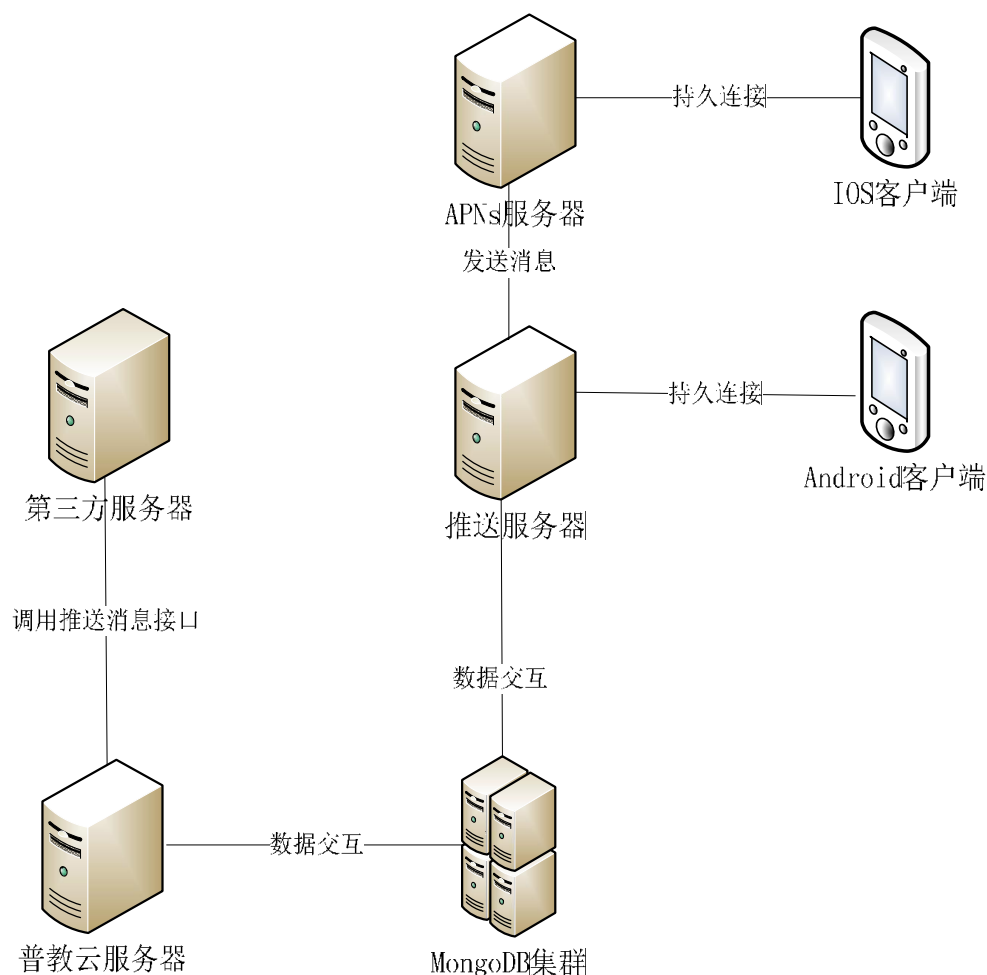


图 5-1 系统物理架构图

从物理架构图 5-1 可以看出，推送服务器只负责从 MongoDB 集群获取需要推送的消息，然后与 Android 客户端建立起持久连接，进行消息推送；对于 IOS 客

户端来说,只需将推送消息发送到 APNs 服务器即可完成推送;如果第三方服务器需要发送消息,只需要调用普教云平台提供的接口;普教云平台会完成与 MongoDB 集群的数据交互,提交需要推送的消息,而且 Web 推送管理平台也是运行在普教云平台上。

5.1.2 系统模块设计

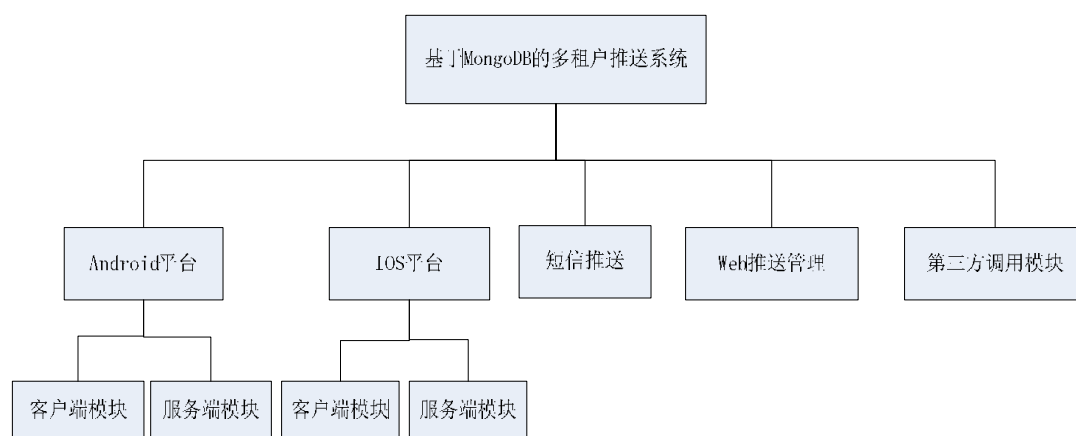


图 5-2 系统模块设计图

系统一共分为五个大模块：Android 平台推送模块、iOS 平台推送模块、短信推送模块、Web 推送管理模块、第三方调用模块。而 Android 平台推送模块、iOS 平台推送模块分别又分为客户端与服务端两个部分。本文接下来会详细地介绍各个模块的设计与实现。

5.2 IOS 平台模块设计与实现

根据前文 2.3.3 小节的介绍，iOS 平台的推送服务会依赖于苹果公司提供的推送服务。本节将对其设计与实现做详细介绍。

5.2.1 准备工作

虽然本文是作为一个基础设施系统,但是对于 iOS 平台的推送,因为其特殊性、封闭性,每个 iOS 平台上的 APP 都需要开发者自己在苹果开发者网站上根据步骤进行创建、配置,并申请开通 Push Service,并且创建 Push 需要的证书 APNs Push Certificate,这一证书与应用一一对应,其作用是用于在与 APNs 建立连接时作为非对称密钥。开发者需要将其应用的 APP ID 与证书提供给我们系统,并在其应用中加入我们提供的.h 头文件与.a 静态库文件。

5.2.2 推送流程

完成了上述准备工作，我们就可以实现 IOS 平台的消息推送了，其推送的总体流程如下图 5-3 所示：

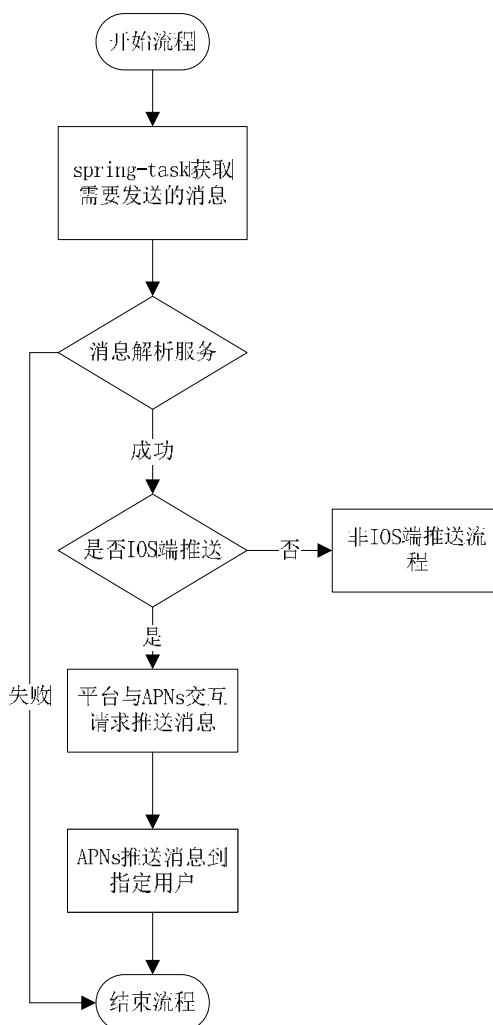


图 5-3 IOS 平台推送流程图

流程中各个步骤的详细设计如下：

- 1、用 spring-task 定时扫描 message collection，消息来自于 MongoDB 中；
- 2、对消息进行解析，消息解析主要完成的工作是通过 tenant_id 查询 tenant collection 中的 p_available，看租户是否开启推送功能；查询 tenant collection 中的 push_type，得到租户的移动推送和短信推送的组合方式；如果有 IOS 端的消息需要推送，通过 user_id 查询 user collection 中的 deviceToken，成功后，就会执行 IOS 平台的推送流程；
- 3、IOS 推送模块会将推送应用的 ID、推送的用户、消息内容以 JSON 格式发

送给 APNs 服务器;

4、只要推送模块完成了与 APNs 服务器的交互, APNs 服务器就能提供可靠的推送机制, 并保证会将消息推送到目标客户端, 完成后流程就结束。

5.2.3 关键部分设计与实现

IOS 平台上实现推送的关键设计实现部分有两项, 客户端绑定 deviceToken 以及 IOS 推送服务端的实现方式。

5.2.3.1 绑定 deviceToken

对于 IOS 端的推送, 在客户端, 最关键的部分就是需要提供 deviceToken 与登录的用户 user_id 的绑定信息。在我们提供的.a 静态库文件中, 会有特定的代码可以获取 deviceToken, 每个 deviceToken 都是全球唯一的。对于第三方开发者来说, 要开发 APP 并使用推送功能, 就需要通过.a 静态库文件中的函数获得设备的 deviceToken, 并将自己的租户 tenant_id, 登录后的用户 user_id 一并传送到推送服务器上。

获取 deviceToken 的方式通过一个函数, 可以在 APP 第一次运行时就获取该设备的 deviceToken, 并将获取的 deviceToken 保存在 APP 数据中, 然后就可以通过代码随时获取 deviceToken, 关键代码如下:

```
(void)application:(UIApplication*)application
didRegisterForRemoteNotificationsWithDeviceToken:(NSData*)deviceToken{
    NSLog(@"My Token is %@",deviceToken);
    //转换成 string
    NSString* dvsToken=
    [[deviceTokendescription]stringByTrimmingCharactersInSet:[NSCharacterSet
characterSetWithCharactersInString:@"<>"]];
    NSLog(@"My dvsToken is %@",dvsToken);
    //保存 deviceToken 到本地
    NSMutableDictionary* dvsTokenDictionary = [NSMutableDictionary
dictionary];
    //将 dvsToken 存入本地, 命名为 dvsToken
    [dvsTokenDictionary setObject:dvsToken forKey:@"dvsToken"];
    [dvsTokenDictionary writeToFile: DEST_PATH_DeviceToken atomically:YES];
}
```

5.2.3.2 推送服务端

因为苹果公司对于安全性、可靠性的要求是很高的，所以会要求与其 APNs 服务器相连接的服务器需要通过传输层安全（TLS: Transport Layer Security）来建立稳定的、安全的连接，其建立连接建立过程如下图 5-4 所示：

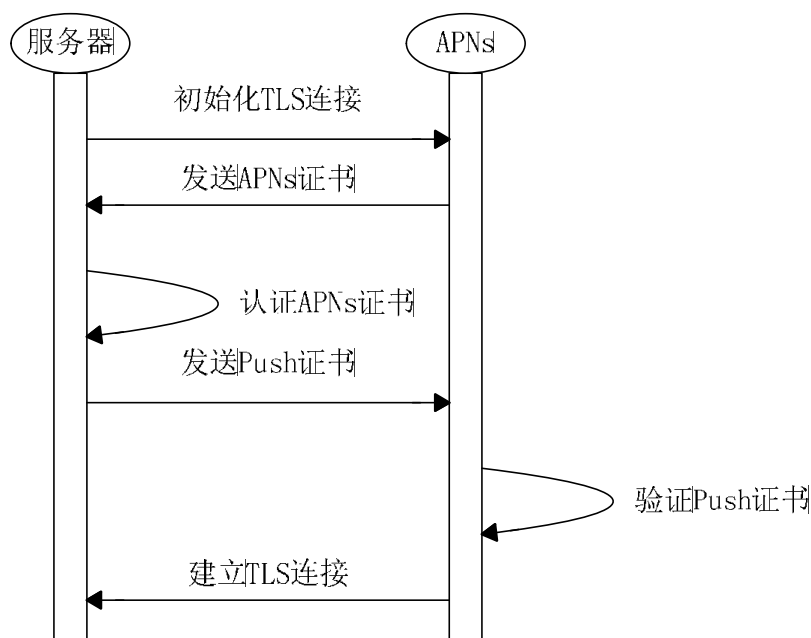


图 5-4 APNs 服务器建立连接序列图

从上图 5-4 可知，我们自己的推送服务器会向 APNs 服务器发起 TLS 连接；连接建立后，APNs 服务器会将自己的证书发送给我们的推送服务器，并且对其进行验证；完成验证后，我们的推送服务器会把需要推送消息的应用证书发给 APNs 进行验证；验证完成以后，双方就会建立起稳定的、安全的 TLS 连接，维护好这个连接，有消息需要推送时，就将消息的内容、deviceToken 发送给 APNs 即可完成推送。

在我们的服务端，会通过 SSLSocket 来与 APNs 服务器建立连接，SSLSocket 是对 Socket 的扩展，提供使用 SSL 或 TLS 协议的安全套接字。要想使用 SSLSocket，就需要使用 SSLContext，它是安全套接字协议的实现，SSLContext 里有一个 getSocketFactory()，可以返回个 SocketFactory，就可以得到 SSLSocket 了。因为每个租户所对应的证书不同，SSLContext 对于每个租户也不相同，所以会建立一个 SSLContextFactory 工厂类，根据租户 tenant_id 返回相应的 SSLContext，其关键代码如下：

```

public static SSLContext getSslContext(String tenantId){
    //根据 tenantId 获取证书名和密码
    Tenant tenant = TenantDao.findById(tenantId);
    String certificateName = tenant.getCertificateName();
    String certificatePsw = tenant.getCertificatePsw();
    //根据证书名称在本地获取应用证书,并生成KeyStore保存密钥
    FileInputStream certFis = new FileInputStream("c:/"+certificateName+".p12");
    KeyStore keyStore = KeyStore.getInstance("jks");
    keyStore.load(certFis, certificatePsw.toCharArray());
    certFis.close();
    //利用保存了密钥的keyStore创建并初始化证书库工厂
    String defaultAlg = KeyManagerFactory.getDefaultAlgorithm();
    KeyManagerFactory kmFact = KeyManagerFactory.getInstance(defaultAlg);
    kmFact.init(keyStore, certificatePsw.toCharArray());
    //创建并初始化SSLContext 实例
    SSLContext context=SSLContext.getInstance("TLS");
    context.init(kmFact.getKeyManagers(), null, null);
    return context;
}

```

通过从指定路径加载 KeyStore, 通过 KeyManagerFactory 这一密钥管理器管理初始化密钥。实现出上文中的工厂类, 就可以通过 `SSLContext sslContext = SSLContextFactory.getSslContext(tenantId)` 这行代码, 来获得各个租户各自证书生成的 sslContext。

我们的服务器程序中, 会有个 ApnsServicePool, 其作为 APNs 连接池, 可以保存维护最近使用过的 APNs 连接, 其内部实现是个固定大小的 HashMap, tenantId 作为 Key, apnsService 作为 Value, 其中 `getApnsService(String tenantId)` 是关键代码, 详细分析都在代码注释中, 其代码如下:

```

public static final int ASM_CAPACITY = 10;
HashMap<String, ApnsService> apnsServiceMap = new HashMap<String,
ApnsService>(ASM_CAPACITY);
public ApnsService getApnsService(String tenantId) {
    ApnsService apnsService = null;
    // 如果APNs连接池中已有当前租户的连接，就可以直接使用
    if (apnsServiceMap.containsKey(tenantId)) {
        apnsService = apnsServiceMap.get(tenantId);
    } else { // 如果APNs连接池中并没有，就需要创建新的连接，并放入连接池
        //通过tenantId获得
        SSLContext sslContext = SSLContextFactory.getSslContext(tenantId);
        //通过apnsService里的静态方法建立连接并启动
        apnsService = apnsService.initConnection(sslContext);
        apnsService.start();
        //连接池未满，可直接放入
        if (apnsServiceMap.size() < ASM_CAPACITY) {
            apnsServiceMap.put(tenantId, apnsService);
        } else { //连接池已满，需要删除后再放入
            this.remove(apnsServiceMap.entrySet().iterator()
                .next().getKey());
            apnsServiceMap.put(tenantId, apnsService);
        }
    }
    return apnsService;
}

```

通过上述方法可以随时获得相应 apnsService，ApnsService 是对 APNs 服务的封装，其中上文已经使用了其静态方法 initConnection 初始化连接、start 方法启动连接。除了这两个重要方法，ApnsService 的 sendMessage 方法就是用于发送需要推送的消息到 APNs 服务器，其发送的方式是通过建立的 SSLSocket 进行发送。

除了以上代码，还有个关键部分的代码就是 feedback 的获取，IOS 平台上的 feedback 只能得到用户点击消息的时间与 deviceToken，所以对于需要得到 feedback 的租户，可以在其连接后运行如下方法，详细分析都在代码注释中，代码如下：


```
//IOS反馈服务器
public static final String FEEDBACK_HOST = "feedback.push.apple.com";
public static final int FEEDBACK_PORT = 2196; //反馈端口号
//IosFeedbackList作为一个租户的feedback缓存
public static final List<IosFeedback> IosFeedbackList = new
    ArrayList<Feedback>();

public void getFeedbacks() {
    Socket socket = null;
    //通过SSLsocket工厂建立与feedback服务器的连接
    socket = factory.createSocket(FEEDBACK_HOST, FEEDBACK_PORT);
    InputStream is = socket.getInputStream();
    while (true) { //通过InputStream持续从服务器读取feedback
        byte[] bytes = new byte[38];
        int size = is.read(bytes);
        byte[] tokenByte = new byte[32];
        System.arraycopy(bytes, 6, tokenByte, 0, 32);
        //通过解码分离deviceToken与点击时间
        String token =
            ApnsTools.encodeHex(tokenByte).toLowerCase();
        long time = ApnsTools.parse4ByteInt(bytes[0],
            bytes[1], bytes[2], bytes[3]);
        //用一个IosFeedback实体对象存储一个反馈
        IosFeedback iosFeedback = new IosFeedback();
        iosFeedback.setTime(time);
        iosFeedback.setToken(token);
        //将当前读取的feedback存放到该租户的反馈缓存中
        IosFeedbackList.add(iosFeedback);
    }
    is.close();
    return;
}
```

对于每个租户来说,可以通过一个线程运行上述方法,可以一直获取到自己租户的 feedback 存放到缓存中,然后只需对 IosFeedbackList 这一缓存进行操作,会将反馈缓存中的数据根据 messageId 存入 message collection 中相应的 document,实现相关的业务逻辑。

5.3 Android 平台模块设计与实现

本文 Android 平台推送的实现是基于 XMPP 协议的,本节将对其设计与实现做详细介绍。

5.3.1 技术基础

5.3.1.1 XMPP 协议

通过第二章的简单介绍,我们可以知道 XMPP 协议是一种以 XML 为基础的开放式的、实时的、持久连接的通信协议,是当前主流的四中 IM 协议之一。本节将对 XMPP 的实现的、关键技术做详细的介绍,为 Android 客户端推送的设计与实现做好铺垫。

XMPP 采用的是 C/S 架构,能够简化客户端的实现,服务器会处理大量的繁杂的工作,所以不必经常更新客户端,大部分更改只需要在服务器端完成。XMPP 协议中包括了三种实体类型,XMPP 服务器、XMPP 客户端以及网关,三种实体间都可以主动发起通信,与其他类型实体完成双向通信。XMPP 协议是基于 TCP/IP 协议的,客户端通过 TCP/IP 协议连接到服务器上,在此基础上建立持久的连接,然后消息就以 XML 的形式进行双向传输^[28]。

XMPP 协议对移动客户端只有三个基本要求:①可以使用 TCP 与 XMPP 服务器进行交互通讯,通常使用 5222 端口;②能够解析 XML 消息包;③能够对解析后的 XML 消息按照 XMPP 协议的约定进行处理。

一个移动客户端在 XMPP 网络环境下拥有一个标示符,称之为 jabber identifier(JID),其是唯一的。一个有效的 JID 包括三个元素:①domain identifier;②node identifier;③resource identifier。它的格式是 node@domain/resource,或是 node@domain。例如 wuyajun 在 ComsysServer 上注册了一个用户,用户名就为 wuyajun,而且 ComsysServer 只为一个应用提供推送服务,那么 wuyajun 的 JID 就是 wuyajun@ComsysServer,在推送消息时,指明 wuyajun@ComsysServer 就可以了,resource 可以不用指定。但是 ComsysServer 可能会为多个应用提供推送服务,比如 wuyajun 会使用普教云平台和信息门户两个应用(可以相当于两个租户),所以 wuyajun 的 JID 可能是 wuyajun@ComsysServer/pujiaoyun,也可能是

wuyajun@ComsysServer/portal, 也就是说一个用户可能会同时使用两个 APP, 其用户名相同, 消息推送服务都是由同一个 XMPP 服务器提供, 此时就会通过 resource 表明消息属于哪个 APP。

在 XMPP 协议中, 定义了三个 XML 顶层标签: IQ、Message、Presence。

IQ 是使用最频繁的标签, 也是本文会设计使用的标签, 它类似于一种请求/响应模式, 它会从一个 XMPP 实体定向发出请求给另一个 XMPP 实体, 在接受请求后会进行应答。其中 type 是请求的类型, 有四种值 get、set、result、error, 其格式如下:

```
<iq from="server@Comsys/pujiaoyun" id="634546" type="result" ></iq>
```

从上文的详细介绍中可以看出 XMPP 协议是一种 client-server-client 的即时通讯协议, 但是其既可以作为即时通讯协议, 也可以将 server-client 的这个过程截取出来作为一个消息推送的过程。

5.3.1.2 asmack

要介绍 asmack, 首先得介绍下 smack, smack 是基于 XMPP 协议实现的开源 Java 客户端类库。这个开源类库对 XMPP 协议做了完成的封装与实现, 即使 Java 工程人员不是很熟悉 XMPP 协议, 也能够开发出使用 XMPP 协议通信的客户端。

由于 Android 里的 Java 类库是进行过精简的, 所以 smack 在 Android 系统里是不能获得其所需支持的, asmack 就是对 smack 进行改进后, 使其能够适应 Android 环境, 虽然有改动, 但是功能都是相同的。所以本文会使用 asmack 类库作为实现持久连接、接收消息的基础类库^[29]。

5.3.1.3 Mina

Mina 是一个 Java 开源框架, 可以向帮助开发人员开发出性能高、伸缩性好的网络应用程序。它通过 Java NIO 技术提供了事件驱动的、异步的 API^[30]。

5.3.1.4 Androidpn

Androidpn 也是 Java 开源项目, 它是基于 XMPP 协议的即时通信服务器, 我们可以在其源码的基础上进行裁减或者二次开发, 让其能够提供消息推送服务。

5.3.2 整体设计

5.3.2.1 系统整体设计

Android 平台上的推送系统要比 IOS 平台上复杂得多, 其设计与实现更加的困

难。其系统整体设计图 5-5 如下：

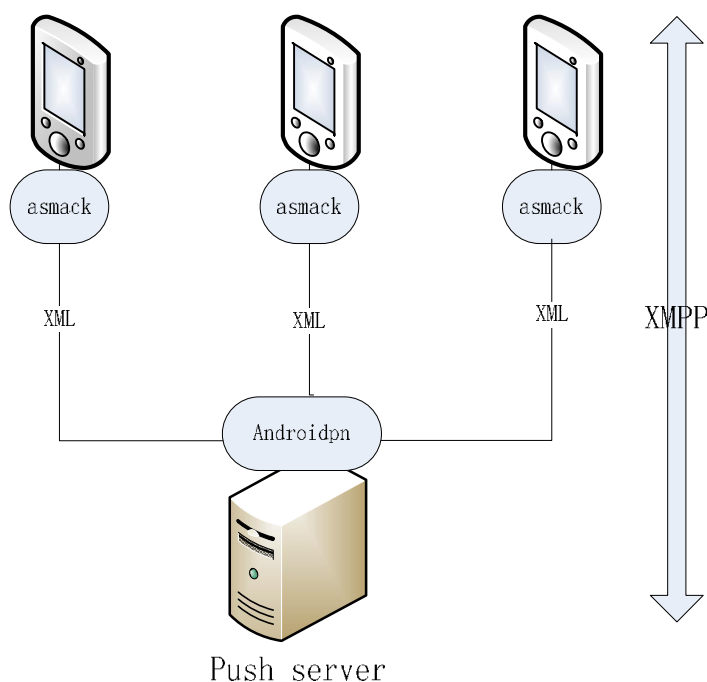


图 5-5 Android 平台推送系统整体设计图

从图 5-5 中可知，移动端与服务端的通信都是基于 XMPP 协议实现的，使用 XML 文件作为信息的载体，移动端是基于 asmack 来开发的，服务端是基于 Androidpn 来开发的，使用两个开源软件完成 XMPP 协议的解析、封装工作，可以节省开发成本与开发时间。

5.3.2.2 JID 的设计

通过上文的介绍，我们知道 JID 的格式为 `node@domain/resource`，所以在我们的系统中，会采用统一的格式：`userId@comsys.net.cn/tenantId`，例如普教云应用下的用户 `wuyajun`，它接收发送消息的唯一 JID 就是 `wuyajun@comsys.net.cn/pujiaoyun`，因为不同的租户的 `tenantId` 不同，而且一个租户下不会有相同的 `userId`，所以 JID 就是一个唯一标识，其中 `server@comsys.net.cn` 会作为消息反馈、登录等功能的服务器入口标识。

5.3.2.3 消息格式的设计

在推送系统中，客户端与服务器之间所传递的消息约定格式有消息推送、消息反馈、登录注册、注册反馈、退出注销。

消息推送是使用频率最高的约定格式，其格式如图 5-6 所示：

```

<notification> <!--表明是推送消息标签-->
  <id>340234234</id> <!--推送消息的Id, 唯一-->
  <tenantId>102034</tenantId> <!--租户Id-->
  <title>通知公告</title> <!--推送消息标题-->
  <content>您有一条放假通知</content> <!--推送消息内容-->
</notification> <!--推送消息标签结束-->

```

图 5-6 消息推送消息格式图

对于每个消息，客户端还可以进行反馈，其约定格式如图 5-7 所示：

```

<iq id="random123321"
  type="set"
  from="wuyajun@comsys.net.cn/pujiaoyun"
  to="server@comsys.net.cn"
  > <!--id是一个随机数；iq的类型是set，与HTTP中的post类似；
  from，to属性就是消息的收发者-->
  <query xmlns="jabber:iq:feedback"></query> <!--表明是feedback-->
    <messageId>340234234</messageId> <!--反馈消息的唯一ID-->
    <clickTime>16-1-8,13:09</clickTime> <!--消息的点击时间-->
    <location></location> <!--消息的点击地点-->
  </query>
</iq> <!--标签结束-->

```

图 5-7 客户端反馈消息格式图

客户端有用户在登陆时，在应用服务器上登录成功后，会向推送服务器发送登录注册消息，其约定格式如图 5-8 所示：

```

<iq id="random123321"
  type="set"
  to="server@comsys.net.cn"
  > <!--id是一个随机数；iq的类型是set，与HTTP中的post类似；to
属性就是消息的接收者-->
  <query xmlns="jabber:iq:register"><!--表明是登录注册-->
    <tenantId>634546</tenantId> <!--租户ID-->
    <userId>wuyajun</userId> <!--用户名-->
    <psw>ff7cyj3cmy86mei69de76lin115zds4d</psw> <!--密码-->
  </query>
</iq> <!--标签结束-->

```

图 5-8 登录注册消息格式图

服务器会对登陆注册消息进行反馈，以便于建立持久连接，其约定格式如图 5-9 所示：

```

<iq id="random123321"
  type="set"
  to="wuyajun@comsys.net.cn"
  > <!--id是一个随机数；iq的类型是set，与HTTP中的post类似；to
属性就是消息的接收者-->
  <query xmlns="jabber:iq:auth"><!--表明是注销登录-->
    <result>true</result> <!--注册认证结果-->
  </query>
</iq> <!--标签结束-->

```

图 5-9 登录注册反馈消息格式图

客户端有用户在注销登录时，也会向推送服务器上发送退出注销消息，其约定格式如图 5-10 所示：

```

<iq id="random123321"
  type="set"
  to="server@comsys.net.cn"
  ><!--id是一个随机数; iq的类型是set, 与HTTP中的post类似; to
属性就是消息的接收者-->
  <query xmlns="jabber:iq:logout"><!--表明是注销登录-->
    <tenantId>634546</tenantId> <!--租户ID-->
    <userId>wuyajun</userId> <!--用户名-->
    <psw>ff7cyj3cmY86mei69de76lin115zds4d</psw> <!--密码-->
  </query>
</iq> <!--标签结束-->

```

图 5-10 注销登录消息格式图

5.3.3 客户端关键模块的设计与实现

对于客户端来说, 最重要的三个部分就是 XMPP 协议消息的封装与解析、连接的建立以及如何保持持久连接。第三方开发者只需要在其 Android 工程加入 comsys-push.jar 包, 再按照下文中的描述保存登录用户信息即可使用。

5.3.3.1 XMPP 协议消息的封装与解析

XMPP 协议消息的封装与解析这个对于客户端和服务端都是很重要的, 因为我们推送所使用的 XMPP 消息的格式都是约定好的, 所以可以很容易通过 asmack 加上工厂模式生成消息。其中工厂模式中生成推送消息的代码如下:

```

public static Notification createPushMessage(String tanantId,
String title,String content) {
    String id = XMPPTools.creatRandomId(); //生成随机ID
    Element message = DocumentHelper.createElement();//创建一个XML节点
    //向节点加入各种约定属性
    message.addElement("id").setText(id);
    message.addElement("tanantId").setText(tanantId);
    message.addElement("title").setText(title);
    message.addElement("content").setText(content);
    //使用asmack中的Notification对XML进行封装
    return new Notification().setChildElement(message).toXMPP();}

```

这是 XMPPMessageFactory 工厂类中的其中一个方法，该工厂类中还有其他生成 XMPP 消息的方法，实现大致相同，通过此工厂类，我们可以很方的通过一行代码就可以得到封装好的 XMPP 消息。对于 XMPP 消息的解析，我们通过开源的 asmack 进行解析，因为我们对 XMPP 消息有着约定的格式，解析代码只需按照既定格式进行即可，代码就不再列出。

5.3.3.2 连接的建立

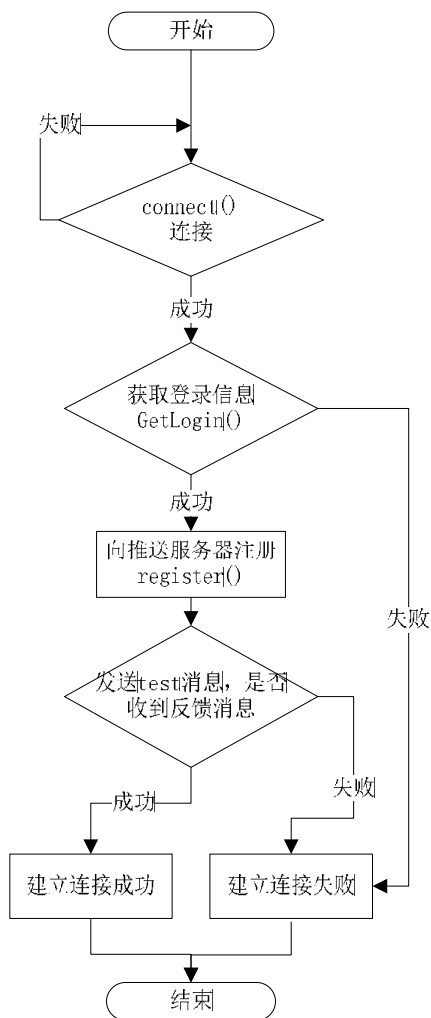


图 5-11 客户端连接建立流程

客户端连接的建立是在 Android 四大组件之一的 Service 上进行的，Service 是会一直在后台运行的服务，在第一次点击应用程序时就会启动 Service，而且可以做到开机启动。

客户端与应用建立连接就是通过 Service 的进行的，其流程图 5-11 如所示。

XMPPService 类中 run()方法就是按照上述流程图 5-11 进行连接服务器，在此

之间 onCreate()方法会准备好所需资源，实例化手机状态监听器，主要用于监听网络连接情况，还会在 onCreate()中进行 XMPPConnection 的初始化工作，会在其中调用 initXMPPConnection()方法得到一个配置好的 XMPP 连接，其代码如下：

```
public XMPPConnection initXMPPConnection() {
    ConnectionConfiguration cc = new ConnectionConfiguration("comsys.net.cn",
5222);
    cc.setSASLAuthenticationEnabled(false);// 不使用SASL验证，设置为false
    cc.setSecurityMode(ConnectionConfiguration.SecurityMode.enabled);//使用安全
模式连接
    cc.setReconnectionAllowed(false); // 不允许自动连接
    XMPPConnection connection = new XMPPConnection(cc);
    return connection;
}
```

connect()方法首先会使用手机状态监听器查看是否有网络连接，如果没有就会5秒之后尝试重连，尝试30次后提醒用户没有网络连接。在网络连接成功后，会使用 onCreate()方法里已经初始化的 XMPPConnection 进行连接（这里的连接还没有建立持久连接，只是完成了通信），连接成功以后，会使用使用 asmack 来完成接下来的操作，实例化一个 IQProvider，用于处理 IQ 标签的 XMPP 消息，让客户端能够接收来自推送服务器端的 XMPP 消息。

在通过 XMPP 连接成功推送服务器端以后，会使用 GetLogin()获取用户的登录信息，如果应用还没用户登录，就不需要进行下一步了，因为只有知道了租户 tenant_id、用户 user_id 才能进行下一步的注册操作。登录信息需要第三方开发者将登录信息存放在一个 SharedPreferences 中，并命名为 comsysPush，数据格式为 {"tenantId":string, "userId":string}，GetLogin()方法会按照固定格式进行读取，然后使用 initXMPPConnection()初始化的 connection 进行向服务器注册登录，注册的代码为 connection.register(userId, COMSYS_PSW, tenantId)，其中 COMSYS_PSW 是固定的密码，因为在我们推送服务器是会对用户进行登录验证的，用户的登录验证由第三方使用者自己验证，验证通过后才能够进行使用 COMSYS_PSW 登录 XMPP 推送服务器这一步骤，这一步骤的前提就是用户已经由第三方使用者验证了其合法性，服务器会有个 RegisterHandler 模块负对接，注册的消息格式按照上文中约定的消息格式进行发送。如果是一个新的用户就会进行注册操作，如果

该用户已经注册过，就不用再注册，服务器返回注册成功。注册成功后可以直接连接上 XMPP 服务器，会向 XMPP 服务器发送一个 test 消息，收到 test 消息反馈后就表明持久连接已经建立。

持久连接建立以后客户端的 Service 就可以随时接收到服务端发来的推送 XMPP 消息了，会通过 asmack 里的 PacketTypeFilter 和 NotificationPacketListener 两个类来实现接收推送的消息的功能。PacketTypeFilter 会对服务器发来的消息进行拦截，并组装成 NotificationIQ 的格式。每个 NotificationIQ 都会传递给 NotificationPacketListener，NotificationPacketListener 会调用的 processPacket()方法对 NotificationIQ 处理，processPacket()方法会按照上文中的约定消息格式解析 NotificationIQ，然后通过 Android 的广播机制实现把消息推送到通知栏，完成整个推送过程。

用户在客户端点击消息之后，会使用 XMPPMessageFactory 工厂类中的 creatFeedBackMessage(String messageId, String location, String clickTime)生成 FeedBack XMPP 消息的方法，其中 location 与 clickTime 都可以为空，然后反馈给服务器，服务器会将反馈根据 messageId 存入 message collection 中相应的 document。

5.3.3.3 维护持久连接

在建立了 XMPP 连接之后，还有一项重要的工作就是如何维护持久连接，主要是通过三个方面进行保证：XMPPService 开机启动、使用 XMPPConnectionListener 持久连接监听器、NetworkListener 网络监听器来维护持久连接。

1、XMPPService 开机启动

要保证 XMPPService 开机启动，需要使用一个随着系统启动的 BroadcastReceiver。Android 系统启动时候，会发送一个开机广播，是一个名为 android.intent.action.BOOT_COMPLETED 的 action，可以通过一个 BroadcastReceiver 来接收开机广播，并在其中启动 XMPPService。

2、XMPPConnectionListener 断线重连

在本文中会实现一个 PersistentConnectionListener 类，它继承自 asmack 包中 ConnectionListener 类，当推送服务器或者客户端出现异常，并且致使持久连接关闭以后，PersistentConnectionListener 会监听此变化，并且会调用 reconnection()方法进行重连，此方法会发送一个广播重启 XMPPService 的广播，当 XMPPBroadcastReceiver 收到这个广播，就会重新启动 XMPPService，并进行连接。

3、NetworkListener 监听网络切换

NetworkListener 网络监听器可以对客户端网络状态进行监听。因为当用户从移动 4G 网络切换到 WiFi 网络，或从 WiFi 网络切换到移动 4G 网络时可能会导致关闭持久连接。这种情况是不会触发 PersistentConnectionListener 进行重连的，因此 NetworkListener 会在客户端网络状态发生改变时，会发送测试包来测试 XMPP 推送连接是否处于连接状态，如果连接断开则会调用 reconnection()方法进行重连，重连过程与 XMPPConnectionListener 中重连过程相同^[31]。

5.3.4 服务端的设计与实现

服务端的实现参考了开源软件 AndroidPN，连接会采用基于 Java NIO 技术的高性能并发框架 mina 进行管理，让服务器可以处理更多的持久连接，并且使用 spring 进行集成管理，方便迭代开发。

5.3.4.1 服务端模块设计

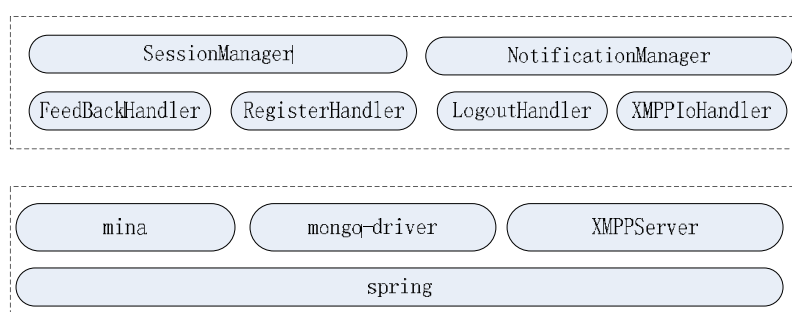


图 5-12 服务端模块设计图

目前我们服务端的主要模块设计如图 5-12 所示，其中下半部分虚线框里的属于支持层模块，为上层模块提供支持，上半部分虚线框里的模块是与推送相关的模块。

spring、mina、mongo-driver 都是常见的开源软件，作为本系统的支持层。spring 主要负责模块的集成管理、模块间的依赖注入、spring-task 定时扫描发送，以及以后 Web 端功能的接入；mina 提供 NIO 异步 Socket 连接；mongo-driver 负责与 MongoDB 的交互。

其余模块都是与推送相关的模块，入口类 XmppServer 是一个单例模式的类，这个类是 XMPP 服务程序的入口，会完成启动工作，以及完成一些初始化的工作；XMPPioHandler 是服务器接收发送 XMPP 消息的“门户”，其父类是 mina 框架中的 IOHandler，用于从 IO 连接中接收消息，以及提供了一些方法供发送 XMPP 消

息时使用；FeedBackHandler、RegisterHandler、LogoutHandler 分别接收并处理命名空间为"jabber:iq:feedback"、"jabber:iq:register"、"jabber:iq:logout"的 XMPP 消息，对应于消息反馈业务、用户注册登录、用户注销业务；SessionManager 是个单例模式的类，用于对所有客户端与服务器之间的连接进行管理，一个连接就对应于一个 session；NotificationManager 也是个单例模式的类，其用于对需要推送的消息进行管理，推送消息给客户端的功能由 NotificationManager 完成。

5.3.4.2 服务端流程设计

推送消息给客户端的整个流程可以分为两大部分，建立连接与推送消息，建立连接的流程图 5-13 如下：

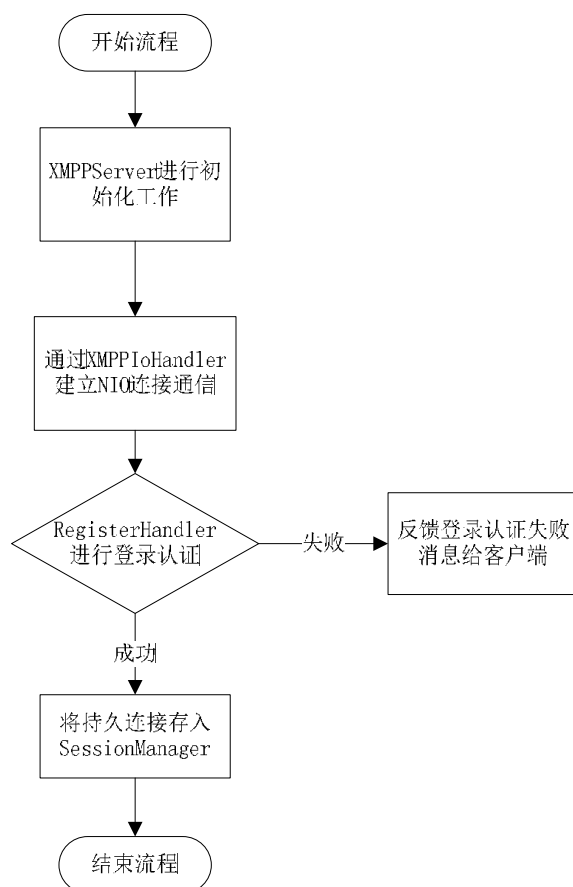


图 5-13 服务端建立连接流程设计图

建立连接流程中各个步骤的详细设计如下：

- 1、XMPPServer 模块负责启动 server 程序，并且实例化 SessionManager、NotificationManager，初始化 NIO 连接接受器 NioSocketAcceptor；
- 2、客户端会向服务器建立 NIO 连接，通过 XMPPHandler 这个类中的

sessionOpened()方法建立连接通信,并通过 messageReceived()方法接收客户端发来的测试消息;

3、然后会通过 RegisterHandler 进行注册,建立持久连接,如果认证失败会反馈失败消息给客户端;

4、系统定义了一个类 Session 用来存储每一个持久连接,每一个 session 对象都代表着一个持久连接,并包含其所有状态信息,持久连接建立成功后,就会以 session 的形式存储在 SessionManager 中^[32],用 Map<String, ClientSession>存放,其中 String 是以 tenantId_userId 的形式保存。

在连接建立以后,推送服务器就可以向建立了连接的 Android 客户端进行消息推送了,消息推送流程图如图 5-14 所示。

推送消息流程中各个步骤的详细设计如下:

1、用 spring-task 定时扫描 message collection,消息来自于 MongoDB 中;

2、对消息进行解析,消息解析主要完成的工作是通过 tenant_id 查询 tenant collection 中的 p_available,看租户是否开启推送功能;查询 tenant collection 中的 push_type,得到租户的移动推送和短信推送的组合方式;若是有 Android 端的消息需要推送,就会执行 Android 平台的推送流程;

3、通过 tenantId 与 userId 构造 tenantId_userId 形式的 String,使用 SessionManager 的 getSession()方法在 Map 中查找相应的客户端连接 session;

4、将需要推送的消息 document 中的 title、content、tenant_id 使用上文中的 XMPPMessageFactory 工厂类进行封装,成为 XMPP 消息;

5、有了客户端连接的 session 和封装好的 XMPP 消息,就可以调用 NotificationManager 中的 sendNotification()方法进行消息推送了,然后再调用 write()方法向客户端推送 XMPP 消息。

完成了推送流程后,客户端可能会有消息反馈发回,通过 FeedBackHandler 接收并处理命名空间为"jabber:iq:feedback"的 XMPP 消息,这一消息是客户端用 XMPPMessageFactory 工厂类中的 creatFeedBackMessage(String messageId, String location, String clickTime)这一方法生成的,其中 location 与 clickTime 都可以为空,收到消息后,服务器会将反馈信息根据 messageId 存入 message collection 中相应的 document。

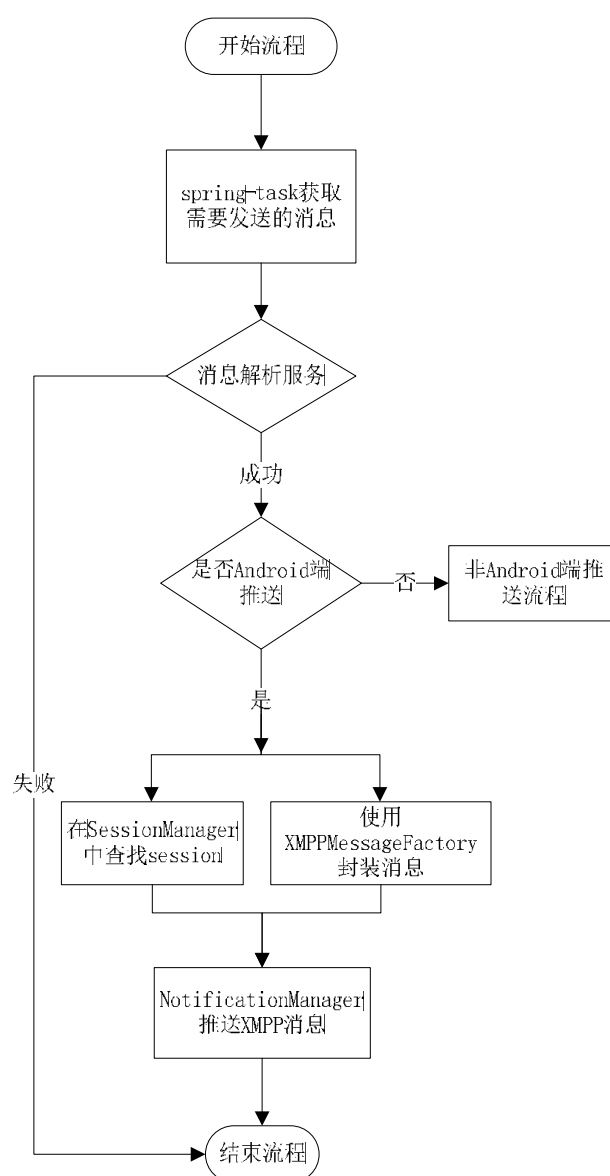


图 5-14 服务端推送流程设计图

5.3.4.3 关键实现详细介绍

1、mina 的使用

客户端和服务端之间的通讯连接都是基于 mina 框架的，在本系统中客户端与服务器的交互都是通过 Handler 来实现，FeedbackHandler 用于接受消息反馈、RegisterHandler 用于客户端注册登录、LogoutHandler 用于客户端注销退出，这些 Handler 都是建立在 XMPPHandler 建立的通信之上的，XMPPHandler 会通过 spring 进行管理，并向其中注入 mina 特性，Handler 会以 mina 提供的异步方式处理所有的 I/O 请求。注入代码如下：

```

<bean id="xmppHandler" class="org.androidpn.server.xmpp.net.XmppIoHandler"
/>
<bean id="ioAcceptor"
    class="org.apache.mina.transport.socket.nio.NioSocketAcceptor"
    init-method="bind" destroy-method="unbind" scope="singleton">
    <property name="defaultLocalAddress" value=":5222" />
    <property name="handler" ref="xmppHandler" />
    <property name="filterChainBuilder" ref="filterChainBuilder" />
    <property name="reuseAddress" value="true" />
</bean>

```

通过上述注入代码, XMPPIoHandler 就会通过 mina 中的 NioSocketAcceptor 来与客户端建立连接, 就能够用使用 NIO 来进行 socket 通信, 比传统 I/O 支持更多的连接。

2、XMPP 消息的接收

XMPPIoHandler 这个类中的 messageReceived()方法是接收客户端 XMPP 消息的方法, 它是其余 Handler 能够正常工作的关键, 详细分析都在代码注释中, 代码如下:

```

public void messageReceived(Session session, Object message){
    //得到一个当前线程hash值构造一个唯一的XMPPP消息解析器, 提供安全性
    int hashCode = Thread.currentThread().hashCode();
    XMPPPacketReader parser = new XMPPPacketReader();//这是smack提供的
    XMPPP消息解析器
    parser.put(hashCode);
    //用消息解析器解析收到的XMPP消息, 并存为DOM XML
    Element doc = parser.read(message).getRootElement();
    if (doc == null) {return;}
    String tag = doc.getName();//得到标签, 如果是iq就进行Handler转发
    if ("iq".equals(tag)) {
        HandlerDispatcher(doc);//可以根据消息的类型转发给相应的Handler
    } else {
        session.close();return;}}

```

3、XMPP 消息的推送

NotificationManager 中的 sendNotification()是向客户端推送消息的关键实现方法，详细分析都在代码注释中，其代码如下：

```
//放回的是消息未能推送成功的user数组
public String[] sendNotification(String tanantId, String username, String title,
String content){
    //通过XMPPMessageFactory生成约定格式的推送消息
    Notification notification =
    XMPPMessageFactory.createPushMessage(tanantId,title,content);
    //username可以是;隔开的多个user，也可以是单个user
    if(username.indexOf(";")!=-1){//多个user推送的情况
        String[] users = username.split(";");//得到多个user
        List<String> usersList = new ArrayList<String>();
        //list存储全部user，便于去除成功推送的
        Collections.addAll(usersList, users);
        for (String user : users) {
            //调用sendNotifcationToUser进行推送，成功返回true
            if(this.sendNotifcationToUser(notification, tanantId ,user)){
                usersList.remove(user);//推送成功去除user
            }
        }
        //返回剩下的未推送成功的user数组
        return (String[]) usersList.toArray();
    }else{//单个user推送的情况
        if(this.sendNotifcationToUser(notification, tanantId, username)){
            return new String[]{};
        }else{
            return new String[]{username};
        }
    }
}
```

sendNotification()中的 sendNotifcationToUser()是具体的推送过程，详细分析都

在代码注释中，其代码如下：

```
public Boolean sendNotificationToUser(Notification notification, String
tenantId, String username){
    //通过tenantId_userId在sessionManager取得响应连接
    ClientSession session = sessionManager.getSession(tenantId+"_"+username);
    if (session != null) {
        //设置notification的格式为username@comsys.net.cn/tenantId
        notification.setTo("comsys.net.cn");
        notification.setUsername(username);
        notification.setResource(tenantId);
        //根据推送结果返回，调用的是Androidpn中的方法
        return session.deliver(notification);
    }else{//推送未成功
        return false;
    }
}
```

5.4 短信推送模块的设计与实现

短信推送模块的实现比较简单，其是基于速达移动 WebService 实现的，只需要添加接口地址和相关参数，参数包括手机号码、消息内容，调用 WebService 后会返回推送结果。

5.4.1 短信推送流程

短信模块推送流程图 5-15 如下：

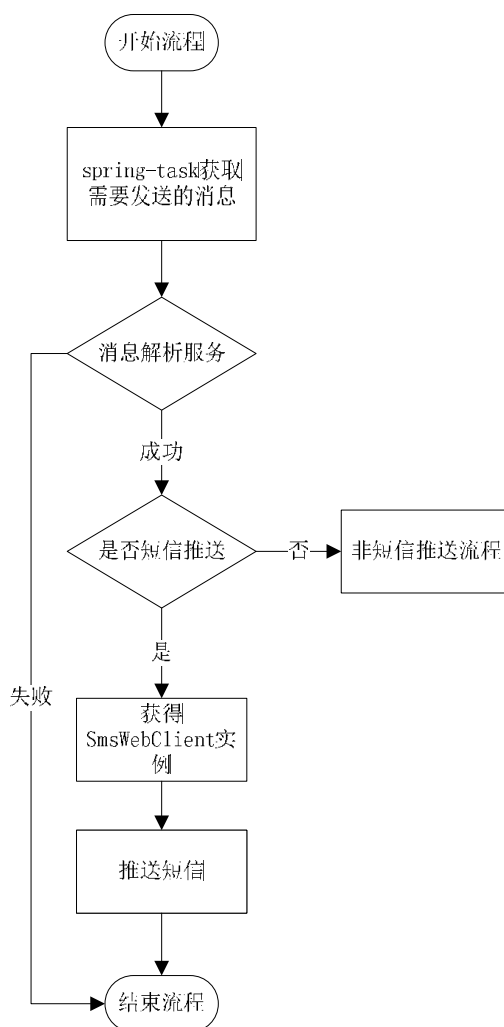


图 5-15 短信推送流程设计图

流程中各个步骤的详细设计如下：

- 1、用 spring-task 定时扫描 message collection，消息来自于 MongoDB 中；
- 2、对消息进行解析，消息解析主要完成的工作是通过 tenant_id 查询 tenant collection 中的 p_available，看租户是否开启推送功能；查询 tenant collection 中的 push_type，得到租户的移动推送和短信推送的组合方式；如果是短信推送，通过 user_id 查询 user collection 中的 telephone，获得用户手机号码，成功后，就会执行短信推送流程；
- 3、首先会去获得 SmsWebClient，这是 WebService 提供的客户端接口，我们使用单例模式对其进行封装；
- 4、得到 SmsWebClient 后，就会根据手机号 telephone，message 的 content 进行推送。

5.4.2 关键实现详细介绍

5.4.2.1 SmsWebClient 的封装

本文使用单例模式对第三方提供的 WebService 客户端进行封装，详细分析都在代码注释中，代码如下：

```
public class SingletonSmsWebClient extends SmsWebClient {
    private static SingletonSmsWebClient instance; //唯一单例声明
    private SingletonSmsWebClient(){}; //私有化构造函数，防止其他类创建
    //用单例模式获取唯一实例
    public static SingletonSmsWebClient getInstance() {
        if (instance == null) { //实例未创建
            //同步，防止两个线程同时创建
            synchronized (SingletonSmsWebClient.class) {
                if (initClient()) { //调用 initClient()创建实例
                    return instance; //返回实例
                }
            }
        }
        return instance; //实例已经实例化就直接返回
    }

    public static Boolean initClient() {
        // 判断客户端是否已经初始化，没有则初始化
        if (!SmsWebClient.enable()) {
            //init方法初始化实例，参数为第三方提供，为保密用名称代替
            if (instance.init("url", "userName", "passWord")) {
                return true;
            } else {return false; }
        } else {return true; }
    }
}
```

5.4.2.2 短信的发送

短信的发送很简单，只需使用我们的 SingletonSmsWebClient 实例的 sendMessage 方法进行发送，详细分析都在代码注释中，代码如下：

```
public static boolean sendMessage(String telephone, String content) {  
    // 使用SingletonSmsWebClient单例进行发送，并得到发送结果  
    SmsReturnObj retObj = instance.webSendMessage(telephone, content);  
    if (retObj.getReturnCode() != 1) { //将发送失败原因记录到日志  
        log.info("短信发送失败， 原因为: " + retObj.getReturnMsg());  
        return false;  
    } else {  
        return true;  
    }  
}
```

5.5 Web 推送管理模块

Web 推送管理模块是向普教云租户提供的特色服务，方便租户管理员对推送进行管理、查看推送情况、查看定制化报表等功能。

5.5.1 管理平台流程设计

本小节会给出管理平台的流程设计，会对管理平台的使用流程进行简要的介绍，流程图如图 5-16 所示。

由流程图 5-16 可以看出，用户的登陆、权限的验证都由普教云平台完成，在成功验证租户管理员身份以后，管理员就可以使用流程图里 Web 推送管理平台的各种功能。

5.5.2 功能模块设计

各个功能模块设计的重点主要在于界面设计、根据业务与数据库交互，由于这不是本文的重点，会对其中比较重要的新建消息、查询定制化报表做详细介绍。推送管理隶属于普教云平台校务办公模块，其菜单设计如图 5-17 所示。

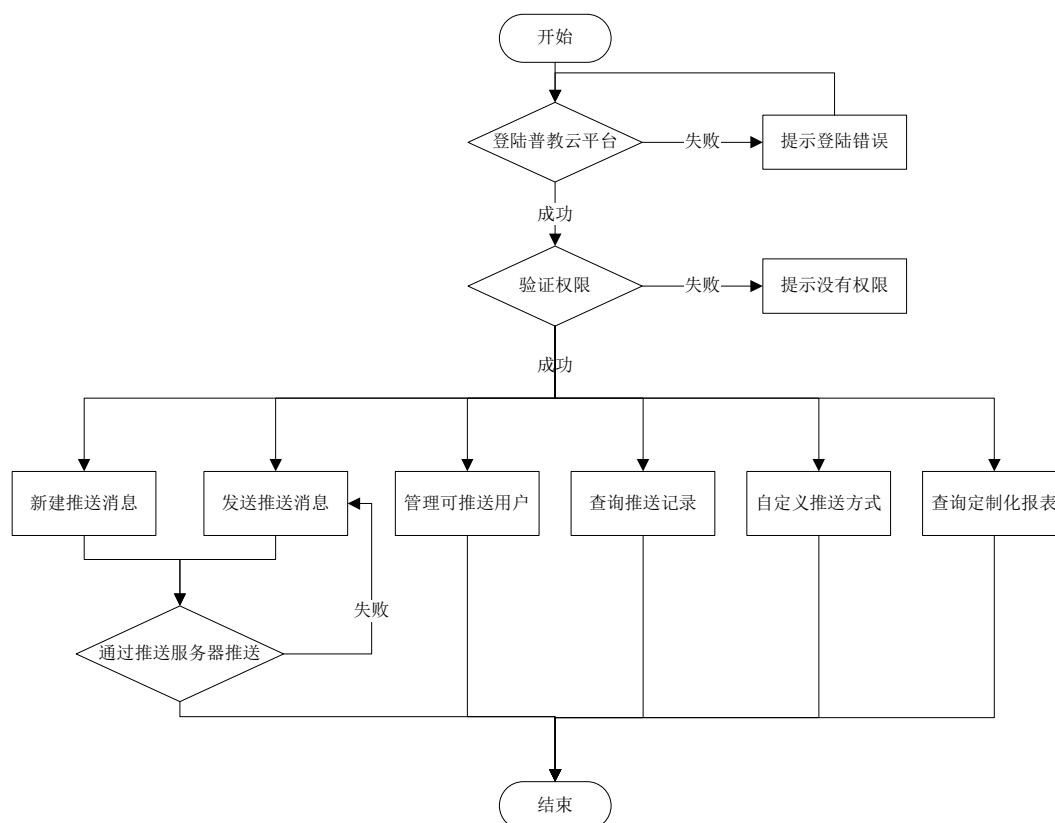


图 5-16 管理平台流程设计图

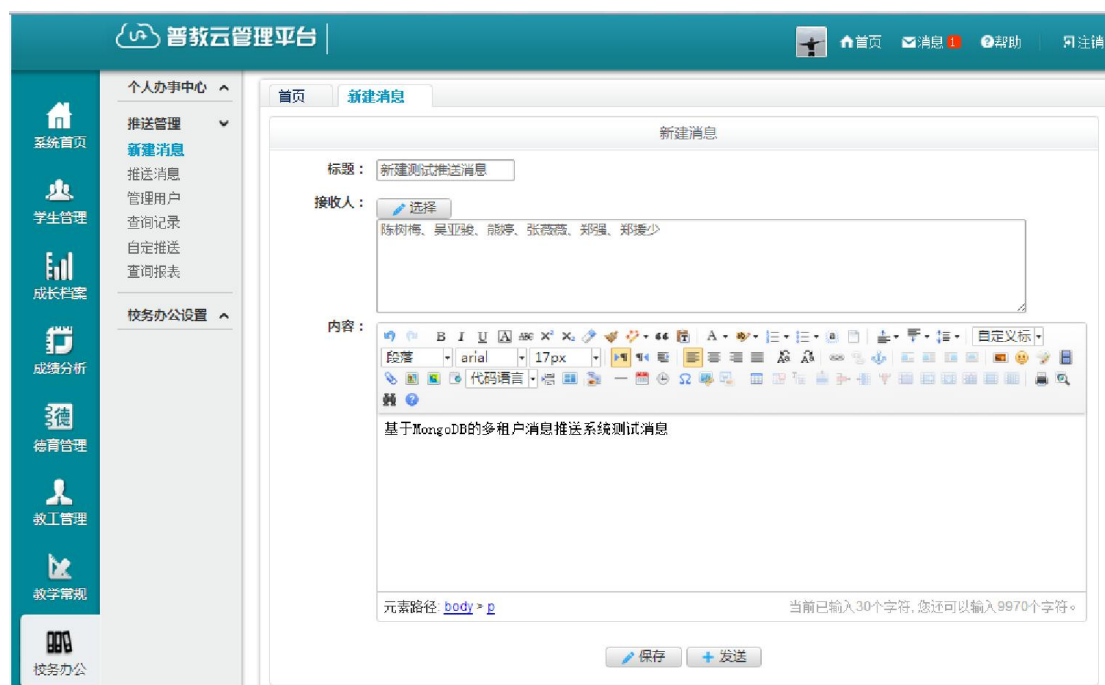


图 5-17 管理平台菜单界面图

5.5.2.1 新建消息

新建消息这一模块主要用于手动新建推送消息，其主界面如图 5-18 所示：

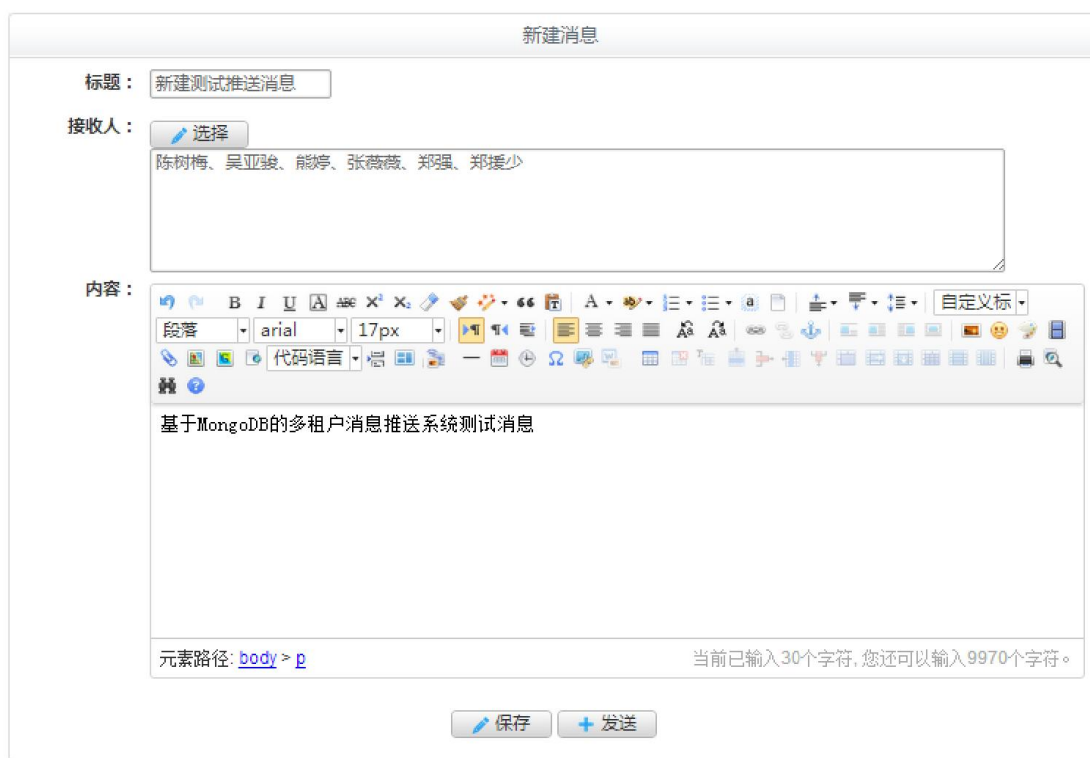


图 5-18 新建推送消息界面图

管理员需要填写消息的标题，内容，其中标题不超过 20 个字符，内容不超过 120 个字符，然后管理员需要通过点击选择按钮，通过弹出的人员树选择接收人，人员树如图 5-19 所示。

完成选择之后，可以通过点击保存按钮保存消息，或者发送按钮直接发送推送消息。

新建消息主要是和 MongoDB 中的 user collection, message collection 进行数据交互，其中人员树是从普教云的租户用户表中读出，通过其获取接收人的 tenant_id、user_id，通过这两个属性查询 user collection，看用户是否处于可以推送的状态。如果允许对其推送就会给此消息按照“租户 id_用户 id_格式化时间”的形式生成 messageId，然后将 messageId、title、content、tenant_id、user_id 插入到 message collection，完成数据交互。



图 5-19 人员树界面图

5.5.2.2 查询定制化报表

这一模块主要用于根据租户定制化的需求提供报表，其实现方式是根据业务需求，从 MongoDB 中读取数据进行加工，并在前台通过 HigCharts 展示。

其中有些报表是每个租户都有的，比如消息推送数报表，界面如图 5-20 所示：



图 5-20 消息推送数报表界面图

除了这种通用的报表，还会根据租户的定制化需求提供特色的报表，如用户点击消息的时间分布折线图，界面如图 5-21 所示：

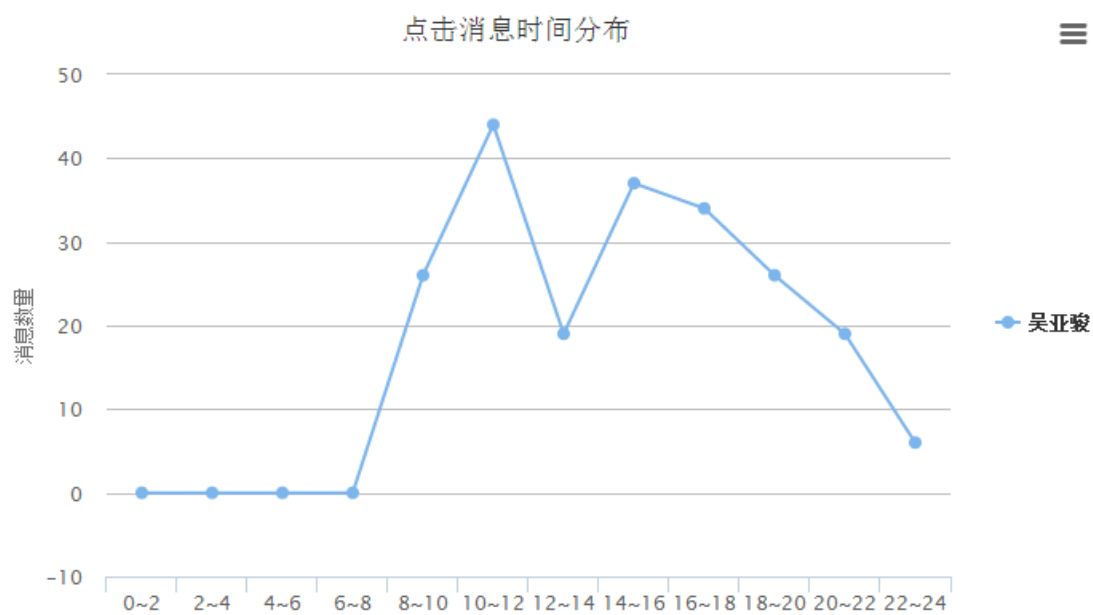


图 5-21 用户点击消息时间分布报表界面图

其余报表就不一一列出，对于这一模块的实现，主要是通过对 MongoDB 中的 message collection 进行各种各样的查询，并结合 cus_metadata_collection 实现定制化元数据查询，最终将查询结果通过 HigCharts 进行展示。

对于通用报表，只需按照业务需求查询 message collection 即可，以消息推送数报表为例，只需通过 tenantId 与时间为条件查询出各个时间段的推送数据数量，然后通过柱状图 HigCharts 展示。

对于定制化报表，在进行查询之前，需要根据租户的 tenantId 去查询 cus_metadata_collection，查看租户的定制化属性，才能进行进一步查询。比如上例中的点击消息时间分布报表，就是此租户具备了 click_time 这一定制化属性，才能生成此报表，然后会进行查询 message collection，生成折线图 HigCharts 展示。

5.6 第三方推送服务调用模块

由系统的物理架构可知，第三方服务器会以 POST 的方式将需要推送的消息信息提交到普教云服务器上，然后再通过普教云服务器写入 MongoDB 中的 message collection 中，具体流程如图 5-22 所示：

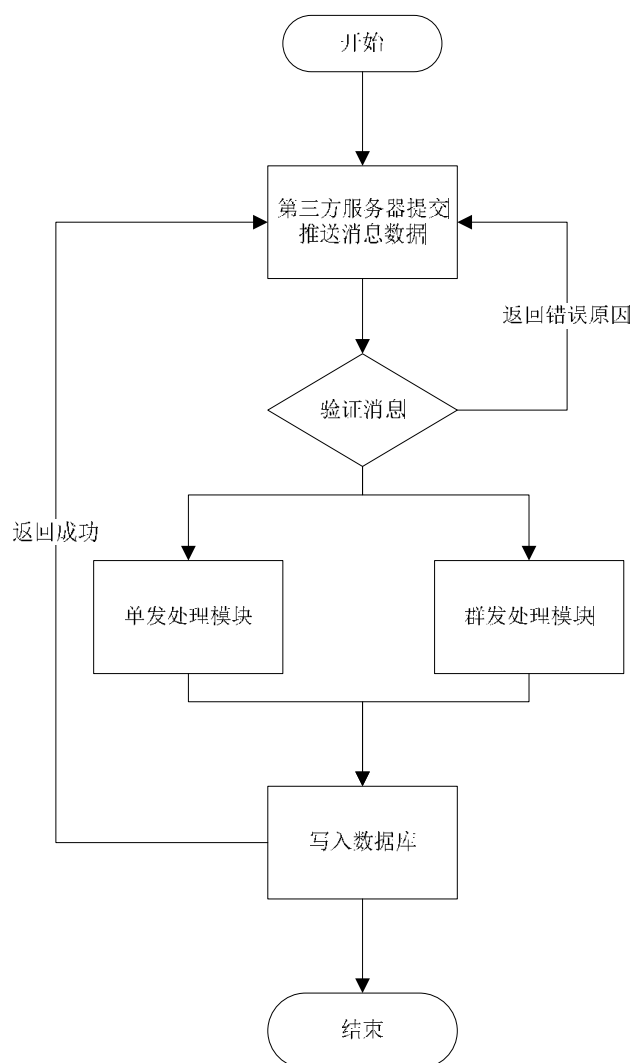


图 5-22 第三方推送服务调用流程设计图

1、第三方服务器通过 POST 的方式向普教云服务器提交数据，其中包含消息标题 title，消息内容 content，租户 tenantId，其中标题不超过 20 个字符，内容不超过 120 个字符，还需要提供我们提供的 COMSYS_PSW，以及需要推送的用户 user_id 数据，如果是调用单发接口，就提供用户 user_id，如果是群发接口，就将 user_id 用 “;” 隔开，作为一个字符串提交，最大用户数不超过 200 个

2、会按照数据格式要求对各个字段进行验证，验证不通过会返回给第三方服务器错误原因。

3、验证通过后，会根据单发还是群发进行不同的处理，群发需要多出个分离用户 userId 的步骤，然后将 title、content、tenantId、userId 以及生成的 messageId 封装成一个 message 对象，并将其存入一个基于 ConcurrentLinkedQueue 实现的消息队列中，作为生产者存入 message 对象。

4、会有消费者线程从消息队列中读取 `message` 存入 MongoDB 的 `message collection` 中，并返回成功消息给第三方服务器。

5.7 本章小结

本章主要以应用层工程性内容为主，首先进行系统的整体设计，包括物理架构设计、系统模块设计，接着会通过流程图以及关键代码对系统各个模块的设计与实现进行详细说明，包括 Android 平台客户端、服务端，IOS 平台客户端、服务端，短信推送模块、Web 推送管理模块、第三方推送服务调用模块。

通过本章，详细的介绍了系统的设计与实现工作内容，对关键部分进行详细说明，系统在项目截止时间前上线，并首先提供给普通教育云平台使用，目前公司有其它两个项目组也在使用本系统进行消息推送。

第六章 系统测试

本章将介绍与测试相关的内容，首先会通过对比实验，将 MongoDB 与传统关系型数据库进行定制化时间性能对比，并给出实验结果及总结。然后会对系统测试进行介绍。

6.1 MongoDB 与 MySQL 多租户下性能试验

在完成数据库设计的基础上，论证 MongoDB 在多租户条件下时间性能上的优势，将针对上文中的 MongoDB 与关系型数据库设计模型进行试验设计，通过对比实验，来验证其时间性能上的优势。

6.1.1 实验说明

需要说明的是，第一之所以选择 MySQL 作为对比对象，是因为普教云这个项目目前是建立在 MySQL 之上的，可以使用现有的环境；第二时间性能试验的 MongoDB 只安装在一台机器上，没有做集群，没有引入其高可扩展性和高可用性，所以做简单测试的读写性能是最高的；第三，已经有大量的实验验证过 MongoDB 其优于关系型数据库的读写性能，所以本文只针对多租户模型进行性能实验；第四，对于定制化模式，根据上文分析，只会对关系型中的 Extension Table 方式与 MongoDB 中的无模式方案进行对比实验。

首先对实验的条件进行简单的描述，实验环境主要就是局域网环境下五台 PC 机，一台作为数据库 Server，其余作为客户端，CPU 都是 2.6GHz*4，RAM 为 8G，因为是局域网环境，网络情况非常好。

软件条件，由于以前普通教育云平台是基于 MySQL，为了性能着想，与推送相关的业务表会在推送成功后删除与之相关的记录，所以本文的实验只能根据需求通过程序生成模拟数据。根据普通教育云平台现在的体量与发展情况，模拟生成 200 个租户的 5000000 条与推送系统相关数据。

本次实验建立在上文的基础之上，模拟推送系统中的定制化消息查询业务，其中各个租户所共有的消息属性为消息主键 row_id、租户标识 tenant_id、用户标识 user_id、消息标题 title、消息内容 content，而定制化相关的属性包括地理位置 location，消息查看时间 click_time、消息反馈 feedback、电话号码 telephone、推送次数 push_times、推送方式 push_type。设计 5 种定制化模式，200 个租户从 5 种模式中随机选取，然后通过程序分别生成 MySQL 关系型数据和 MongoDB 非关系型

数据。

然后在模拟数据集的基础上，对多租户的定制化数据进行查询，然后通过计算出的 QPS(每秒查询量)来对时间性能进行评估。

6.1.2 MySQL 实验设计

本文会采用 Extension Table 的方案进行实验，根据上文的设计，在 MySQL 中的表之间的关系如图 6-1 所示。

三张表中的部分数据展示分别如表 6-1、6-2、6-3 所示。

表 6-1 message 表数据存储结构表

row_id	tenant_id	user_id	title	content
62350	1	12003	请假申请	病假
62349	3	11032	通知公告	放假通知
62348	3	11145	通知公告	放假通知
62347	3	11156	通知公告	放假通知

表 6-2 configuration 表数据存储结构表

config_id	tenant_id	table_name	field_name	type	is_null
24	1	message	click_time	date	1
25	1	message	location	varchar	1
26	2	message	location	varchar	1
27	3	message	feedback	varchar	1

表 6-3 extension 表数据存储结构表

row_id	config_id	ext_value
62347	27	已收到
62348	27	
62350	25	2016/1/4 16:33
62350	24	53°33'E,33°51'N

实验会用两种方式进行查询：查询定制化属性和定制化属性作为查询条件，因为使用 Extension Table 的方案，在 MySQL 中肯定会涉及到级联查询，查询语句的示例如下：

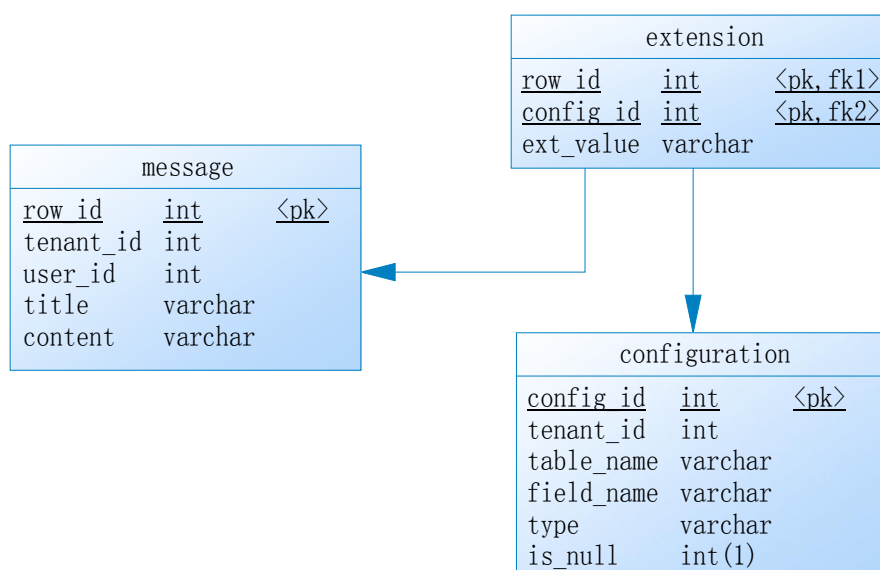


图 6-1 MySQL 实验数据库表之间关联关系图

1、查询定制化属性

```

SELECT  extension.ext_value  AS  feedback
FROM    extension
LEFT JOIN  message
ON  message.row_id = extension.row_id
LEFT JOIN  configuration
ON  configuration.config_id = extension.config_id
WHERE  configuration.table_name = 'message'
AND  configuration.field_name = 'feedback'
AND  message.tenant_id = 2
AND  message.content IS NOT NULL
    
```

2、定制化属性作为查询条件

```

SELECT  message.title, message.content, message.user_id
FROM    extension
LEFT JOIN  message
ON  message.row_id = extension.row_id
LEFT JOIN  configuration
ON  configuration.config_id = extension.config_id
WHERE  configuration.table_name = 'message'
AND  configuration.field_name = 'push_times'
    
```

```

AND message.tenant_id = 3
AND extension.ext_value > 3

```

通过以上的语句可以看出，在 MySQL 中进行定制化查询的语句较为简单，而且模式固定，但是其都会涉及到多表关联操作，会用 LEFT JOIN ON 来进行级联查询，而且不管是查询定制化属性，还是将定制化属性作为查询条件，都会涉及到三张表的级联查询，实验中的 SQL 语句都是与之相似，只是对查询的内容进行改变，比如 filed_name、tenant_id 等。

6.1.3 MongoDB 实验设计

在 MongoDB 中，根据上文的设计，消息的数据都会存放到 message collection 中，不需要扩展表，只需要两元数据 collection，pub_metadata_collection 与 cus_metadata_collection。

表 6-4 是在 pub_metadata_collection 中与 message collection 相关的数据：

表 6-4 pub_metadata_collection 中相关数据表

_id	coll_name	attr_name	type	is_null
56aec1f79b09f814e4a5ee55	message_collection	tenant_id	string	False
56aec2f59b09f814e4a5ee56	message_collection	title	string	False
56aec3059b09f814e4a5ee57	message_collection	content	string	True
56b345d99b09f80dac2efe68	message_collection	user_id	string	False
56b34f8a9b09f80dac2efe6c	message_collection	push_times	string	True

表 6-5 是在 cus_metadata_collection 中与 message collection 相关的部分数据：

表 6-5 cus_metadata_collection 中相关数据表

_id	tenant_id	coll_name	attr_name	type	is_null
56aec72d9b09f814e4a5ee5a	1	message_collection	click_time	date	True
56b34e809b09f80dac2efe6a	1	message_collection	telephone	string	True
56b34eb89b09f80dac2efe6b	1	message_collection	jid	string	True
56b34faf9b09f80dac2efe6d	1	message_collection	push_times	string	True
56b34dca9b09f80dac2efe69	2	message_collection	location	string	True
56b34fc09b09f80dac2efe6e	2	message_collection	click_time	date	True
56b34fc89b09f80dac2efe6f	2	message_collection	push_times	string	True
56aec78b9b09f814e4a5ee5b	3	message_collection	push_times	string	True

当一个租户对一个 `collection` 进行查询操作时，我们首先需要查询其元数据。其语句是十分的简单，假如查询租户一的 `message collection` 元数据，其代码如下。

1、查询共有元数据：

```
db.pub_metadata_collection.find({"coll_name":"message"});
```

2、查询定制化元数据：

```
db.cus_metadata_collection.find({"coll_name":"message", "tenant_id":1});
```

在实际应用中元数据不会每次用户请求都进行数据库查询，而是会在应用程序里肯定会做缓存的，这个步骤中的查询消耗可以忽略。

然后我们就可以通过租户的元素据进行查询定制化属性和定制化属性作为查询条件的查询。下面的例子就是查询语句示例，与 MySQL 中查询的内容相同，查询语句如下：

1、查询定制化属性

```
db.message.find({"content":{"$in":[null],"$exists":true},"tenant_id":2}, {"feedback":1});
```

2、定制化属性作为查询条件

```
db.message.find({"push_times":{"$gt":18},"tenant_id":3}, {"title":1, "content":1, "user_id":1});
```

通过以上查询语句可以看出，在 MongoDB 中进行定制化查询的语句与 MySQL 相比十分的简单，而且简单易懂，更加的灵活。和 MySQL 实验类似，实验中的查询语句都与以上语句结构相似，只需改变查询的属性值，比如 `filed_name`、`tanant_id` 等。

6.1.4 测试程序

在测试程序方面，两个程序都使用 Java 语言，在 MyEclipse 上进行开发，MongoDB 方面，需要导入一个 MongoDB Java Driver 的 jar 包，就可以通过 Java 来实现 MongoDB 的访问，MySQL 的访问就使用 JDBC 来完成，两种测试程序语句都比较简单，只需改变查询参数即可，测试程序会按照上文中的语句进行编写，改变特定的参数进行查询。需要特别说明的，因为四个应用服务器的性能上可能会存在差别，为了让多线程测试程序更加平衡稳定，测试程序会使用 Java 中的线程池技术，线程池的大小会根据机器的可用 CPU 核心数进行设置。

通过以上多线程测试程序模拟多个租户并发访问的环境，会对一次访问的响应时间进行记录，供评估性能使用。当测试程序完成后，会通过完成的事务完成数，以及平均响应时间，计算出 $QPS = \text{成功查询数} / \text{平均响应时间}$ ，通过 QPS 来评估性能。测试程序的事务数分为三个数量级，一万级、十万级、百万级，在上文的基础上，为了更大程度的减小误差，对每个数量级会进行五次实验，然后对相应数量级的五次实验分别取平均值。

6.1.5 实验结果及结论

根据以上实验设计进行实验，通过计算得出结果如图 6-2、6-3 所示。

从实验结果可以得出五个结论：

第一，不管是定制化查询还是普通查询，其 QPS 都大幅度领先 MySQL，MongoDB 的性能优势都十分的明显；

第二，虽然 MongoDB 的 QPS 在图 4-19 与图 4-20 中看着有明显波动，而 MySQL 的 QPS 比较平滑，这是因为 MongoDB 的 QPS 基数大，所以在图中看着波动比 MySQL 的 QPS 波动更大，但是通过计算可知 MongoDB 波动的百分比是很小的，在 10% 以内，而 MySQL 的波动的百分比是很大，会高达 30% 左右，所以 MongoDB 表现更为稳定；

第三，在多租户定制化的环境下，因为 MySQL 每次查询都会关联 Extension 表，必然会涉及到多张表的级联查询，由实验结果可知，随着事务数量级地增加，MySQL 的 QPS 下降趋势十分的明显，反观 MongoDB，随着事务数量级地增加，在实验结果中可以看出 QPS 有一定的波动，但是其波动是只是正常范围的波动，而且不是随着事务数量级的增加而下降的，这也与上文中的分析结果相同；

第四，对于两种查询定制化属性和定制化属性作为查询条件的需求，MongoDB 的性能表现得都相同，而 MySQL 的定制化属性作为查询条件这一方式的性能明显要好于查询定制化属性。

第五，从实验使用的查询语句可以看出，MongoDB 查询语句简单易懂，与 SQL 语句相比，易读性更强，更容易记忆，可以间接的提供程序的开发效率。

从结论可以看出在多租户的环境中，MongoDB 的查询性能比 MySQL 更稳定，更快，进一步验证了 MongoDB 在定制化上的时间性能优势。

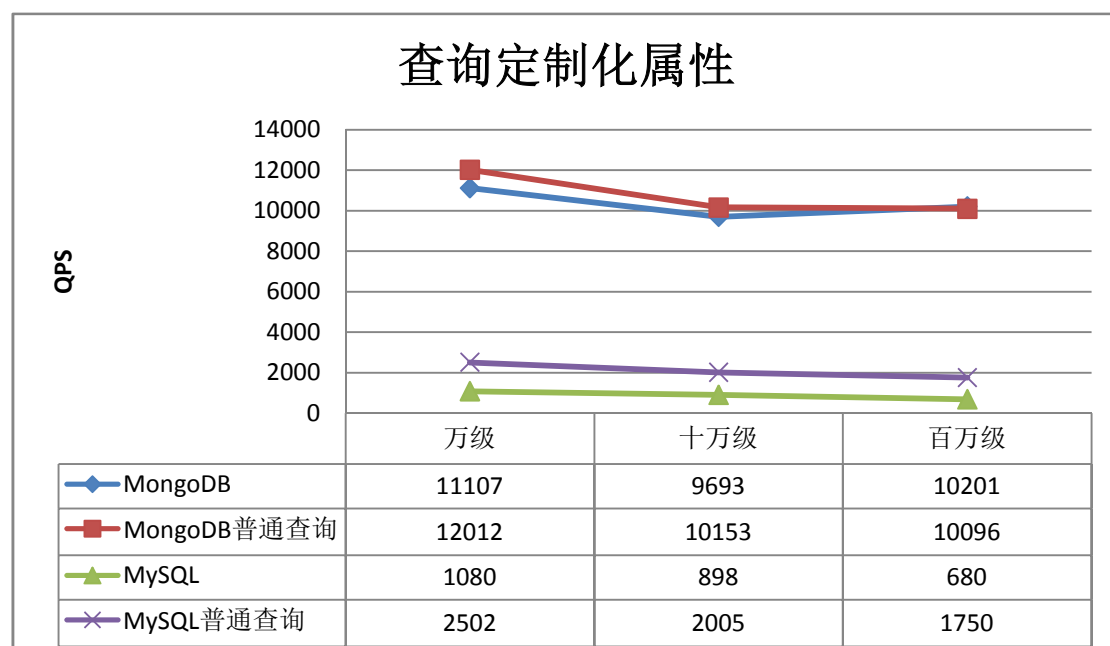


图 6-2 查询定制化数据实验结果图

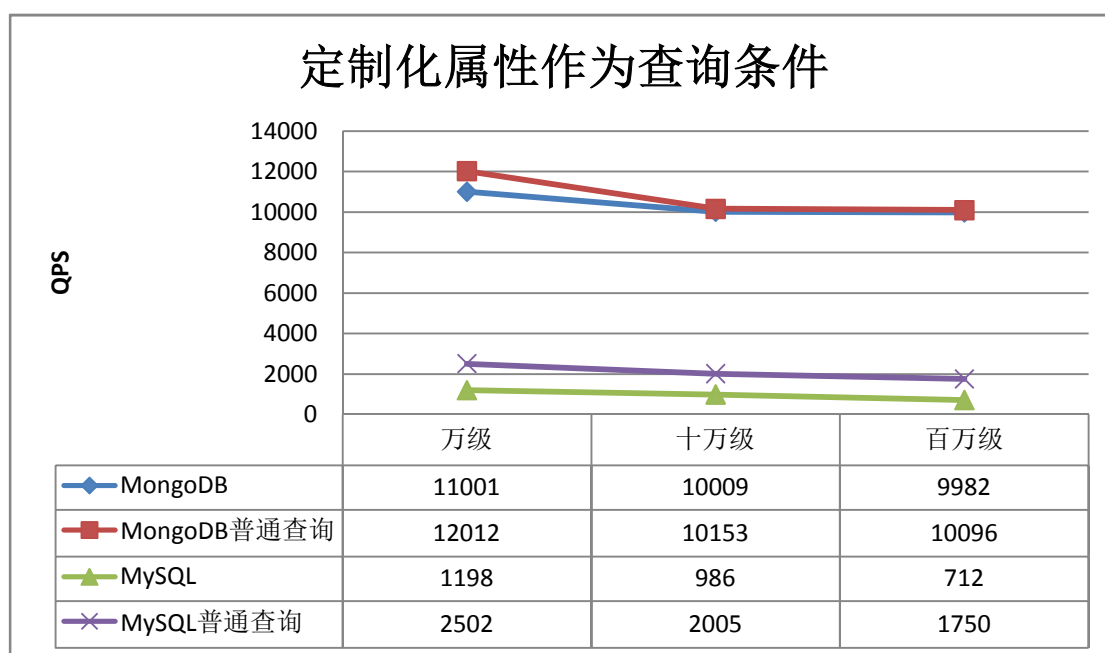


图 6-3 定制化属性作为查询条件结果图

6.2 系统测试

系统测试的工作由所在实习公司的测试人员完成，测试人员根据系统的需求、设计以及用户体验编写测试用例，一共 156 个测试用例。按照公司规定的测试流程进行黑盒测试、回归测试。本人配合测试人员进行 BUG 的修复工作，没有参与

测试工作，所以本文只给出部分关键测试用例与系统整体测试结果报告，不对测试过程以及全部用例做详细介绍。

6.2.1 功能测试

IOS 平台模块关键测试用例如表 6-6 所示

表 6-6 IOS 平台模块关键测试用例表

测试模块	功能点	测试步骤	期待结果	实际结果
IOS 平台模块	推送消息	1、苹果手机安装推送测试应用； 2、根据苹果手机的 deviceToken 新建需要推送的消息“hello world”；	目标用户收到推送消息“hello world”	目标用户收到推送消息“hello world”
IOS 平台模块	反馈信息收集	1、目标手机用户点击推送消息； 2、记录下点击消息的时间；	数据库中的相应消息的 click_time 字段与记录的点击消息时间相同	数据库中的相应消息的 click_time 字段与记录的点击消息时间相同
IOS 平台模块	ApnsServicePool, APNs 连接池测试	1、建立 5 个测试用的租户，每个租户下有一个 IOS 端用户； 2、循环地向五个 IOS 端用户推送消息“hello world”，一共循环 10 次，共 50 次消息推送； 3、查看与 APNs 连接池测试相关的日志，看 ApnsService 新建的次数；	五个租户的 ApnsService 都只会被新建一次	五个租户的 ApnsService 都只被新建一次

Android 平台模块关键测试用例如表 6-7 所示。

表 6-7 Android 平台模块关键测试用例表

测试模块	功能点	测试步骤	期待结果	实际结果
Android 平台 模块	推送消息	1、安卓手机安装推送 测试应用； 2、使用测试账号、测 试租户登录； 3、向测试账号新建推 送消息“hello world”；	测试账号收到推 送消息 “hello world”	测试账号收到推 送消息 “hello world”
Android 平台 模块	反馈信息收集	1、用模拟软件模拟不 同的地理位置； 2、目标手机用户点击 推送消息； 3、记录下点击消息的 时间与模拟的地理位 置；	数据库中的相应 消息的 click_time 与 location 字段与 记录的数据相同	click_time 字段 与记录的点击消 息时间相同， location 字段会 出现不准确情 况，与不同的机 型和 Android 版 本有关
Android 平台 模块	断线重连	1、断开手机网络连 接； 2、一段时间后恢复手 机网络连接；	恢复手机网络连 接后能够收到推 送消息	恢复手机网络连 接后能够收到推 送消息，但是在 清除后台应用后 不能收到推送消 息，需要重新打 开应用
Android 平台 模块	网络切换	1、手机网络连接由移 动网络切换至 wifi； 2、手机网络连接由 wifi 切换至移动网 络；	切换网络后能够 收到推送消息	切换网络后能够 收到推送消息

Web 推送管理模块关键测试用例如表 6-8 所示。

表 6-8 Web 推送管理模块关键测试用例表

测试模块	功能点	测试步骤	期待结果	实际结果
Web 推送管理模块	手动推送消息	1、测试租户的管理员填写消息标题、消息内容，以及选择测试用户； 2、直接点击发送按钮发送推送消息	用户手机收到推送消息	用户手机收到推送消息
Web 推送管理模块	租户定制化推送	1、测试租户的管理员在自定推送页面中以组合的方式勾选两种方式，移动端推送、短信推送，两者可以同时勾选，但不能都不勾选； 2、在不同组合方式下进行手动推送消息；	测试用户在不同的组合方式下收到对应的推送消息	测试用户在不同的组合方式下收到对应的推送消息
Web 推送管理模块	查询报表	1、测试租户的管理员点击查询报表； 2、查看实际数据与报表内容是否一致；	按时间查看柱状图，其消息数总量与实际数量相同；以饼状图的形式查看当前租户移动端情况与实际相同；对于单个用户，可以查看其消息反馈的点击时间的变化趋势与实际相同	按时间查看柱状图，其消息数总量与实际数量相同；以饼状图的形式查看当前租户移动端情况与实际相同；对于消息反馈的点击时间的变化趋势报表与实际结果不相同

短信推送模块关键测试用例如表 6-9 所示。

表 6-9 短信推送模块关键测试用例表

测试模块	功能点	测试步骤	期待结果	实际结果
短信推送模块	短信推送消息	向测试手机号新建推送消息“hello world”	测试手机收到短信“hello world”	测试手机收到短信“hello world”

6.2.2 整体测试

系统整体测试结果报告如表 6-10 所示。

表 6-10 系统整体测试结果表

缺陷状态统计			
项目名称	多租户推送系统	版本号	V1.0
用例总数	156	缺陷总数	27
新建	0	缺陷关闭率	96.29%
已修复	0		
重新开放	0		
已关闭	26		
拒绝	0		
推后处理	1		

通过系统测试报告可以看出，当前系统的 BUG 解决率是超过了百分之 95 的，证明该系统是合格的，可以上线供用户使用。而且截止本论文完成时，该系统已经上线稳定运行 5 个月，并能按照系统的需求向用户提供服务。

在性能方面，经过测试，性能的压力主要来自于 XMPP 持久连接上，现有的阿里云服务器能够支持 60000 个 XMPP 持久连接，通过对现有租户的分析，Android 客户端同时在线用户数最多为 20000，低于 60000 个 XMPP 持久连接数，但是随着系统不同发展，迟早会突破最大连接限制，须要在下一个版本中对其进行提前解决。

6.3 本章小结

本章对测试相关的内容进行了介绍，首先根据设计特性与数据库设计，通过对比实验，将 MongoDB 与传统关系型数据库进行定制化时间性能对比，并给出实验结果及总结，从结论可以看出在多租户的环境中，MongoDB 的查询性能比 MySQL

更稳定，更快，进一步验证了 MongoDB 在定制化上的时间性能优势。然后会对系统功能测试进行介绍，最后通过系统整体测试结果报告以及上线运行结果证明系统是合格的。

第七章 总结与展望

7.1 总结

本文设计并实现了基于 MongoDB 的多租户推送系统,可以向租户提供 Android 平台、IOS 平台,以及手机短信消息推送服务,让用户能够及时地、准确地获取信息,并且向普教云租户提供了 Web 推送管理平台,提供了手动推送、查询报表等实用功能,帮助租户更好地利用推送系统提升办公效率,以及进行决策分析。

首先,本文对选题的背景、依据、研究意义做了介绍,并分析国内外研究现状,以及对相关理论基础与涉及的关键技术进行简要介绍,在此基础上,通过自己的使用、分析对一些现有第三方推送服务进行了很客观、全面的总结,为本文的研究、设计与实现工作做好铺垫。

然后本文主要的研究工作,主要包括以下方面:

1、根据多租户数据层的设计特性,包括数据隔离性、定制化、可扩展性、数据安全,在 MongoDB 上进行研究设计实现。

2、根据需求分析对系统数据库进行设计,并通过对比实验与传统关系型数据库进行定制化时间性能对比,说明其相对于传统关系型数据库的优势,并提供安全访问数据的方案。

3、根据实际项目完成两项优化研究工作,并将《一种基于 MongoDB 的负载均衡优化方法》与《一种基于 MongoDB 的数据分页优化方法》作为专利发表。

4、设计并实现的基于 MongoDB 的多租户消息推送系统,以很简便、轻量级的方式向普教云租户,包括第三方使用者提供 Android 平台、IOS 平台、短信消息推送服务,

5、还为普教云租户提供了 Web 推送管理,而且可以根据租户的不同需求提供定制化服务,包含多种实用功能,例如手动推送、定制化报表、自定义推送方式等。向第三方提供了简单的推送接口,方便他们进行消息推送。

7.2 展望

由于本人的知识面以及精力有限,本论文与本系统可能在细节上有不完善之处,也还有很多理论还有技术需要进一步深入研究,所以在以后的研究工作中,可以从以下几个方面进行拓展:

1、虽然现有的阿里云服务器能够支持 60000 个 XMPP 持久连接,通过对现有

租户的分析，Android 客户端同时在线用户数大约为 20000，低于 60000 个 XMPP 持久连接数，但是随着系统不断发展，迟早会突破最大连接限制。所以可以通过添加服务器的方式进行横向扩展，然后根据用户 `user_id` 的 hash 值进行映射到特定的服务器上，进行负载均衡，使系统能够承受更多的持久连接数，并且提高系统的推送能力。

2、目前系统的定制化服务只包括自定义推送方式、查看定制化报表，在以后的系统中可以提供更多的定制化服务，比如自动推送、定时推送、按群组推送等功能。

3、目前推送的消息只能是文本的形式，而且格式固定，在以后的系统中可以尝试更丰富的推送形式，比如图片、声音、视频等等。

4、目前 Web 推送管理模块只提供给普教云平台的租户使用，没有提供给第三方使用，在未来，可以让其从普教云平台中独立出来，成为一个产品，能够向用户提供更丰富、更全面的服务。

5、对于 XMPP 协议可以进行优化工作，因为现在其消息是基于 XML 文件传输的，其无关冗余信息较多，导致流量消耗较大，可以考虑对其进行优化，使用 JSON 进行消息传输，降低流量消耗。

致 谢

时光荏苒，岁月如梭，素时锦年，稍纵即逝，七年的青春华年，似沙漏般，弹指间，流在昨天。

三年的研究生生涯即将结束，这也是我在电子科技大学的第七个年头，在本论文撰写结束之际，我深知这篇论文的顺利完成不是我一个人的功劳，而是集合了唐雪飞老师悉心的悉心教诲、父母的大力支持、电子科技大学的无私培养、公司同仁的鼎力帮助，没有他们就没有这篇论文。为此，我要向以下这些人致以崇高的敬意和深深的感谢！

唐雪飞老师，黑发积霜织日月，粉笔无言写春秋，蚕丝吐尽春未老，烛泪成灰秋更稠，从我大三保研后认识唐雪飞老师以来，他那严谨的治学态度和广博的学识都深深地感染了我，在他身上我能够真真切切的体会到老成电人“求实求真，大气大为”的优良品质，除此之外，通过四年多的相处，我从唐老师身上学到很多做人做事的原则，我相信这些都是以后工作生活中的宝贵财富。在读研期间的各个阶段，唐老师都给予我很大的关注，也给予了很多指导，从而使我的论文得以顺利完成，请允许我在此致以崇高和衷心的感谢！

我的父母，焉得援草，言树之背，养育之恩，无以回报，我的父母给了我巨大的精神上的支持，在我遇到挫折时，是他们给予我精神力量，让我坚定信念，步步向前，他们的支持是我一生最宝贵的精神财富，请允许我在此致以崇高和衷心的感谢！

电子科技大学，春去秋来，潮起潮落，七年弹指一瞬间，“求实求真，大气大为”谆谆如父语，殷殷似友亲，轻盈数行字，浓抹一生人，我很庆幸人生中最重要七年能够在电子科技大学度过，它教会了我太多东西，无以言表，请允许我在此致以崇高和衷心的感谢！

公司的同仁，在公司实习期间里，公司架构师陈科博士、研发中心的同事在技术上给予了我无私的指导，使我学到了许多书本上学不到的东西，从而为本论文奠定了许多实践基础，也让我了解了职场，了解公司，知道如何做一个称职的员工，请允许我在此致以崇高和衷心的感谢！

最后，我要向百忙之中抽时间对本文进行审阅，评议和参与本人论文答辩的各位老师表示感谢，也谢谢所有帮助过我的人，是你们的支持和关怀让我成长和进步。

参考文献

- [1] Mark de Reu, Guido Ongen, Hany Bouwman. Should mobile Internet be an extension to the fixed web? Fixed-mobile reinforcement as mediator between context of use and future use[J]. Telematics and Informatics, 2012(6):254 - 262.
- [2] 李晓东. 2015 年第 36 次中国互联网络发展状况统计报告[R]. 北京: 中国互联网络信息中心, 2015.
- [3] 王宏江. 多租户模式在基层医疗卫生机构信息化中的应用研究[D]. 成都: 电子科技大学, 2013, 2-6.
- [4] 程昱峰. MongoDB 权威指南[M]. 北京: 人民邮电出版社, 2011, 26 - 36.
- [5] CISCO. WebEx Social[EB/OL]. <http://www.cisco.com/>. 2015-9-5
- [6] 殷昊. 基于 Android 平台的消息推送能力的研究与实现[D]. 北京: 北京邮电大学, 2013, 11-15.
- [7] 顾正敏. 一种面向 Android 平台的轻量级推送技术研究与应用[D]. 北京: 北京大学, 2013, 25-27.
- [8] 科普中国. SaaS 百科词条[EB/OL]. <http://baike.baidu.com/>. 2015-9-15
- [9] Greg Gianforte. Multiple-Tenancy Hosted Applications: The Death and Rebirth of the Software Industry[EB/OL]. RightNow Technologies Inc., www.rightnow.com, 2007.
- [10] 刘盛彬. 数据层的多租户浅谈[EB/OL]. <http://www.ibm.com/>. 2015-9-17
- [11] 孔兰菊. 面向 SaaS 应基于键值对模式的多租户索引研究[J]. 计算机学报, 2010(12):2240-2247.
- [12] S. Aulbach, T. Grust, D. Jacobs, A. Kemper, and J. Rittinger. Multi-tenant databases for software as a service: schema-mapping techniques[J]. In SIGMOD '08: Proceedings of the 2008 ACM SIGMOD international conference on Management of data, 2008(02): 1195-1206.
- [13] Huang Xian-li. NoSQL non-relational database development and application[J]. Fujian PC, 2010(07): 30-45.
- [14] Li Y, Guo L, Guo Y. An Efficient and Performance-Aware Big Data Storage System[J]. Communications in Computer & Information Science, 2013(367):102-116.
- [15] Fay C, Jeffrey D, Sanjay G, et al. Bigtable : A Distributed Storage System for Structured Data[J]. Acm Transactions on Computer Systems, 2008(26):4-26.
- [16] 徐栋. 基于 MongoDB 的 SaaS 数据层多租户关键技术研究与应用[D]. 上海: 华东理工大学, 2012.
- [17] 王光磊. MongoDB 数据库的应用研究和方案优化[J]. 中国信息科技, 2011(20):94-96.

- [18] 樊旭.基于 Mina 的 Android 消息推送系统的研究与实现[D]. 厦门:厦门大学, 2012, 10 - 13.
- [19] Hunkeler, U., IBM Zurich Res.Lab., Zurich,Hong Linh Truong, Stanford-Clark. .MQTT-S:A publish/subscribe protocol for Wireless Sensor Networks[J]. Telematics and Informatics, 2008(6-10 Jan):791 - 798.
- [20] 关庆余. 基于 MQTT 协议的移动社交网络客户端的设计与实现[D]. 北京:中国科学院, 2014, 30-32.
- [21] BradyChen.iPhone 的 Push(推送通知)功能原理浅析[EB/OL].
<http://xiaolife.com/wordpress/an-introduce-to-iphone-push/>. 2014-07-18
- [22] Wei Sun, Zhen Zhang, Wen Hao An. A Study and Performance Evaluation of the Multi-Tenant Data Tier Design Patterns for Service Oriented Computing[C], IEEE International Conference on e-Business Engineering, 2008: 76-78.
- [23] M. Grund, M. Schapranow, J. Krueger, J. Schaffner, and A. Bog. Shared table access pattern analysis for multi-tenant applications[J]. IEEE Computer Society, 2008: 1-5.
- [24] C. D. Weissman, S. Bobrowski. The Design of the Multi-tenant Internet Application Development Platform [J]. SIGMOD, 2010: 34-37.
- [25] 覃左言, 朱青, 李伏. HyDB:集成 MapReduce 和数据库的高效 SaaS 架构[J]. 小型微型计算机系统, 2012, 33(3):512-518.
- [26] M. Grund, M. Schapranow, J. Krueger, J. Schaffner, and A. Bog. Shared table accesspattern analysis for multi-tenant applications[J]. IEEE Computer Society, 2008: 01-05.
- [27] 兰小机, 徐齐行, 敖杰刚. MongoDB 环境下 SaaS 多租户体系模型研究[J]. 测绘通报, 2012(10):21-23.
- [28] 石清超.基于 XMPP 推送的 Android 个性化信息集成系统的研究与实现[D]. 青岛:中国海洋大学, 2013, 17-20.
- [29] 宋新.基于 Android 的 PUSH 平台功能的实现[D]. 北京:北京交通大学, 2011, 35-36.
- [30] 曾冠东. 基于 MINA 构建简单高性能的 NIO 应用[J]. 程序员, 2008(02): 120~123
- [31] selkor. NetworkListener.java 源码解析[EB/OL].
http://www.codeforge.cn/read/237718/NetworkListener.java__html. 2014-08-25
- [32] 浙江 SHEN.解剖 AndroidPN 消息推送中 session [EB/OL].
<http://www.miui.com/forum.php?mod=viewthread&tid=1049140>. 2014-3-14

攻读硕士学位期间取得的成果

发明专利

- [1] 唐雪飞, 吴亚骏 一种 MongoDB 负载均衡优化方法[P]. 中国, 发明专利, ZL201510509279.0, 2015 年 08 月 19 日
- [2] 唐雪飞, 吴亚骏 一种基于 Hadoop 的分布式孤立点数据清洗方法及系统[P]. 中国, 发明专利, ZL201510537346.X, 2015 年 09 月 28 日
- [3] 唐雪飞, 吴亚骏 一种基于 HDFS 小文件存储的优化方法[P]. 中国, 发明专利, ZL201511004711.7, 2015 年 12 月 29 日
- [4] 唐雪飞, 吴亚骏 一种基于 MongoDB 的数据分页优化方法[P]. 中国, 发明专利, ZL201610080695.8, 2016 年 02 月 05 日

参与项目情况

- [1] 成都康赛信息技术公司, 普通教育云平台
- [2] 成都康赛信息技术公司, 普通教育云平台移动版, 项目负责人
- [3] 成都康赛信息技术公司, 多租户消息推送系统, 项目负责人

硕士研究生期间获奖情况

- [1] 研究生一等奖学金
- [2] 优秀研究生



硕士学位论文

MASTER THESIS